GUÍA DIDÁCTICA

Arquitectura de un backend profesional

Capas, modularidad, separación de responsabilidades y principios SOLID

Dirigida a: estudiantes universitarios de Computación y Desarrollo de Software.

Enfoque: explicación progresiva, ejemplos en Python, backend HTTP, decisiones de diseño, ejercicios guiados y autoevaluación.

Requisitos previos: funciones, clases, objetos, conceptos básicos de HTTP y persistencia.

Material educativo · Octubre de 2026

Presentación y resultados de aprendizaje

El backend es la parte de un sistema que ejecuta reglas de negocio, controla el acceso a recursos, gestiona datos y ofrece servicios a clientes web, móviles o a otros sistemas. Su arquitectura define cómo se organizan sus piezas y cómo se relacionan.

Al finalizar esta guía, el estudiante podrá:

- Explicar qué significa arquitectura de software y diferenciarla de tecnologías o carpetas.
- Reconocer las responsabilidades de las capas de presentación, aplicación, dominio e infraestructura.
- Diseñar módulos de un backend con bajo acoplamiento y alta cohesión.
- Aplicar los cinco principios SOLID a ejemplos reales en Python.
- Identificar olores de diseño y refactorizar código mezclado.
- Definir pruebas y criterios verificables para valorar una arquitectura.

Ruta de estudio

Unidad	Contenido y evidencia
1. Fundamentos	Conceptos, atributos de calidad y decisiones de diseño.
2. Capas	Flujo de una petición y reglas de dependencia.
3. Modularidad	Organización por funcionalidades, cohesión, acoplamiento.
4. SOLID	Cinco principios explicados con ejemplos.
5. Taller	Construcción progresiva de un módulo de inscripciones.
6. Evaluación	Rúbrica, ejercicios, autoevaluación y glosario.

1. Conceptos fundamentales

1.1 ¿Qué es la arquitectura de software?

Es el conjunto de decisiones estructurales que organiza componentes, responsabilidades, dependencias, comunicación y restricciones de un sistema. No se reduce a elegir Django, Express o PostgreSQL, ni a dibujar cajas: indica por qué esas piezas están separadas y qué reglas deben respetar.

Analogía: en un edificio, la arquitectura establece distribución, circulación, estructura e instalaciones. Los materiales pueden cambiar; las decisiones de organización determinan cómo se utiliza, adapta y mantiene.

1.2 Calidad arquitectónica

Atributo	Pregunta para el equipo
Mantenibilidad	¿Podemos cambiar una regla sin modificar diez módulos?
Testabilidad	¿Podemos probar reglas sin ejecutar servidor ni base de datos?
Seguridad	¿Dónde se aplica la autorización de cada operación?
Rendimiento	¿Hay consultas, cachés o procesos costosos que deban aislarse?
Escalabilidad	¿Puede aumentar la capacidad sin rediseñar toda la aplicación?
Evolución	¿Podemos sustituir una integración externa con impacto limitado?

Idea clave: una arquitectura es adecuada si resuelve los problemas reales del contexto. Agregar más capas o patrones no la hace automáticamente mejor.

2. Arquitectura por capas

Separar en capas significa agrupar responsabilidades y limitar dependencias. Una propuesta didáctica para un backend HTTP es: presentación, aplicación, dominio e infraestructura.

Capa	Responsabilidad	No debería hacer
Presentación/API	Rutas, parseo, autenticación de entrada, respuestas HTTP.	Consultas SQL y reglas complejas.
Aplicación	Coordinar casos de uso, transacciones, permisos de aplicación.	Conocer detalles HTTP o SQL concretos.
Dominio	Entidades, invariantes y reglas de negocio.	Depender de framework, ORM o servidor.
Infraestructura	Implementar repositorios, envío de correo, servicios externos.	Decidir las reglas fundamentales del negocio.

2.1 Recorrido de una petición

```
POST /api/v1/inscripciones
       |
       v
[API] valida formato del JSON y autentica solicitud
       |
       v
[Aplicacion] ejecutar InscribirEstudiante
       |
       v
[Dominio] comprobar cupos y restricciones
       |
       v
[Puerto repositorio] obtener/guardar informacion
       ^
       |
[Infraestructura] PostgreSQL mediante ORM
       |
       v
[API] respuesta HTTP 201 o error apropiado
```

Este dibujo representa el flujo de ejecución. No debe confundirse con la dirección de las dependencias del código: en un diseño con inversión de dependencias, las políticas internas no importan clases concretas de infraestructura.

2.2 Límites y dependencias

- La API conoce el caso de uso, pero el dominio no debe conocer HTTP ni serializadores.
- La aplicación utiliza contratos (interfaces o protocolos) para persistencia.
- La infraestructura implementa contratos definidos hacia el interior.
- Las entidades y reglas esenciales pueden probarse sin iniciar el framework.

2.3 Un error frecuente: el endpoint gigante

```
# EJEMPLO PROBLEMATICO: todo mezclado en una vista
@app.post('/inscripciones')
def inscribir(request):
    datos = request.json()
    conexion = abrir_bd()
    estudiante = conexion.buscar(datos['estudiante_id'])
    if estudiante.deuda > 0:  # regla de negocio
        return {'error': 'Deuda pendiente'}, 409
    conexion.insertar_inscripcion(datos)  # SQL/acceso a datos
    smtp.enviar(estudiante.email, 'Inscrito')  # integracion
    return {'ok': True}, 201
```

Problema: validar HTTP, consultar datos, ejecutar reglas y enviar correos se han combinado en una sola función. Cualquier cambio puede afectar múltiples comportamientos y dificultar las pruebas.

3. Modularidad y separación de responsabilidades

3.1 ¿Qué es un módulo?

Un módulo es una unidad que reúne comportamientos relacionados y expone una interfaz clara. Puede ser un paquete de Python, una aplicación de Django o un componente mayor. Idealmente, cada módulo responde a un propósito coherente y oculta los detalles que no necesitan conocer otros módulos.

3.2 Cohesión y acoplamiento

Alta cohesión: los elementos de un módulo trabajan en una capacidad de negocio relacionada. Bajo acoplamiento: un módulo depende de pocos detalles internos de otros módulos. Conviene maximizar la primera y limitar el segundo.

Diseño débil	Diseño más sólido
Un archivo utils.py contiene pagos, matrícula, reportes y notificaciones.	Módulos inscripciones, pagos, reportes y notificaciones con límites claros.
El módulo de inscripciones lee tablas privadas de pagos.	Consume una operación/contrato publicado por pagos.
Todas las capas importan el ORM.	Solo adaptadores de persistencia conocen el ORM.

3.3 Organización por funcionalidad

```
backend/
  config/                       # configuracion global
  modules/
    inscripciones/
      api/                      # rutas, serializers
      application/              # casos de uso
      domain/                   # entidades y reglas
      infrastructure/           # ORM, adaptadores
      tests/
    estudiantes/
      api/
      application/
      domain/
      infrastructure/
  shared/                       # solo codigo transversal real
```

Otra organización válida es por capas globales (controllers/, services/, repositories/). Para sistemas medianos y grandes, organizar primero por capacidad de negocio suele reducir la dispersión de cambios. No se debe convertir shared/ en un depósito indiscriminado.

3.4 Encapsulamiento y contratos

Un contrato define qué ofrece un módulo, qué recibe, qué devuelve y qué errores puede producir. Un contrato de repositorio, por ejemplo, puede prometer obtener una inscripción por identificador sin revelar si usa PostgreSQL, un archivo o memoria.

4. Principios SOLID

SOLID agrupa cinco principios de diseño orientado a objetos. Ayudan a gestionar cambios y dependencias. No son reglas mecánicas ni justifican crear una clase por cada línea de código.

4.1 S — Single Responsibility Principle (SRP)

Principio de responsabilidad única: una unidad debería tener una sola razón relevante para cambiar, asociada a una responsabilidad o actor de negocio. No significa que cada clase tenga un solo método.

```
# ANTES: razones de cambio mezcladas
class InformeInscripciones:
    def calcular(self, inscripciones): ...
    def convertir_pdf(self, resultado): ...
    def enviar_correo(self, archivo): ...

# DESPUES: responsabilidades separadas
class CalculadorEstadisticas:
    def calcular(self, inscripciones): ...

class ExportadorPDF:
    def exportar(self, estadisticas): ...

class NotificadorCorreo:
    def enviar(self, destinatario, archivo): ...
```

¿Por qué mejora? Un cambio en el formato del PDF no obliga a modificar las reglas estadísticas. Precaución: el caso de uso puede orquestar estas piezas sin violar SRP.

4.2 O — Open/Closed Principle (OCP)

Principio abierto/cerrado: diseñar puntos de extensión para incorporar variantes sin modificar repetidamente una lógica estable. No implica prohibir cualquier edición del código existente.

```
# ANTES: cada nuevo descuento cambia esta funcion
if tipo == 'regular': descuento = 0
elif tipo == 'becado': descuento = 0.5

# DESPUES: estrategias intercambiables
class DescuentoRegular:
    def calcular(self, monto): return 0

class DescuentoBeca:
    def calcular(self, monto): return monto * 0.5

def total(monto, politica):
    return monto - politica.calcular(monto)
```

Se aplica cuando realmente existen variantes. Para dos casos estáticos, una función sencilla puede ser suficiente; conviene evitar abstracciones prematuras.

4.3 L — Liskov Substitution Principle (LSP)

Principio de sustitución de Liskov: un subtipo debe poder utilizarse donde se espera el tipo base sin romper las garantías de comportamiento que los clientes necesitan. La similitud sintáctica no basta.

```
# MAL DISEÑO: no todos los notificadores soportan SMS
```

```
class Notificador:
    def enviar_sms(self, telefono, texto): ...

class NotificadorCorreo(Notificador):
    def enviar_sms(self, telefono, texto):
        raise NotImplementedError('No soportado')

# MEJOR: contratos alineados con capacidades reales
class NotificadorEmail:
    def enviar_email(self, destino, mensaje): ...

class NotificadorSMS:
    def enviar_sms(self, destino, mensaje): ...
```

La prueba clave no es únicamente que el código compile: también importan las precondiciones, postcondiciones y expectativas del cliente. Si una implementación rompe lo prometido por el contrato, viola LSP.

4.4 I — Interface Segregation Principle (ISP)

Principio de segregación de interfaces: ningún consumidor debería estar obligado a depender de operaciones que no necesita. En Python los contratos pueden representarse con typing.Protocol.

```
from typing import Protocol

class LectorEstudiantes(Protocol):
    def obtener(self, identificador: int): ...

class EscritorEstudiantes(Protocol):
    def guardar(self, estudiante): ...

class ConsultarEstudiante:
    def __init__(self, lector: LectorEstudiantes):
        self.lector = lector
```

El caso de consulta depende solo de lectura y no necesita operaciones para borrar, editar o exportar estudiantes.

4.5 D — Dependency Inversion Principle (DIP)

Principio de inversión de dependencias: la lógica de alto nivel no debería depender directamente de detalles de bajo nivel. Ambas deben apoyarse en abstracciones coherentes con las necesidades del dominio o aplicación.

```
from typing import Protocol

class RepositorioInscripciones(Protocol):
    def guardar(self, inscripcion) -> None: ...

class InscribirEstudiante:
    def __init__(self, repositorio: RepositorioInscripciones):
        self.repositorio = repositorio

    def ejecutar(self, inscripcion):
        self.repositorio.guardar(inscripcion)

# Infraestructura implementa el puerto solicitado
class RepositorioPostgres:
    def guardar(self, inscripcion):
        ...  # persistencia con ORM
```

En producción, el punto de composición crea RepositorioPostgres y lo inyecta en InscribirEstudiante. En las pruebas, puede inyectarse un repositorio en memoria.

Comparación breve de SOLID

Principio	Pregunta para revisar el diseño
SRP	¿Existen motivos de cambio independientes mezclados?
OCP	¿Una nueva variante exige editar condicionales en muchos sitios?
LSP	¿Puedo sustituir una implementación sin sorpresas?
ISP	¿Dependo de métodos que no necesito?
DIP	¿La regla central importa detalles de base de datos o servicios?

5. Taller guiado: módulo de inscripciones

Contexto: una universidad debe inscribir estudiantes a cursos. Una inscripción procede solo cuando hay cupos y el estudiante no está inscrito. Se requiere una API para crear inscripciones.

5.1 Especificación funcional mínima

- Entrada: identificador del estudiante e identificador del curso.
- Regla R1: no admitir inscripciones duplicadas.
- Regla R2: no admitir inscripciones cuando no haya cupos.
- Salida: identificador de inscripción o un error de negocio.
- Persistencia: guardar la inscripción de manera transaccional.

5.2 Paso 1. Definir el dominio

```
from dataclasses import dataclass

class SinCupos(Exception): pass
class YaInscrito(Exception): pass

@dataclass(frozen=True)
class SolicitudInscripcion:
    estudiante_id: int
    curso_id: int

@dataclass
class Curso:
    id: int
    cupos_disponibles: int

    def reservar_cupo(self):
        if self.cupos_disponibles <= 0:
            raise SinCupos('No quedan cupos disponibles')
        self.cupos_disponibles -= 1
```

La entidad Curso protege una invariante: los cupos disponibles no pueden reducirse por debajo de cero mediante reservar_cupo. Las excepciones representan errores del negocio, no códigos HTTP.

5.3 Paso 2. Definir los puertos

```
from typing import Protocol

class InscripcionesPort(Protocol):
    def existe(self, estudiante_id: int, curso_id: int) -> bool: ...
    def crear(self, solicitud: SolicitudInscripcion) -> int: ...

class CursosPort(Protocol):
    def obtener(self, curso_id: int) -> Curso: ...
    def guardar(self, curso: Curso) -> None: ...
```

5.4 Paso 3. Implementar el caso de uso

```
class Inscribir:
    def __init__(self, inscripciones: InscripcionesPort,
                 cursos: CursosPort):
        self.inscripciones = inscripciones
        self.cursos = cursos

    def ejecutar(self, solicitud: SolicitudInscripcion) -> int:
        if self.inscripciones.existe(
                solicitud.estudiante_id, solicitud.curso_id):
            raise YaInscrito('El estudiante ya consta inscrito')
        curso = self.cursos.obtener(solicitud.curso_id)
        curso.reservar_cupo()
        self.cursos.guardar(curso)
        return self.inscripciones.crear(solicitud)
```

Advertencia de concurrencia: el código mostrado es didáctico, no una implementación completa para producción. Guardar el curso y crear la inscripción debe ocurrir dentro de una misma transacción. Para solicitudes simultáneas, usar bloqueo de filas, actualización condicional u otra estrategia de control de concurrencia; añadir una restricción UNIQUE(estudiante_id, curso_id). La validación previa por sí sola no impide carreras.

5.5 Paso 4. Adaptar la API HTTP

```
# Pseudocodigo independiente del framework
POST /api/v1/inscripciones
Body: {"estudiante_id": 12, "curso_id": 8}

201 Created -> {"data": {"id": 35}}
409 Conflict -> {"error": "YA_INSCRITO"}
409 Conflict -> {"error": "SIN_CUPOS"}
400 Bad Request -> entrada invalida
```

El adaptador API transforma JSON en SolicitudInscripcion, ejecuta el caso de uso y traduce errores del dominio a respuestas HTTP. La aplicación y el dominio no deben importar el framework web.

5.6 Paso 5. Prueba sin base de datos

```
class InscripcionesMemoria:
    def __init__(self): self.registros = []
    def existe(self, estudiante_id, curso_id):
        return (estudiante_id, curso_id) in self.registros
    def crear(self, s):
        self.registros.append((s.estudiante_id, s.curso_id))
        return len(self.registros)

class CursosMemoria:
    def __init__(self): self.cursos = {8: Curso(8, 1)}
    def obtener(self, curso_id): return self.cursos[curso_id]
    def guardar(self, curso): self.cursos[curso.id] = curso

def test_inscripcion_reserva_cupo():
    repo = InscripcionesMemoria()
    cursos = CursosMemoria()
    resultado = Inscribir(repo, cursos).ejecutar(
        SolicitudInscripcion(12, 8))
    assert resultado == 1
    assert cursos.obtener(8).cupos_disponibles == 0
```

Para completar el taller: crear pruebas de cupos agotados, duplicados, entradas inválidas, concurrencia y rollback transaccional sobre una base de datos real.

6. Prácticas y decisiones de arquitectura

6.1 Monolito modular y microservicios

Monolito modular: una aplicación desplegable con límites internos definidos. Microservicios: múltiples servicios desplegables con límites y comunicación distribuida. Los microservicios agregan complejidad de red, consistencia, observabilidad, despliegues y operación; no se justifican únicamente por el tamaño de las carpetas.

Criterio	Monolito modular	Microservicios
Despliegue	Una unidad	Varias unidades independientes
Transacciones	Normalmente más simples	Frecuentemente distribuidas
Operación	Menor carga inicial	Mayor complejidad operativa
Uso educativo	Excelente punto de partida	Útiles tras dominar límites y observabilidad

6.2 Cuándo no agregar otra capa

Si una operación CRUD no tiene reglas complejas, puede bastar una estructura más simple, manteniendo la validación, seguridad y pruebas. Añadir abstracciones solo por moda produce más archivos, navegación y mantenimiento.

6.3 Olores de diseño

- Controlador con cientos de líneas y lógica de negocio.
- Servicios que importan directamente rutas HTTP y serializadores.
- Una carpeta common o utils con responsabilidades inconexas.
- Modelos de base de datos expuestos sin control como contrato público.
- Dependencias circulares entre módulos.
- Tests que requieren iniciar toda la infraestructura para probar una fórmula.
- Interfaces enormes y abstracciones sin variantes ni consumidores reales.

7. Guía de revisión arquitectónica

Criterio	Evidencia observable
Límites de módulos	Árbol del repositorio y descripción de cada responsabilidad.
Reglas de dependencia	Imports o tests arquitectónicos que eviten dependencias prohibidas.
Independencia del dominio	Pruebas del dominio sin servidor ni base de datos.
Contratos claros	Interfaces/Protocol y especificación OpenAPI.
Robustez de datos	Restricciones, transacciones y prueba de concurrencia.
Trazabilidad	Diagrama de componentes y registro breve de decisiones (ADR).
Verificación	Pruebas de éxito, errores de negocio, permisos y persistencia.

7.1 Definition of Done arquitectónica

- Los módulos modificados tienen una responsabilidad declarada.
- La API contiene adaptación HTTP; el dominio conserva sus invariantes.
- No existen dependencias circulares nuevas.
- Las abstracciones propuestas tienen una necesidad demostrable.
- Las reglas de negocio están cubiertas por pruebas automatizadas.
- Operaciones multiescritura y concurrencia tienen garantías de integridad.
- Se actualizan diagrama y ADR si cambia una decisión relevante.

8. Actividades de aprendizaje

Actividad 1. Diagnóstico (individual, 20 minutos)

Examine un controlador que valida JSON, consulta SQL, calcula descuentos y envía notificaciones. Identifique al menos cuatro responsabilidades, dibuje las dependencias actuales y proponga una separación. Producto: diagrama antes/después y argumento de 150-200 palabras.

Actividad 2. Refactorización (equipos, 60-90 minutos)

Implemente el módulo de inscripciones del taller con un framework de su elección. Añada los errores YA_INSCRITO y SIN_CUPOS, una capa de aplicación y repositorios intercambiables. Producto: repositorio, pruebas y README con decisiones.

Actividad 3. Debate de arquitectura (equipos, 20 minutos)

Una plataforma de 500 usuarios debe administrar matrículas, reportes y mensajería. Compare monolito modular y microservicios en despliegue, transacciones, equipo y costos de mantenimiento. Producto: decisión justificada mediante un ADR.

Actividad 4. Evaluación cruzada (parejas, 30 minutos)

Revise el código de otro equipo mediante los cinco principios SOLID. Señale un acierto, un problema real y una mejora priorizada. Evite clasificar como violación de SOLID una preferencia estética sin impacto comprobable.

9. Rúbrica de evaluación sugerida

Criterio	Logrado (2)	En proceso (1)	No logrado (0)
Separación de capas	Responsabilidades y dependencias coherentes.	Mezcla menor o dependencia discutible.	Reglas, HTTP y persistencia mezclados.
Modularidad	Límites claros, cohesión alta.	Algunos módulos demasiado amplios.	Acoplamiento fuerte o circular.
SOLID	Aplicación justificada en cambios reales.	Aplicación parcial o ritualista.	No distingue principios.
Pruebas	Éxito, errores y datos críticos probados.	Pruebas incompletas.	Sin pruebas significativas.
Documentación	Diagrama, contratos y decisiones claras.	Evidencia parcial.	Sin justificación técnica.

Interpretación: máximo 10 puntos; la rúbrica puede adaptarse al contexto de la asignatura. La calidad no se mide únicamente por cantidad de clases o cobertura porcentual.

10. Autoevaluación

- 1. ¿Qué diferencia existe entre flujo de ejecución y dependencia de código?
- 2. ¿Por qué una entidad de dominio no debería conocer la respuesta HTTP 409?
- 3. ¿Qué consecuencia tiene una dependencia circular entre módulos?
- 4. ¿Qué diferencia SRP de “una clase = un método”?
- 5. ¿Qué problema resuelve OCP y qué riesgo tiene aplicarlo prematuramente?
- 6. ¿Por qué un subtipo que arroja NotImplementedError puede violar LSP?
- 7. ¿Cómo se relaciona ISP con los Protocol de Python?
- 8. ¿Cómo probarías el caso de uso sin PostgreSQL?
- 9. ¿Por qué la comprobación de cupos necesita control de concurrencia?
- 10. ¿Cuándo recomendarías un monolito modular?

Respuestas orientativas

1) Ejecución es la secuencia de llamadas; dependencia es qué módulo importa/conoce a otro. 2) HTTP es una decisión de interfaz. 3) Complica cambios, pruebas y evolución. 4) SRP se refiere a razones de cambio. 5) Facilita variantes, pero puede introducir complejidad accidental. 6) No respeta el comportamiento esperado. 7) Permiten contratos pequeños. 8) Mediante dobles en memoria. 9) Dos peticiones podrían reservar el último cupo. 10) Cuando el equipo y operación se benefician de una sola unidad desplegable con límites internos claros.

11. Glosario esencial

Término	Definición breve
Acoplamiento	Grado de dependencia entre componentes.
Cohesión	Relación interna entre responsabilidades de un módulo.
Caso de uso	Operación de aplicación que logra un objetivo del usuario.
Entidad	Objeto con identidad y reglas relevantes del negocio.
Puerto	Contrato utilizado para interactuar con otra capacidad.
Adaptador	Implementación que conecta un puerto con una tecnología.
Invariante	Regla que debe mantenerse verdadera en un estado válido.
Inyección de dependencias	Suministrar colaboraciones desde fuera del componente.
ADR	Registro breve de una decisión arquitectónica y sus razones.

12. Lecturas y referencias

- Robert C. Martin. Clean Architecture: A Craftsman’s Guide to Software Structure and Design. Prentice Hall, 2017.
- Robert C. Martin. Agile Software Development: Principles, Patterns, and Practices. Prentice Hall, 2002.
- Martin Fowler. Patterns of Enterprise Application Architecture. Addison-Wesley, 2002.
- Eric Evans. Domain-Driven Design: Tackling Complexity in the Heart of Software. Addison-Wesley, 2003.
- Len Bass, Paul Clements y Rick Kazman. Software Architecture in Practice. 4.ª ed., Addison-Wesley, 2021.
- Python Software Foundation. typing — Support for type hints: https://docs.python.org/3/library/typing.html
- Django documentation: https://docs.djangoproject.com/
- The Twelve-Factor App: https://12factor.net/

GUÍA DIDÁCTICA

Diseño de APIs profesionales

REST · OpenAPI · Versionado · Contratos · Códigos HTTP · Método QUERY

Dirigido a: estudiantes universitarios de Computación y desarrollo de software.

Propósito: comprender los fundamentos y aplicar buenas prácticas para diseñar, implementar, documentar, probar y evolucionar una API HTTP.

Ejemplo transversal: sistema de gestión académica con recursos estudiantes, asignaturas y matrículas.

Actualización normativa: octubre de 2026. Se incluye el método QUERY del RFC 10008 (junio de 2026).

Ruta de aprendizaje: estudiar conceptos → analizar solicitudes y respuestas → diseñar un contrato → practicar OpenAPI → resolver los casos de evaluación.

1. Resultados de aprendizaje

• Explicar la relación entre HTTP, API, recursos, representaciones y REST.

• Seleccionar métodos HTTP correctos, incluido QUERY, según sus propiedades.

• Diseñar URIs, parámetros, cuerpos de mensajes y códigos de estado coherentes.

• Describir contratos en OpenAPI y establecer criterios de compatibilidad y versionado.

• Probar endpoints y resolver errores habituales de diseño.

2. Conceptos fundamentales

API (Application Programming Interface): contrato que establece operaciones, datos y reglas para que dos componentes se comuniquen. Una API web suele utilizar HTTP, pero API no significa necesariamente REST.

Endpoint: combinación de una dirección y una operación disponible. Ejemplo: GET `/api/v1/estudiantes/42`.

Recurso: entidad conceptual identificable por una URI: estudiante, asignatura o matrícula. Representación: descripción transferida de ese recurso, por ejemplo JSON.

Cliente y servidor: el cliente formula una petición; el servidor interpreta método, URI, encabezados y contenido y genera una respuesta.

```
GET /api/v1/estudiantes/42 HTTP/1.1
Host: api.universidad.example
Accept: application/json

HTTP/1.1 200 OK
Content-Type: application/json

{"id":42,"nombre":"Ana Ruiz","estado":"activo"}
```

Anatomía de la comunicación HTTP

Parte	Ejemplo	Función
Método	GET	Intención de la operación
Ruta	/api/v1/estudiantes/42	Identifica el recurso
Query string	?periodo=2026B	Ajusta la selección o presentación
Headers	Authorization, Accept	Metadatos y preferencias
Body	{"nombre":"Ana"}	Contenido enviado, si aplica
Status	200 OK	Resultado protocolario

3. Arquitectura REST

REST (Representational State Transfer) es un estilo arquitectónico descrito por Roy Fielding. No es un protocolo ni una librería. Una API que utiliza JSON y los verbos HTTP no es automáticamente RESTful.

• Cliente-servidor: responsabilidades diferenciadas.

• Sin estado (stateless): cada solicitud proporciona contexto suficiente; el servidor no debe depender de una sesión conversacional implícita.

• Cacheable: las respuestas indican cuándo pueden reutilizarse.

• Interfaz uniforme: identificación de recursos, manipulación mediante representaciones, mensajes autodescriptivos e hipermedia como motor del estado de la aplicación.

• Sistema en capas: proxies, gateways y servicios intermediarios.

• Código bajo demanda: restricción opcional.

Idea clave: REST favorece la evolución independiente de clientes y servidores mediante una interfaz uniforme. HATEOAS es parte del estilo REST; no toda API HTTP orientada a recursos lo implementa completamente.

4. Diseño de recursos y URI

• Utilizar sustantivos, preferentemente plurales: `/estudiantes`, `/matriculas`.

• Evitar verbos RPC innecesarios en la ruta: preferir `POST /matriculas` frente a `POST /crearMatricula`.

• Modelar relaciones según utilidad: `/estudiantes/42/matriculas`.

• Establecer identificadores estables y formato consistente de nombres.

• No colocar datos secretos en URLs: pueden aparecer en registros, historial y herramientas de monitoreo.

```
GET    /api/v1/estudiantes
GET    /api/v1/estudiantes/42
POST   /api/v1/estudiantes
PATCH  /api/v1/estudiantes/42
DELETE /api/v1/estudiantes/42
```

5. Métodos HTTP: significado y selección

Método	Uso típico	¿Seguro?	¿Idempotente?
GET	Obtener representación	Sí	Sí
HEAD	Obtener encabezados, sin cuerpo de respuesta	Sí	Sí
POST	Procesar contenido / crear recurso	No necesariamente	No necesariamente
PUT	Sustituir estado del recurso	No	Sí
PATCH	Aplicar modificación parcial	No	No necesariamente
DELETE	Eliminar asociación de recurso	No	Sí
OPTIONS	Conocer capacidades de comunicación	Sí	Sí
QUERY	Consultar usando contenido de solicitud	Sí	Sí

Seguro significa que la semántica de la petición no solicita modificar el estado del recurso. Idempotente significa que repetir la misma operación tiene el mismo efecto pretendido sobre el servidor que ejecutarla una vez; no significa respuestas idénticas.

GET, POST, PUT, PATCH y DELETE: ejemplos

```
GET /api/v1/asignaturas/7              # consultar
POST /api/v1/matriculas                # crear inscripción
PUT /api/v1/estudiantes/42            # reemplazar representación
PATCH /api/v1/estudiantes/42          # cambiar un campo
DELETE /api/v1/matriculas/105         # eliminar matrícula
```

Con PUT, el contrato debe precisar qué significa reemplazo completo y qué atributos son administrados por el servidor. Con PATCH, se debe declarar el formato usado (por ejemplo, JSON Merge Patch) y las validaciones de consistencia.

6. Parámetros de consulta (query parameters)

Una query string es la parte de la URI que sigue a ?; no es un método HTTP. Sus parámetros suelen utilizarse con GET para filtrar, ordenar, buscar o paginar.

```
GET /api/v1/estudiantes?estado=activo&ciclo=5&page=2&page_size=20
GET /api/v1/asignaturas?search=software&sort=-creditos
```

Parámetro	Ejemplo	Recomendación
filter	estado=activo	Lista blanca de campos permitidos
sort	-fecha	Definir criterio y desempate
page/page_size	page=2&page_size=20	Límites y total o navegación
cursor	cursor=eyJ...	Adecuado para conjuntos cambiantes

Parámetro	Ejemplo	Recomendación
search	search=arquitectura	Documentar concordancia y normalización

Buenas prácticas: validar tipos y rangos, codificar correctamente caracteres especiales, impedir filtros costosos sin límites y especificar el comportamiento cuando se repite un parámetro.

7. El método HTTP QUERY (RFC 10008)

En junio de 2026, el RFC 10008 estandarizó QUERY como método HTTP de tipo Standards Track. QUERY permite realizar consultas seguras e idempotentes, transportando la definición de la consulta dentro del cuerpo de la solicitud. Es útil cuando el filtro es complejo o grande y no conviene expresarlo en una URL.

```
QUERY /api/v1/estudiantes HTTP/1.1
Host: api.universidad.example
Content-Type: application/json
Accept: application/json

{
  "filtros": {"estado":"activo", "ciclos":[3,4,5]},
  "orden": ["apellido"],
  "limite": 20
}
```

La estructura JSON es una convención de ejemplo de nuestra API: RFC 10008 no impone este esquema. El servidor debe validar el tipo de contenido; si `Content-Type` falta o es incompatible, debe rechazar la petición. Puede anunciar formatos admitidos mediante `Accept-Query`.

Criterio	GET	QUERY	POST para búsquedas
Dónde va el filtro	URI/query string	Cuerpo	Cuerpo
Seguro por semántica	Sí	Sí	No está garantizado
Idempotente	Sí	Sí	No está garantizado
Uso sugerido	Filtros pequeños	Filtros complejos con intención de lectura	Compatibilidad con sistemas sin QUERY

Precaución de interoperabilidad: aunque QUERY está estandarizado, algunos frameworks, gateways, navegadores y herramientas OpenAPI pueden requerir actualizaciones o extensiones para soportarlo. Verificar toda la cadena cliente → proxy → servidor. No asumir que un método estandarizado funciona en cualquier infraestructura.

Cuándo usar QUERY y cuándo no

• Sí: analítica con árboles de filtros, expresiones complejas y operaciones estrictamente de lectura.

• No: crear matrículas, ejecutar pagos o modificar registros. Utilizar POST, PUT, PATCH o DELETE según semántica.

• Si el ecosistema aún no soporta QUERY, documentar un POST de búsqueda explícitamente de solo lectura, sin adjudicarle automáticamente las propiedades HTTP de QUERY.

8. Códigos de estado HTTP

Los códigos representan el resultado protocolario de la petición. No deben sustituirse con un campo JSON `"status": "error"` acompañado sistemáticamente de HTTP 200.

Familia	Significado	Ejemplos
1xx	Informativos	100 Continue
2xx	Éxito	200, 201, 202, 204
3xx	Redirección	301, 302, 304, 307, 308
4xx	Problemas del cliente	400, 401, 403, 404, 405, 409, 415, 422, 429
5xx	Problemas del servidor	500, 502, 503, 504

Códigos más utilizados en una API

Código	Situación didáctica
200 OK	Consulta completada o actualización con representación
201 Created	Recurso creado; preferible incluir Location
202 Accepted	Trabajo aceptado pero pendiente de ejecución
204 No Content	Operación exitosa sin contenido
304 Not Modified	Respuesta condicional: reutilizar representación en caché
400 Bad Request	Sintaxis o datos de petición incorrectos
401 Unauthorized	Faltan credenciales válidas (en la práctica, no autenticado)
403 Forbidden	Cliente identificado, pero sin permiso
404 Not Found	Recurso ausente o no revelado
405 Method Not Allowed	Método reconocido, pero no permitido; incluir Allow
409 Conflict	Conflicto con estado actual: duplicidad o concurrencia
412 Precondition Failed	Falló condición como If-Match / ETag
415 Unsupported Media Type	Content-Type no aceptado
422 Unprocessable Content	Contenido válido en sintaxis, inválido semánticamente
429 Too Many Requests	Límite de solicitudes excedido
500 Internal Server Error	Error inesperado de servidor
503 Service Unavailable	Servicio temporalmente no disponible

Casos frecuentes: respuestas apropiadas

```
POST /api/v1/matriculas
HTTP/1.1 201 Created
Location: /api/v1/matriculas/105
Content-Type: application/json

{"id":105,"estudiante_id":42,"asignatura_id":7}

HTTP/1.1 409 Conflict
Content-Type: application/problem+json

{"type":"https://api.example/problems/matricula-duplicada",
 "title":"Matrícula duplicada", "status":409,
 "detail":"Ya existe una inscripción para esta asignatura."}
```

9. Contratos de API: acuerdos verificables

Un contrato API formaliza cómo se comunican cliente y servidor. Debe especificar operaciones, rutas, método, parámetros, tipos, validaciones, autenticación, esquemas de entrada y salida, posibles errores y semántica de negocio.

• Contrato de sintaxis: campos, tipos, formatos, obligatoriedad, enumeraciones y restricciones.

• Contrato de comportamiento: qué se modifica, cuándo se devuelve cada estado y qué reglas rigen la operación.

• Contrato de evolución: compatibilidad, deprecaciones y garantías de versión.

• Contrato de seguridad: esquemas de autenticación, alcance de permisos y restricciones por recurso.

Ejemplo de contrato de inscripción

Elemento	Definición
Operación	POST /api/v1/matriculas
Entrada	estudiante_id: entero; asignatura_id: entero
Precondición	Estudiante activo; cupo disponible; sin matrícula duplicada
Salida éxito	201 Created; id, estudiante_id, asignatura_id
Errores	400 datos incorrectos; 403 sin permiso; 404 referencia inexistente; 409 cupo/duplicado
Seguridad	Token válido; rol autorizado para matricular

10. OpenAPI: contrato legible por personas y herramientas

OpenAPI es una especificación para describir APIs HTTP en formato estructurado. Puede utilizarse para documentación interactiva, generación de clientes, validación contractual y pruebas. La versión más reciente consultada para esta guía es OpenAPI 3.2.1; el ecosistema conserva herramientas que usan versiones anteriores.

```yaml
openapi: 3.2.1
info:
  title: API Academica
  version: 1.0.0
servers:
  - url: https://api.universidad.example
paths:
  /api/v1/estudiantes/{id}:
    get:
      summary: Consultar un estudiante
      parameters:
        - name: id
          in: path
          required: true
          schema:
            type: integer
      responses:
        "200":
          description: Estudiante encontrado
          content:
            application/json:
              schema:
                $ref: "#/components/schemas/Estudiante"
        "404":
          description: No encontrado
components:
  schemas:
    Estudiante:
      type: object
      required: [id, nombre]
      properties:
        id: {type: integer}
        nombre: {type: string}
```

OpenAPI y QUERY: consultar la compatibilidad de la versión y del visualizador elegido para métodos HTTP adicionales. No inventar una operación `query:` en un contrato si el validador no la reconoce; utilizar extensiones documentadas o una definición complementaria cuando sea necesario.

Diseño primero frente a código primero

Design-first: negociar el contrato OpenAPI antes de implementar. Code-first: generar la descripción desde anotaciones del backend. Ambos son viables; en proyectos académicos es útil la primera estrategia para discutir compatibilidad y criterios de aceptación antes de programar.

11. Versionado y compatibilidad

Versionar permite evolucionar una API sin romper consumidores inesperadamente. Un ejemplo simple es versionar por ruta: `/api/v1`, `/api/v2`. También existen estrategias mediante encabezados o negociación de medios.

Cambio	Habitualmente compatible	Riesgo
Agregar campo opcional en respuesta	Sí, si clientes ignoran desconocidos	Clientes estrictos pueden fallar
Eliminar campo consumido	No	Ruptura de contrato
Cambiar string a integer	No	Incompatibilidad de tipos

Cambio	Habitualmente compatible	Riesgo
Agregar endpoint	Sí	Bajo
Exigir nuevo campo de entrada	No	Solicitudes previas inválidas
Modificar semántica de estado	No	Error de comportamiento

Buenas prácticas: documentar deprecaciones; mantener pruebas de contrato; comunicar fechas de retiro; evitar versiones nuevas por cambios menores; medir consumo de endpoints obsoletos antes de eliminarlos.

12. Paginación, filtros, caché e idempotencia

```
GET /api/v1/matriculas?page=1&page_size=10

HTTP/1.1 200 OK
{
  "data": [{"id":105,"estado":"activa"}],
  "meta": {"page":1,"page_size":10,"total":145},
  "links": {"next":"/api/v1/matriculas?page=2&page_size=10"}
}
```

Caché: utilizar encabezados como ETag, Cache-Control e If-None-Match cuando correspondan. Concurrencia: combinar ETag con If-Match para impedir pérdidas de actualización (412). Idempotencia aplicada: para ciertos POST de creación o pago, una clave de idempotencia puede prevenir duplicados, aunque POST no se vuelve por ello un método idempotente a nivel de especificación.

13. Seguridad y tratamiento de errores

• Utilizar HTTPS y controlar autenticación y autorización por recurso, no solo por endpoint.

• Validar contenido JSON, tamaño de cuerpos, formatos, rangos y expresiones de filtros complejos.

• Usar 401 para falta de autenticación y 403 para permisos insuficientes; evitar filtrar detalles sensibles.

• Emplear mensajes de error consistentes. RFC 9457 define el tipo application/problem+json.

• Aplicar rate limiting y vigilar abuso de búsquedas costosas, incluidos endpoints QUERY.

• Registrar trazas y request-id; nunca registrar tokens ni información personal innecesaria.

14. Taller guiado: API de gestión académica

Escenario: implementar recursos de estudiantes, asignaturas y matrículas. La matrícula se acepta solo si el estudiante está activo, existen cupos y no tiene otra inscripción a la misma asignatura.

Actividad A. Diseño del contrato

• Definir rutas y métodos para listar, consultar, crear y actualizar estudiantes.

• Definir el contrato de creación de matrículas con códigos de éxito y errores esperados.

• Proponer tres parámetros de filtro para la colección de estudiantes.

Actividad B. Comparar GET y QUERY

```
GET /api/v1/estudiantes?estado=activo&ciclo=5

QUERY /api/v1/estudiantes
Content-Type: application/json

{"filtros":{"estado":"activo","ciclos":[3,4,5],
"promedio":{"min":8.5}},"orden":["apellido"]}
```

Justificar por qué ambas solicitudes pretenden ser de lectura, cuál es más simple para filtros pequeños y qué se debe validar antes de desplegar QUERY en producción.

Actividad C. Implementación de ejemplo en Django REST Framework

```
from rest_framework import serializers, viewsets
from .models import Estudiante

class EstudianteSerializer(serializers.ModelSerializer):
    class Meta:
        model = Estudiante
        fields = ["id", "nombre", "estado", "ciclo"]

class EstudianteViewSet(viewsets.ModelViewSet):
    queryset = Estudiante.objects.all()
    serializer_class = EstudianteSerializer

    def get_queryset(self):
        qs = super().get_queryset()
        estado = self.request.query_params.get("estado")
        if estado:
            qs = qs.filter(estado=estado)
        return qs
```

Este fragmento es didáctico y omite autenticación, permisos, validación estricta, paginación y tests. El método HTTP QUERY no se implementa automáticamente por añadir `query_params`: este último accede a parámetros de URI.

Actividad D. Pruebas

• GET de estudiante existente: 200; inexistente: 404.

• POST de matrícula válida: 201 y cabecera Location.

• POST duplicado: 409 (si así lo define el contrato).

• Petición sin token: 401; token sin rol: 403.

• QUERY con Content-Type ausente: respuesta 4xx apropiada.

• Documentar diferencias de comportamiento entre repetir PUT, POST y QUERY.

15. Ejercicios de autoevaluación

1. ¿Por qué REST no equivale simplemente a JSON sobre HTTP?

2. ¿Cuál es la diferencia entre recurso, endpoint y representación?

3. ¿Por qué GET y QUERY se consideran seguros?

4. ¿Qué cambia entre query string y método QUERY?

5. ¿Qué código usarías al crear una matrícula y qué encabezado conviene adjuntar?

6. ¿En qué se distinguen 400, 401, 403, 404, 409 y 422?

7. ¿Por qué PATCH no es necesariamente idempotente?

8. ¿Qué debe incluir un contrato además de rutas y schemas?

9. ¿Cuándo se justificaría una nueva versión mayor de la API?

10. ¿Por qué un endpoint de búsqueda complejo puede necesitar límites de costo?

16. Rúbrica formativa (2 / 1 / 0)

Criterio	2 - Logrado	1 - En proceso	0 - Insuficiente
Semántica HTTP	Métodos adecuados y justificados	1-2 confusiones	Selección incorrecta
Contratos	Entradas, salidas y errores definidos	Definiciones parciales	Sin contrato verificable
OpenAPI	Descripción válida y coherente	Errores menores	No válida / ausente
Versionado	Estrategia y compatibilidad justificadas	Criterios incompletos	Sin estrategia
QUERY	Diferencia correctamente método y parámetros	Reconoce parcialmente	Confunde conceptos
Pruebas	Cubre éxito, error y autorización	Casos incompletos	Sin pruebas

Puntaje máximo: 12 puntos. Es posible ajustar el instrumento a ACD, APE o trabajo autónomo según la actividad.

17. Glosario breve

URI: Identificador de un recurso.

HTTP: Protocolo semántico de intercambio de mensajes web.

REST: Estilo arquitectónico basado en restricciones.

OpenAPI: Descripción estructurada y legible por máquinas de una API HTTP.

Schema: Definición de estructura y restricciones de datos.

Idempotencia: Propiedad de repetición con el mismo efecto previsto.

Método seguro: No solicita alteración del estado del recurso objetivo.

QUERY: Método HTTP seguro e idempotente que transporta una consulta en el cuerpo.

18. Referencias técnicas y lecturas recomendadas

Fielding, R. (2000). Architectural Styles and the Design of Network-based Software Architectures.
https://ics.uci.edu/~fielding/pubs/dissertation/top.htm

RFC 9110. HTTP Semantics.
https://www.rfc-editor.org/rfc/rfc9110.html

RFC 10008 (2026). The HTTP QUERY Method.
https://www.rfc-editor.org/rfc/rfc10008.html

OpenAPI Initiative. OpenAPI Specification 3.2.1.
https://spec.openapis.org/oas/latest.html

RFC 9457. Problem Details for HTTP APIs.
https://www.rfc-editor.org/rfc/rfc9457.html

OWASP. API Security Top 10.
https://owasp.org/API-Security/

Cierre: una buena API no es solo un conjunto de rutas que funcionan. Es un contrato comprensible, interoperable, seguro, consistente y capaz de evolucionar.

SERIE BACKEND PROFESIONAL / GUÍA 17

GitFlow y gestión profesional de ramas

Del primer commit a la publicación de una versión: colaboración, revisión, integración, releases y correcciones urgentes.

> Propósito: aprender a trabajar colaborativamente con Git y GitFlow aplicando un flujo verificable a un backend de matrícula académica, con Django REST Framework, PostgreSQL y pruebas automatizadas.

Resultados de aprendizaje: (1) distinguir los papeles de cada rama; (2) implementar feature, release y hotfix sin perder cambios; (3) utilizar pull requests, revisiones y CI; (4) etiquetar versiones y documentar el historial; (5) resolver conflictos y recuperar errores comunes.

Prerrequisitos: Git instalado; repositorio remoto; terminal; conocimientos básicos de commits y backend. Duración sugerida: 4 a 6 horas de trabajo guiado y 4 a 6 de práctica autónoma.

Ruta del documento

01 Conceptos y motivación · 02 Modelo de ramas · 03 Preparación · 04 Feature · 05 Release · 06 Hotfix · 07 Conflictos · 08 Colaboración y seguridad · 09 CI/CD · 10 Taller · 11 Evaluación y glosario.

1. Fundamentos: Git, GitHub y GitFlow

Git es un sistema de control de versiones distribuido que registra cambios mediante commits. Una rama es una referencia móvil a una secuencia de commits, no una copia física completa del repositorio. GitHub es un servicio de alojamiento y colaboración que incorpora pull requests (PR), revisiones y automatizaciones. GitFlow es una estrategia de organización de ramas y publicaciones; no es Git ni exige una herramienta adicional.

GitFlow es útil cuando existen versiones planificadas, mantenimiento de producción y varios cambios paralelos. Para servicios desplegados decenas de veces al día, un flujo basado en tronco (trunk-based) puede ser más simple: se debe escoger la estrategia según el contexto y no por costumbre.

Términos clave

Concepto	Explicación / ejemplo
Commit	Unidad registrada de cambio; debe ser coherente y rastreable.
Branch	Línea de trabajo: feature/cupos-matricula.
Merge	Integra los cambios de otra rama.
Pull request	Propuesta revisable para integrar cambios en una rama destino.
Tag	Referencia inmutable por convención para una publicación: v1.2.0.
Release	Conjunto de cambios probado y publicado en una versión.

2. Las ramas del modelo GitFlow

Rama	Origen → destino	Responsabilidad
main	Producción	Historial de versiones publicadas; acceso restringido.
develop	Nace de main	Integración de funcionalidades aprobadas.
feature/*	develop → develop	Nueva funcionalidad aislada.
release/*	develop → main y develop	Preparación, estabilización y publicación.
hotfix/*	main → main y develop	Corrección urgente de producción.
bugfix/*	Según política del equipo	Corrección no urgente; frecuente en release o develop.

> Regla central: nadie desarrolla funcionalidades directamente en main. Las integraciones a main y develop deben pasar revisiones y comprobaciones automáticas. Una rama release que reciba arreglos debe devolverlos a develop.

3. Preparar el repositorio

Ejemplo: repositorio matriculas-api. Antes de comenzar, comprueba identidad y estado; no registres secretos, archivos .env, bases de datos reales ni credenciales.

```
git config --global user.name "Equipo Backend"
git config --global user.email "equipo@example.edu"
git clone https://example.org/organizacion/matriculas-api.git
cd matriculas-api
git status
git switch main
git pull --ff-only origin main
git switch -c develop
git push -u origin develop
```

Si develop ya existe en remoto, usa git fetch origin y git switch --track origin/develop, en vez de crear una rama nueva. Una rama local puede tener cambios pendientes antes de cambiar a otra: ejecuta siempre git status.

Archivos iniciales sugeridos

```
README.md                # finalidad, instalación y pruebas
.gitignore               # .env, __pycache__, .venv, etc.
pyproject.toml           # herramientas y dependencias
.env.example             # nombres de variables, NO secretos
.github/workflows/ci.yml # pruebas y calidad
CONTRIBUTING.md          # ramas, PR, convenciones
CHANGELOG.md             # historial de versiones
```

4. Flujo de una funcionalidad (feature)

Historia de usuario: «Como estudiante, necesito solicitar una matrícula para una asignatura y recibir una respuesta uniforme». La rama parte de develop y vuelve a ella mediante PR.

```
git switch develop
git pull --ff-only origin develop
git switch -c feature/solicitud-matricula
# implementar pruebas, servicio de dominio y endpoint
git add .
git commit -m "feat(matriculas): validar cupos disponibles"
git push -u origin feature/solicitud-matricula
```

Abre un PR: feature/solicitud-matricula → develop. Adjunta descripción, requisitos, evidencia de pruebas y ejemplos del contrato universal. Tras revisión y CI satisfactorio, integra siguiendo la política del equipo y elimina la rama temporal.

Checklist del PR

- La funcionalidad tiene pruebas de éxito, error de negocio y autorización.
- Los endpoints respetan OpenAPI y el contrato success/message/data/errors/meta.

• No contiene secretos; los cambios de base de datos incluyen migraciones revisadas.

• El título del PR explica el valor; el autor no se autoaprueba cuando la política exige revisión independiente.

5. Preparar una versión (release)

Cuando develop contiene el conjunto previsto, crea una rama release. Allí se permiten ajustes de estabilización (pruebas, documentación, corrección de defectos y versión), no características grandes nuevas.

```
git switch develop
git pull --ff-only origin develop
git switch -c release/1.1.0
git push -u origin release/1.1.0
# estabilizar, probar en staging, actualizar CHANGELOG
# PR 1: release/1.1.0 -> main
# Tras publicar, etiquetar el commit publicado:
git switch main
git pull --ff-only origin main
git tag -a v1.1.0 -m "Publicacion 1.1.0"
git push origin v1.1.0
# PR 2: release/1.1.0 -> develop (sin perder arreglos)
```

> Importante: el tag debe apuntar al commit realmente desplegado. En plataformas que usan squash merge, verifica ese SHA antes de etiquetar. No supongas que el mismo SHA de release termina en main.

6. Corrección urgente (hotfix)

Supongamos que producción permite inscribir estudiantes pese a cupos agotados. El parche debe nacer de la versión publicada, no de develop, porque develop podría contener trabajo incompleto.

```
git switch main
git pull --ff-only origin main
git switch -c hotfix/1.1.1-cupos
# agregar prueba de regresion y corregir el defecto
git add .
git commit -m "fix(matriculas): impedir sobreasignacion"
git push -u origin hotfix/1.1.1-cupos
# PR hacia main + pruebas + despliegue
# Crear tag v1.1.1 en el commit publicado
# Integrar el mismo arreglo en develop (o release activa)
```

Un hotfix no evita CI, revisión ni seguimiento del incidente: estos pasos pueden ejecutarse con una política de aprobación acelerada, pero trazable.

7. Conflictos y recuperación

Un conflicto aparece cuando Git no puede integrar automáticamente modificaciones incompatibles. El procedimiento seguro es identificar archivos, leer ambos cambios, resolver conscientemente, ejecutar pruebas y registrar la solución.

```
git status
git diff --check
# editar el archivo conflictivo, retirar marcadores
# <<<<<<<, ======= y >>>>>>>
git add ruta/al/archivo.py
git merge --continue        # si estabas realizando merge
# O git rebase --continue, si realizabas rebase
# Para cancelar: git merge --abort / git rebase --abort
```

Evita usar git push --force sobre main o develop. Si resulta necesario reescribir una rama personal, prefiere --force-with-lease después de coordinarlo; nunca lo uses como mecanismo rutinario de integración.

Diagnóstico rápido

Situación	Comando o acción
¿Qué cambió?	git diff y git log --oneline --graph --decorate --all
¿Qué está preparado?	git diff --staged
¿Hay conflictos?	git status
¿Deshacer cambios sin publicar?	git restore (con cuidado de no perder trabajo)
¿Revertir un commit publicado?	git revert <sha> y un PR

8. Colaboración, calidad y seguridad

Definir una convención explícita evita que las ramas y commits pierdan significado. Un ejemplo es feature/123-validar-prerrequisitos, release/1.2.0 y hotfix/1.2.1-login. Los commits pueden adoptar Conventional Commits, como feat, fix, docs, refactor y test, sin convertir esa convención en un requisito de Git.

Política mínima de ramas protegidas

- PR obligatorio y al menos una aprobación distinta del autor para main y develop.
- CI verde: lint, pruebas, migraciones revisadas y análisis de dependencias cuando corresponda.
- Prohibición de pushes directos y force push sobre ramas protegidas.
- CODEOWNERS o revisión especializada para autenticación, migraciones y despliegue.
- Prohibir secretos en Git e invalidar inmediatamente cualquier secreto filtrado.

9. Integración con CI/CD

GitFlow organiza cambios; CI/CD automatiza controles y despliegues. Una política razonable valida los PR hacia develop/main, despliega develop en un entorno de integración, release en staging y publica únicamente versiones aprobadas y etiquetadas.

```yaml
name: CI backend
on:
  pull_request:
    branches: [develop, main]
  push:
    branches: [develop]
jobs:
  tests:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: '3.12'
      - run: python -m pip install -r requirements.txt
      - run: python -m pip install pytest ruff
      - run: ruff check .
      - run: pytest -q
```

Este ejemplo es introductorio: un proyecto Django real suele requerir variables de entorno, base de datos de pruebas y dependencias bloqueadas. En un proyecto productivo, fija las acciones a versiones/SHA aprobados y restringe permisos del token de CI.

10. Laboratorio integrador: API de matrículas

Escenario: equipos de 4 o 5 estudiantes desarrollan un módulo de matrículas con autenticación, consulta de cupos y generación de constancias. Cada equipo mantiene main, develop y al menos dos feature; publica una release y simula un hotfix.

Paso	Actividad / evidencia
1	Crear repositorio, README, develop y política de ramas.
2	Feature A: endpoint para listar asignaturas con paginación.
3	Feature B: solicitud de matrícula; validación de cupos.
4	Dos PR a develop con revisión y pruebas registradas.
5	Release/1.0.0: congelación, pruebas, CHANGELOG, PR a main y tag.
6	Simular falla de cupos y hotfix/1.0.1 desde main.
7	Reintegrar hotfix a develop; demostrar historia de commits.
8	Presentar bitácora, capturas de PR y evidencia del pipeline.

Evidencia de aprendizaje

Entregar la URL del repositorio, árbol de ramas, dos PR revisados, ejecución de CI, etiquetas v1.0.0 y v1.0.1, CHANGELOG y un informe de una página donde se explique qué habría ocurrido si el hotfix se hubiera creado desde develop.

11. Evaluación: rúbrica de 2 / 1 / 0

Criterio	2: logrado	1: parcial	0: no logrado
Modelo de ramas	Uso correcto de cinco ramas	Un flujo incorrecto	No distingue ramas
Features + PR	PR revisados y CI verde	Revisión incompleta	Cambios directos a main
Release + tag	Versión y merge-back correctos	Falta sincronización	Sin versión verificable
Hotfix	Desde main y retorna a develop	Retorno parcial	Parche mal integrado
Evidencia	Historia y bitácora claras	Evidencia incompleta	Sin trazabilidad

Interpretación: puntaje máximo 10. Sugerencia didáctica: 9–10 dominio alto, 7–8 satisfactorio, 5–6 en desarrollo, 0–4 requiere refuerzo. La escala puede adaptarse a los criterios de la asignatura.

12. Preguntas de autoevaluación

• ¿Por qué una feature nace de develop y no de main?

• ¿Por qué un hotfix empieza en main y debe regresar a develop?

• ¿Qué diferencia existe entre commit, pull request y tag?

• ¿Qué problema genera publicar un tag sin comprobar el commit desplegado?

• ¿Cuándo sería preferible trunk-based development frente a GitFlow?

• ¿Qué controles impiden publicar inadvertidamente secretos?

13. Glosario y referencias

Merge-back: reincorporar a otra rama los arreglos hechos durante una publicación. SHA: identificador de un objeto/commit. Branch protection: reglas que limitan operaciones sobre ramas. CI: integración continua. CD: entrega o despliegue continuo, según contexto.

Referencias recomendadas:
• Atlassian, Gitflow Workflow: https://www.atlassian.com/git/tutorials/comparing-workflows/gitflow-workflow
• Git, documentación oficial: https://git-scm.com/docs
• GitHub Docs, pull requests y protección de ramas: https://docs.github.com
• Vincent Driessen, A successful Git branching model: https://nvie.com/posts/a-successful-git-branching-model/
• Conventional Commits: https://www.conventionalcommits.org/

Conclusión: un flujo de ramas es una política de colaboración, no solo una colección de comandos. La calidad del trabajo depende de cambios pequeños, pruebas, revisión, trazabilidad y publicaciones reproducibles.

SERIE BACKEND PROFESIONAL · GUÍA 16

Modelado de Arquitectura con C4

Contexto, Contenedores, Componentes y Código

Guía didáctica universitaria con un caso transversal de matrícula académica, diagramas conceptuales, especificaciones Structurizr DSL, PlantUML, actividades y rúbrica.

Destinatarios	Tecnologías	Producto esperado
Estudiantes de Computación; equipos de desarrollo	Django REST Framework, PostgreSQL, Angular, Structurizr DSL, PlantUML	Paquete arquitectónico trazable: C1, C2, C3, C4 selectivo, ADR y evidencias de revisión

Propósito de aprendizaje: que el estudiante sea capaz de representar un sistema desde diferentes niveles de abstracción, justificar los límites arquitectónicos y mantener los diagramas sincronizados con la implementación.

Conocimientos previos: HTTP/REST, entidades y relaciones, modularidad, separación por capas y fundamentos de UML.

1. Resultados de aprendizaje y ruta

- Distinguir sistemas, contenedores, componentes y elementos de código sin confundir sus fronteras.
- Elaborar C1–C3 mediante un modelo único reutilizable en Structurizr DSL y C4 selectivamente con UML.
- Explicar relaciones con dirección, verbo y tecnología cuando sea relevante.
- Relacionar decisiones arquitectónicas, código, contratos API y pruebas.

Etapa	Pregunta principal	Producto
C1 · Contexto	¿Quién utiliza el sistema y de qué depende?	Diagrama de contexto
C2 · Contenedores	¿Dónde se ejecutan las responsabilidades?	Diagrama de aplicaciones y datos
C3 · Componentes	¿Qué responsabilidades hay dentro de la API?	Vista interna del backend
C4 · Código	¿Cómo se implementa el componente crítico?	Diagrama UML acotado
Complementos	¿Cómo se comporta y despliega?	Vista dinámica, despliegue y ADR

Regla central: no deben dibujarse todos los detalles en una sola figura. Cada vista responde una pregunta y corresponde a una audiencia. C4 no exige producir los cuatro niveles para todos los sistemas; C4 suele ser opcional. Fuente: c4model.com/diagrams y c4model.com/diagrams/code.

2. Conceptos arquitectónicos fundamentales

2.1 Elementos y límites

- Persona: rol humano que usa el sistema, por ejemplo estudiante o secretario.
- Sistema de software: producto completo bajo estudio; mantiene un límite de responsabilidad.
- Contenedor C4: aplicación o almacén de datos ejecutable/desplegable, no necesariamente Docker.
- Componente: agrupación coherente de funcionalidades tras una interfaz explícita; no se corresponde necesariamente con cada archivo Python.
- Elemento de código: clase, interfaz, módulo, función o estructura concreta.

2.2 Relaciones

Una flecha se interpreta desde el consumidor hacia el proveedor. Las relaciones deben describir una interacción observable, como «consulta matrículas vía HTTPS/JSON». Evitar flechas ambiguas con etiquetas como «usa» cuando se conoce la interacción.

2.3 Abstracción y consistencia

Una relación entre dos elementos de alto nivel debe ser compatible con las relaciones detalladas en los niveles inferiores. Ejemplo: si C2 dice que API → Base de datos, C3 no debería mostrar que el navegador accede directamente a PostgreSQL sin justificación.

3. Caso transversal: sistema de matrícula académica

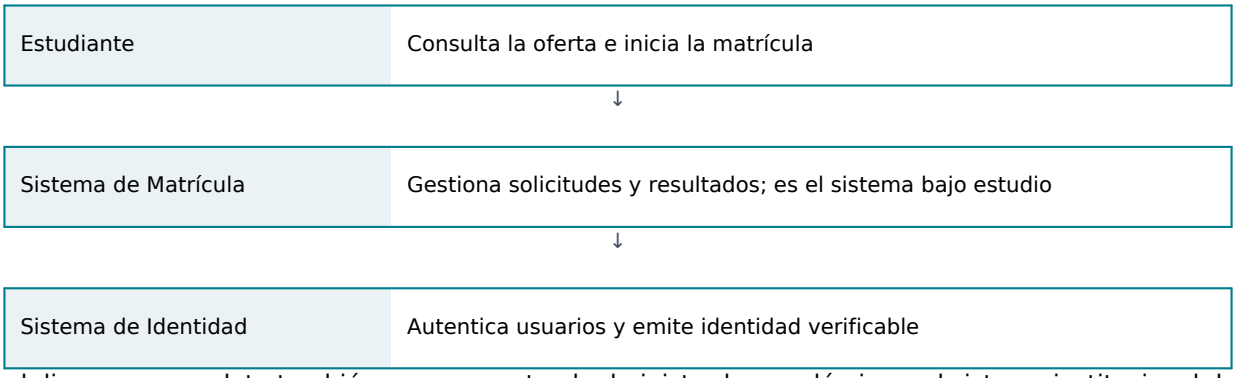
El sistema permite consultar asignaturas ofertadas, solicitar matrícula, verificar prerrequisitos, reservar cupos y emitir una constancia. Los estudiantes se autentican con un proveedor institucional de identidad. Los administradores supervisan procesos y reportes.

Regla / requisito	Implicación arquitectónica
Solo el estudiante autorizado puede consultar sus matrículas	Autenticación y permisos a nivel de objeto
No se puede superar el cupo de una asignatura	Transacciones y control de concurrencia
Una matrícula aprobada genera constancia	Tarea asíncrona e identificador de trabajo
Todas las respuestas JSON respetan el contrato universal	Adaptadores HTTP y manejo global de errores
Cambios de estado quedan auditados	Registro de auditoría y correlación por request_id

4. C1 — Diagrama de contexto del sistema

El C1 expresa el propósito y entorno del sistema, sin tecnologías internas ni endpoints. Su audiencia incluye responsables funcionales, usuarios clave y personal técnico.

Figura 1. Vista didáctica de contexto (lectura simplificada)



En el diagrama completo también se representa al administrador académico y al sistema institucional de oferta, si este es realmente externo. Una conexión entre sistemas debe contar con una responsabilidad explícita.

Ejercicio C1

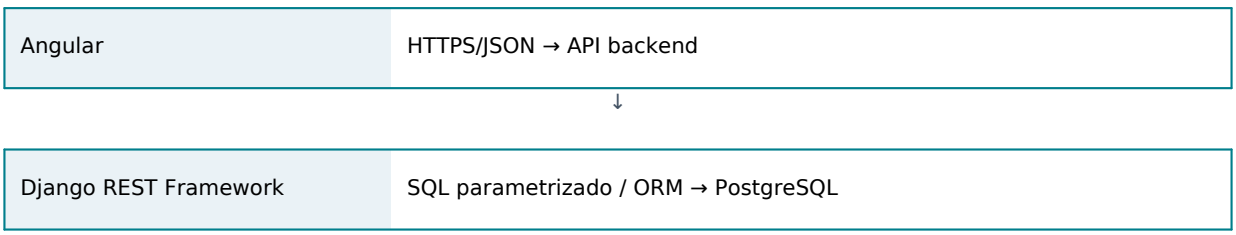
- Delimita el sistema bajo estudio con una oración y enumera tres servicios que ofrece.
- Identifica dos actores humanos y uno o dos sistemas externos reales.
- Traza las relaciones dirigidas con etiquetas que contengan acciones comprensibles.

5. C2 — Diagrama de contenedores

El C2 abre el sistema y muestra las aplicaciones/almacenes que lo integran. Un contenedor C4 puede ejecutarse en infraestructura física, virtual o en un contenedor Docker; son conceptos distintos.

Contenedor	Tecnología	Responsabilidad
Aplicación web	Angular	Experiencia de usuario y consumo de la API
API backend	Django REST Framework	Reglas de negocio, autorización y contrato REST
Base de datos	PostgreSQL	Persistencia transaccional e integridad
Worker de reportes	Celery	Generación diferida de constancias
Broker	Redis	Cola de trabajos y transporte de mensajes

Figura 2. Ruta principal de una solicitud



↓

PostgreSQL	Confirma o revierte la transacción

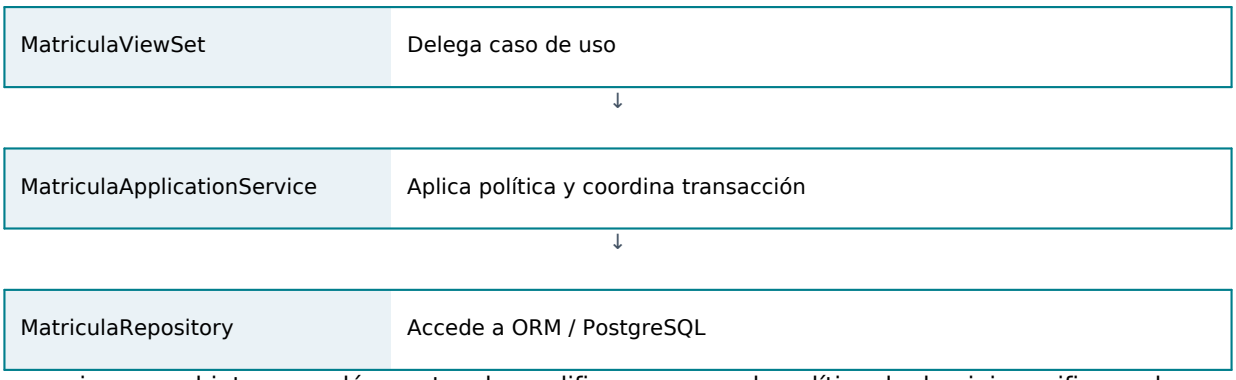
Advertencia: no se añaden tecnologías «porque sí». Si aún no existen workers o broker, se documentan como arquitectura propuesta, diferenciada de la arquitectura actual.

6. C3 — Diagrama de componentes del backend

El C3 profundiza en un contenedor, aquí la API Django. Sus componentes representan responsabilidades y límites lógicos verificables, no una lista exhaustiva de archivos o funciones.

Componente	Responsabilidad	Dependencia principal
Controlador REST	Recibir HTTP, validar transporte, serializar resultado	Servicio de aplicación
Servicio de matrícula	Coordinar caso de uso y transacción	Políticas, repositorio
Política de matrícula	Aplicar prerrequisitos y estados	Entidades de dominio
Repositorio de matrícula	Acceso persistente a datos	ORM / PostgreSQL
Adaptador de respuestas	Producir envelope success/message/data/errors/meta	Controlador y errores
Adaptador de auditoría	Registrar evento y correlación	Almacenamiento de auditoría

Figura 3. Dependencias dentro del contenedor API



Los permisos por objeto se evalúan antes de modificar recursos; la política de dominio verifica reglas propias de la matrícula. Ambos son controles necesarios y diferentes.

7. C4 — Vista de código selectiva

El C4 muestra la implementación de un componente especialmente importante: clases, interfaces, métodos y dependencias. Puede apoyarse en UML o en un diagrama entidad-relación; es opcional y conviene mantenerlo pequeño para evitar desactualización.

Ejemplo: servicios y estados

```
@startuml
skinparam classAttributeIconSize 0
interface MatriculaRepository {
  +obtener(id)
  +guardar(matricula)
}
class MatriculaService {
  +solicitar(estudiante_id, asignatura_id)
  +aprobar(matricula_id)
}
class PoliticaMatricula {
  +validar_prerrequisitos(estudiante, asignatura)
  +validar_cupo(asignatura)
}
class Matricula {
  +estado: EstadoMatricula
  +aprobar()
  +anular()
}
enum EstadoMatricula {
  BORRADOR
  PENDIENTE
  APROBADA
  ANULADA
}
MatriculaService --> MatriculaRepository
MatriculaService --> PoliticaMatricula
MatriculaService --> Matricula
Matricula --> EstadoMatricula
@enduml
```

Lectura: el servicio coordina, la política comprueba las reglas y la entidad protege transiciones válidas. La persistencia se sustituye por una interfaz, lo que simplifica las pruebas unitarias.

8. Structurizr DSL: el modelo como código

Structurizr DSL permite definir un modelo de elementos/relaciones y varias vistas derivadas, evitando redibujar entidades. El siguiente ejemplo es una base autocontenida para construir C1–C3.

```
workspace "Matricula academica" "Ejemplo C4" {
    model {
        estudiante = person "Estudiante" "Solicita matriculas"
        admin = person "Administrador" "Gestiona oferta"
        identidad = softwareSystem "Identidad institucional" "Autenticacion"
        matricula = softwareSystem "Sistema de Matricula" "Sistema central" {
            web = container "Aplicacion web" "UI" "Angular"
            api = container "API" "Servicios REST" "Django REST Framework" {
                controller = component "Controlador REST" "HTTP y envelope"
                servicio = component "Servicio de matricula" "Caso de uso"
                repo = component "Repositorio" "Persistencia"
            }
            db = container "Base de datos" "Informacion" "PostgreSQL" "Database"
        }
        estudiante -> web "Solicita matricula" "HTTPS"
        admin -> web "Administra oferta" "HTTPS"
        web -> api "Invoca API" "HTTPS/JSON"
        api -> identidad "Valida identidad" "OIDC"
        api -> db "Lee y escribe" "SQL"
        controller -> servicio "Ejecuta casos de uso"
        servicio -> repo "Consulta y guarda"
        repo -> db "Accede" "ORM/SQL"
    }
    views {
        systemContext matricula "C1Contexto" {
            include *
            autoLayout lr
        }
        container matricula "C2Contenedores" {
            include *
            autoLayout lr
        }
        component api "C3Backend" {
            include *
            autoLayout lr
        }
    }
}
```

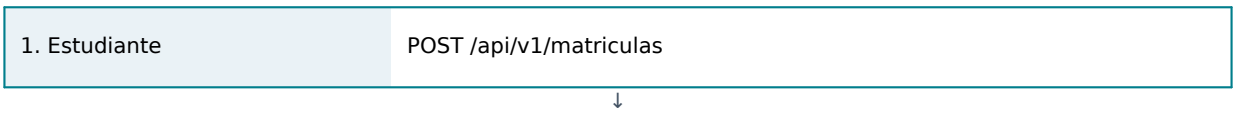
Nota técnica: el bloque anterior sirve como punto de partida para C1–C3. Para un repositorio real, documenta explícitamente las relaciones web/sistema externas y verifica la sintaxis con la versión del parser utilizada; los diagramas deben evolucionar junto con la solución.

9. Diagramas complementarios

9.1 Vista dinámica

Representa el orden temporal de interacciones de un caso de uso: solicitud → validación → transacción → respuesta. Aporta el «cómo ocurre» que no se observa en la vista estática.

Figura 4. Secuencia de solicitud de matrícula



2. API	Autentica, autoriza y valida formato
↓	
3. Dominio	Verifica cupos y prerrequisitos
↓	
4. PostgreSQL	Confirma matrícula de forma atómica
↓	
5. API	Devuelve 201 y contrato universal

9.2 Vista de despliegue

Muestra instancias de contenedores ejecutadas en nodos de infraestructura (servidores, redes, bases de datos gestionadas). No debe confundirse con C2, que refleja estructura lógica de aplicaciones.

10. Alineación de C4 con contratos, código y ADR

El diagrama es una explicación del diseño y no reemplaza especificaciones de comportamiento. Relaciona cada componente con requisitos, contratos API, código, pruebas y decisiones.

Artefacto	Ejemplo	Relación con C4
Requisito	RN-07: impedir matrículas con cupos agotados	C3 política de matrícula
Contrato API	POST /api/v1/matriculas; 201/409	C2 API, C3 controlador
Código	services/matriculas.py	C4 servicio
Prueba	test_no_matricular_sin_cupo	C3 política / dominio
ADR	ADR-002: transacciones en PostgreSQL	C2 base de datos y C3 repositorio

Ejemplo de ADR breve

ADR-002: Control transaccional de cupos. Estado: aceptada. Contexto: dos solicitudes simultáneas pueden competir por el último cupo. Decisión: verificar y actualizar capacidad en una transacción, aplicando bloqueo o actualización atómica. Consecuencia: mayor consistencia a cambio de considerar contención y reintentos. Evidencia: prueba concurrente automatizada.

11. Reglas de calidad para diagramas C4

- Cada diagrama tiene título, alcance, audiencia y leyenda cuando es necesaria.
- Los nombres identifican responsabilidades, no solo tecnologías: «API de matrículas», no «Django».
- Las relaciones tienen dirección correcta, verbo y protocolo relevante.
- No se mezclan los niveles: personas y sistemas en C1, aplicaciones/almacenes en C2, componentes en C3.
- Evitar detalles de clases en C2 y detalles de despliegue físico en C1.
- Se identifican fronteras y sistemas externos; no se confunden módulos con aplicaciones independientes.
- No se dibuja C4 para todos los elementos; se justifica su valor.
- El modelo coincide con implementación, contratos, pruebas y ADR; no representa aspiraciones como hechos.

12. Antipatrones explicados

Antipatrón	Por qué perjudica	Corrección
«Caja de tecnologías»	No comunica responsabilidades	Nombrar capacidades y relaciones
Una figura con todo	Mezcla niveles y saturación	Separar C1, C2 y C3
C3 = carpeta por carpeta	Demasiado detalle accidental	Agrupar por responsabilidad
C4 enorme	Queda obsoleto rápidamente	Dibujar solo módulo crítico
Flechas sin etiqueta	Ambigüedad en dependencias	Usar verbos y protocolos

13. Taller integrador guiado

Situación: diseñar un backend de matrículas que gestione cupos, prerequisitos, roles y constancias. Trabajo en equipos de 3 a 5 estudiantes.

Fase A — Análisis (producto: alcance)

- Enumerar cinco requisitos funcionales y cinco atributos de calidad; distinguir condiciones actuales de propuestas.
- Identificar actores, sistemas externos y límites de responsabilidad.

Fase B — C1 y C2 (producto: workspace.dsl)

- Modelar C1 con actores y dependencias externas; explicar tres relaciones.
- Dibujar C2 y justificar por qué cada contenedor se despliega separadamente o no.

Fase C — C3 y C4 (producto: diagramas de detalle)

- Seleccionar el contenedor API y modelar sus componentes principales.
- Escoger un componente crítico y elaborar su diagrama UML con clases y dependencias relevantes.

Fase D — Evidencias (producto: trazabilidad)

- Registrar un ADR, enlazar dos endpoints OpenAPI y definir al menos cuatro pruebas pertinentes.
- Entregar una explicación de cómo la autorización, las transacciones y la auditoría aparecen en la arquitectura.

Fase E — Revisión entre pares

- Otro equipo comprueba legibilidad, direcciones, fronteras, consistencia entre C1–C4 y correspondencia con el código.

14. Matriz de evaluación (escala 2/1/0)

Criterio	2 · Logrado	1 · Parcial	0 · No logrado
C1	Límite, actores y relaciones claros	Alguna omisión	No interpretable
C2	Responsabilidades y tecnologías correctas	Confunde algún límite	Incorrecto
C3	Componentes coherentes y trazables	Acoplamientos sin justificar	Desconectado
C4	Código representado con rigor	Relaciones incompletas	No corresponde
Consistencia	Sin contradicciones entre niveles	Inconsistencias menores	Contradicciones graves
Evidencias	DSL, ADR, API y pruebas vinculados	Una parte falta	Sin evidencias

Puntaje máximo: 12 puntos. Orientación: 11–12 excelente; 9–10 satisfactorio; 7–8 requiere ajustes; 0–6 insuficiente. Ajustar estos rangos a las normas de evaluación de la asignatura.

15. Preguntas de autoevaluación

- ¿Qué diferencia existe entre un contenedor C4 y un contenedor Docker?
- ¿Cuándo conviene omitir el diagrama de código?
- ¿Por qué un componente no es necesariamente una clase o una carpeta?
- ¿Cómo se detecta una contradicción entre los niveles C2 y C3?
- ¿Qué evidencia demostraría que una regla de cupos tiene soporte arquitectónico?
- ¿Qué información aporta una vista dinámica que no ofrece C3?

16. Glosario

Término	Definición breve
Abstracción	Representación intencional que oculta detalles innecesarios.
Contenedor	Aplicación o almacenamiento ejecutable según la terminología C4.
Componente	Conjunto lógico de responsabilidades detrás de una interfaz.
Vista	Selección de elementos y relaciones para una pregunta específica.
ADR	Registro de una decisión de arquitectura, su contexto y consecuencias.
DSL	Lenguaje específico de dominio para describir un modelo.
Límite	Separación explícita entre responsabilidades o sistemas.

17. Referencias y recursos de consulta

- C4 Model. «Diagrams». https://c4model.com/diagrams
- C4 Model. «Code diagram». https://c4model.com/diagrams/code
- Structurizr. «DSL tutorial». https://docs.structurizr.com/dsl/tutorial
- Structurizr. «DSL language reference». https://docs.structurizr.com/dsl/language
- Structurizr. «Dynamic view». https://docs.structurizr.com/dsl/cookbook/dynamic-view/
- PlantUML. «Class diagram». https://plantuml.com/class-diagram

Cierre: un buen modelo C4 es pequeño, coherente y comprobable. Comunica las decisiones de arquitectura y ayuda a identificar responsabilidades; su objetivo no es dibujar la mayor cantidad posible de cajas.

GUÍA DIDÁCTICA 15

Integración de un Backend Profesional

Ruta de construcción integral · Arquitectura, APIs, datos, seguridad, pruebas, calidad, despliegue y operación

Propósito: transformar las 14 guías anteriores en una estrategia de implementación cohesionada, incremental y verificable. Caso transversal: sistema universitario de matrícula académica, implementado con Python, Django REST Framework y PostgreSQL.

Resultados de aprendizaje: (1) planificar dependencias entre componentes; (2) implementar funcionalidades verticales con contratos consistentes; (3) establecer puertas de calidad; (4) desplegar y operar con trazabilidad y recuperación.

1. Principios para integrar los componentes

• Verticalidad: implementar un caso de uso completo, desde la petición HTTP hasta la persistencia y las pruebas, antes de multiplicar endpoints.

• Arquitectura evolutiva: iniciar con monolito modular y extraer servicios únicamente ante una necesidad concreta.

• Seguridad y calidad desde el principio: no posponer controles esenciales hasta el despliegue.

• Contrato único: respuestas de JSON consistentes en operaciones aplicables, con excepciones propias del protocolo HTTP.

• Automatización: cada entrega debe poder instalarse, probarse, inspeccionarse y revertirse.

• Trazabilidad: conectar requisito → diseño → implementación → prueba → despliegue → monitoreo.

2. Secuencia recomendada y dependencias

Etapa	Construir	Condición para avanzar
0. Descubrimiento	Actores, requisitos, reglas de negocio, riesgos y criterios de aceptación	Casos de uso aprobados
1. Arquitectura	Módulos, C4, ADR inicial y convenciones	Responsabilidades y dependencias claras
2. Contratos/datos	OpenAPI, esquemas, modelo ER y migraciones	Interfaces y restricciones revisadas
3. Base operativa	Repositorio, entornos, CI mínima, Docker	Proyecto reproducible
4. Caso vertical	Dominio, servicios, repositorios, endpoints, permisos	Pruebas funcionales y seguridad
5. Endurecimiento	Errores, observabilidad, rendimiento, asíncronos	Objetivos medidos
6. Producción	CD, respaldo, alertas, operación, rollback	Checklist de salida completado

Dependencias: el modelo de datos y la API se diseñan de forma iterativa. No es necesario terminar toda la base de datos antes de iniciar la primera funcionalidad, pero cada cambio debe estar versionado y probado.

3. Etapa 0 — requisitos y especificación (SDD)

• Identificar usuarios y roles: estudiante, personal administrativo y administrador.

• Registrar casos de uso: consultar oferta, solicitar matrícula, confirmar, anular, descargar comprobante.

• Definir invariantes: no duplicar matrícula; no exceder cupo; respetar prerrequisitos; autorizar acciones por rol y objeto.

• Definir atributos de calidad medibles: latencia objetivo, seguridad, disponibilidad, recuperación y auditoría.

• Crear trazabilidad con identificadores de requisitos (RF-001, RN-001, RNF-001).

Artefacto	Ejemplo mínimo
Especificación	RF-003: matricular estudiante en asignatura
Regla	RN-004: no superar capacidad configurada
Escenario	Dado un cupo agotado, al solicitar matrícula, se devuelve 409
Prueba	test_matricula_sin_cupo_devuelve_409
Operación	Métrica de rechazos de matrícula por motivo

4. Etapa 1 — arquitectura y estructura del repositorio

Adoptar separación en presentación/API → aplicación/casos de uso → dominio/reglas → infraestructura/persistencia. Los controladores no deben contener la lógica sustancial de negocio.

```
backend/
  config/                    # settings, urls y configuracion
  apps/
    matriculas/
      api/                   # views, serializers, permissions, urls
      application/           # casos de uso / servicios
      domain/                # entidades, politicas, excepciones
      infrastructure/        # ORM, repositorios, integraciones
      tests/                 # unit, integration, api, security
    asignaturas/
    usuarios/
  docs/                     # specs, adr, C4, guias operativas
  .github/workflows/         # CI y CD
  Dockerfile
  compose.yaml
  pyproject.toml
```

Nota: estructura ilustrativa; Django no exige estas carpetas. Evitar abstraer prematuramente cada modelo detrás de múltiples capas sin valor práctico.

• Documentar C4 nivel 1 (contexto), 2 (contenedores) y 3 (componentes); nivel 4 solo para módulos complejos.

• Registrar ADR-001: monolito modular; ADR-002: PostgreSQL; ADR-003: política de autenticación; ADR-004: envelope universal.

5. Etapa 2 — contratos HTTP y modelo de datos

Contratos antes de implementar

• Diseñar rutas versionadas /api/v1; operaciones GET, POST, PUT, PATCH, DELETE y QUERY si el servidor y los intermediarios lo soportan.

• Definir esquemas request/response, campos, restricciones, paginación y códigos HTTP en OpenAPI.

• Separar query parameters (?page=2) del método HTTP QUERY para consultas con cuerpo; documentar compatibilidad.

• Definir errores estables, por ejemplo: VALIDATION_ERROR, BUSINESS_RULE_VIOLATION y RESOURCE_NOT_FOUND.

Modelo de datos y consistencia

• Entidades de ejemplo: Estudiante, Asignatura, Paralelo, PeriodoAcademico y Matricula.

• Definir FOREIGN KEY, UNIQUE, CHECK, NOT NULL, índices y política de borrado.

• Crear migraciones reversibles cuando sea posible; probar migraciones sobre datos de prueba realistas.

• Planificar transacciones y concurrencia para la asignación del último cupo.

6. Contrato universal de respuesta

```
{
  "success": true,
  "message": "Matricula registrada correctamente.",
  "data": {"id": 501, "estado": "confirmada"},
  "errors": null,
  "meta": {
    "request_id": "req_abc123",
    "timestamp": "2026-10-13T15:20:00Z",
    "version": "v1"
  }
}
```

Usar exactamente las claves success, message, data, errors y meta para cuerpos JSON. En éxito errors = null; en fallo data = null. La paginación reside en meta.pagination. La respuesta HTTP conserva códigos semánticos: 201 para creado, 400/422 para datos inválidos según la política, 403 para prohibido y 409 para conflictos.

Excepciones: HEAD y códigos HTTP 204 y 304 no llevan cuerpo; streaming, descargas binarias y errores emitidos antes de alcanzar la aplicación requieren políticas expresas. Nunca forzar un JSON en un 204.

7. Etapa 3 — base del proyecto y calidad automatizada

• Crear repositorio Git con ramas protegidas, revisión por pull request y convenciones de commit.

• Fijar versiones de dependencias y política de actualizaciones; usar pyproject.toml y lockfile apropiado.

• Separar configuración local, test, staging y producción: DEBUG, DATABASE_URL, ALLOWED_HOSTS, SECRET_KEY, CORS/CSRF.

• Configurar análisis estático (Ruff), formato, pruebas pytest, comprobación de migraciones y escaneo básico de dependencias.

• Construir imagen Docker no privilegiada, sin secretos embebidos, con health checks y .dockerignore.

Puerta mínima de CI

```
pull_request -> dependencias -> lint y analisis -> tests ->
  migraciones -> seguridad dependencias -> build imagen
merge protegido -> artifact versionado -> staging -> aprobacion -> produccion
```

8. Etapa 4 — implementar un caso de uso vertical

La primera historia debe recorrer todas las capas. Ejemplo: POST /api/v1/matriculas crea una matrícula solo si el usuario tiene permiso y existe cupo.

Paso	Responsable	Verificación
1. Parsear/validar	Serializer: estructura y tipos	400/422 documentado
2. Autorizar	Permisos por rol y objeto	403 / 404 según política
3. Ejecutar caso de uso	Servicio aplicación + dominio	Invariantes probadas
4. Transaccionar	ORM/repository + atomic()	Sin cupos negativos

Paso	Responsable	Verificación
5. Responder	Envelope universal + 201	Esquema OpenAPI válido
6. Auditar	Evento de negocio + actor	Registro sin secretos
7. Observar	Logs, métricas y trazas	request_id correlacionable

Ejemplo conceptual de servicio

```
from django.db import transaction

@transaction.atomic
def solicitar_matricula(*, estudiante, paralelo):
    paralelo = (Paralelo.objects
        .select_for_update().get(pk=paralelo.pk))
    validar_periodo(paralelo.periodo)
    validar_prerrequisitos(estudiante, paralelo.asignatura)
    validar_sin_duplicados(estudiante, paralelo)
    validar_cupo(paralelo)
    matricula = Matricula.objects.create(
        estudiante=estudiante, paralelo=paralelo)
    return matricula
```

La implementación debe garantizar invariantes también mediante restricciones de base de datos; los bloqueos mostrados son orientativos y deben probarse con el motor real, no únicamente SQLite.

9. Etapa 5 — seguridad, errores y pruebas

• Aplicar autenticación, autorización por objeto, validación de datos y límites de solicitudes; revisar OWASP API Security Top 10.

• Normalizar errores previstos y no previstos sin filtrar información privada; logs internos pueden tener detalles diagnósticos.

• Probar unidades de dominio, integración con PostgreSQL, contrato API, permisos por objeto, regresiones y concurrencia.

• Usar datos de prueba deterministas, fixtures/factories y separación de bases de datos por entorno.

Escenario	Resultado esperado
Solicitud correcta	201, success=true, data con identificador
Sin autenticación	401, success=false, sin información sensible
Sin permiso sobre matrícula ajena	403 o 404 según política, envelope
Cupo agotado	409, code=BUSINESS_RULE_VIOLATION
Matrícula duplicada	409 y base de datos consistente
Error inesperado	500, INTERNAL_ERROR y request_id correlacionable

10. Etapa 6 — rendimiento y trabajo asíncrono

• Establecer línea base: latencia p50/p95/p99, consultas SQL, throughput y tasa de error.

• Corregir N+1 con select_related/prefetch_related; consultar EXPLAIN ANALYZE antes de agregar índices.

• Incorporar paginación en listados y límites explícitos en filtros; medir sobre datos realistas.

• Añadir caché para lecturas apropiadas, con TTL e invalidación al modificar el recurso.

• Enviar reportes y procesos prolongados a workers (Celery/colas); usar 202 Accepted + job_id y un endpoint de estado.

• Diseñar tareas idempotentes y políticas de reintento con backoff; no ejecutar ciegamente operaciones irreversibles.

11. Etapa 7 — observabilidad y trazabilidad operativa

Centralizar los tres pilares: logs estructurados JSON, métricas y trazas. Propagar request_id y, cuando corresponda, trace_id; no incluir PII, contraseñas, tokens o cuerpos confidenciales.

Indicador	Ejemplo de control
Disponibilidad	Comprobación de /health/live y /health/ready
Errores	Proporción de respuestas 5xx
Latencia	p95 por endpoint y por dependencia
Procesamiento	Duración y tasa de fracaso de trabajos
Negocio	Rechazos por cupo, matrículas confirmadas
Recursos	Conexiones DB, CPU, memoria, cola pendiente

Las alertas deben identificar impacto real y responsables. Definir runbook, umbral, ventana de evaluación, canal de contacto y acción inicial; evitar alertas sin propósito.

12. Etapa 8 — despliegue y continuidad

• Promover artefactos inmutables entre entornos; no reconstruir una versión distinta para producción.

• Ejecutar migraciones compatibles con despliegues graduales; prever estrategia expand-contract para cambios delicados.

• Configurar TLS, secretos externos, límites de recursos, réplicas cuando proceda y health checks.

• Elegir rolling, blue/green o canary según infraestructura y riesgo; disponer de rollback de aplicación.

• Definir RPO y RTO; automatizar backups cifrados fuera del servidor principal y ensayar restauraciones.

• Registrar auditoría de acciones sensibles, política de retención, acceso restringido y trazabilidad de cambios.

Checklist de paso a producción

• ☐ Especificación y ADR actualizados; OpenAPI coherente con implementación.

- □ Pruebas y análisis estático aprobados; seguridad revisada.
- □ Migraciones validadas con datos similares a producción.
- □ Métricas, dashboards, logs, alertas y runbooks operativos.
- □ Backup recuperable y ensayo de restauración documentado.
- □ Procedimiento de rollback, responsables y ventana de despliegue definidos.
- □ Smoke test posterior al despliegue y monitoreo reforzado.

13. Ejemplo de cronograma académico: 16 semanas

Semanas	Enfoque	Entregable
1–2	Requisitos, arquitectura, SDD, C4 y ADR	Especificación + arquitectura
3–4	Modelo datos, OpenAPI y estructura base	Migraciones + contrato + CI
5–7	Casos verticales, dominio y seguridad	API funcional probada
8–9	Errores, contrato universal y pruebas completas	Reporte pruebas y cobertura
10–11	Rendimiento, paginación y jobs	Benchmark + tareas asíncronas
12–13	Observabilidad, documentación y QA	Dashboard + guías técnicas
14–15	Contenedores, CI/CD, respaldo y despliegue	Staging reproducible
16	Demostración, auditoría y entrega	Repositorio, defensa y evidencias

Distribución orientativa y adaptable. La calidad y seguridad se revisan durante todas las semanas, no solamente al final.

14. Matriz de cobertura de las 14 guías

Guía	Integración prioritaria
01 Arquitectura	Etapas 1 y 4
02 API	Etapas 2 y 4
03 Base de datos	Etapas 2 y 4
04 Seguridad	Etapas 4 y 5
05 Reglas de negocio	Etapas 0 y 4
06 Gestión de errores	Etapas 4 y 5
07 Pruebas	Etapas 3 a 8
08 Documentación	Todas las etapas
09 Observabilidad	Etapas 4, 7 y 8
10 Rendimiento	Etapa 6
11 DevOps	Etapas 3 y 8
12 Operación	Etapas 7 y 8
13 Buenas prácticas	Todas las etapas
14 Contrato universal	Etapas 2, 4 y 5

15. Taller integrador: entregables evaluables

- ☐ Repositorio ejecutable con README e instrucciones de instalación.
- ☐ Especificación de requisitos y criterios de aceptación trazables.
- ☐ Diagramas C4, modelo de datos y al menos dos ADR.
- ☐ OpenAPI coherente y endpoints de matrículas con contrato universal.
- ☐ Restricciones, transacciones y control de concurrencia.
- ☐ Pruebas unitarias, integración, API, seguridad y regresión.
- ☐ Pipeline CI y despliegue a staging con contenedores.
- ☐ Logs JSON, trazabilidad, métricas, alertas y job asíncrono.
- ☐ Backup de PostgreSQL, restauración ensayada y runbook.
- ☐ Demostración de escenarios de éxito, error y recuperación.

Rúbrica breve (2 / 1 / 0 por criterio)

Criterio	2: logrado	1: parcial	0: no logrado
Arquitectura y dominio	Capas y reglas verificables	Separación incompleta	Lógica dispersa
API y datos	Contrato y restricciones consistentes	Inconsistencias menores	Sin contrato verificable
Seguridad y pruebas	Permisos y pruebas automáticas	Cobertura incompleta	Vulnerable / sin pruebas
Entrega y operación	CI/CD, métricas, backups probados	Operación incompleta	Sin control operativo

16. Referencias para ampliar

Django REST Framework: https://www.django-rest-framework.org/

Django deployment checklist: https://docs.djangoproject.com/en/stable/howto/deployment/checklist/

OpenAPI Specification: https://spec.openapis.org/oas/latest.html

The C4 model: https://c4model.com/

PostgreSQL documentation: https://www.postgresql.org/docs/current/

OWASP API Security Top 10: https://owasp.org/API-Security/

OpenTelemetry: https://opentelemetry.io/docs/

GitHub Actions: https://docs.github.com/en/actions

Docker documentation: https://docs.docker.com/

RFC 9457 (Problem Details): https://www.rfc-editor.org/rfc/rfc9457

Mensaje de cierre: un backend está integrado cuando sus decisiones de diseño, comportamiento de negocio, contratos, controles, pruebas y procesos operativos se validan conjuntamente. La evidencia más sólida es una funcionalidad desplegada y recuperable, no un conjunto aislado de documentos.

SERIE BACKEND PROFESIONAL

GUÍA DIDÁCTICA 14

Contrato universal de respuestas de API

Estandarización, excepciones, paginación, trazabilidad y pruebas con Django REST Framework

Material para estudiantes de Computación. Caso transversal: sistema de matrícula académica. Nivel: intermedio. Se presupone conocimiento básico de Python, HTTP, REST y Django REST Framework.

Resultados de aprendizaje: al finalizar, el estudiante podrá diseñar un contrato JSON uniforme; implementar respuestas y excepciones normalizadas; mantener paginación y trazabilidad; reconocer excepciones del protocolo HTTP; y verificar el contrato con pruebas automatizadas.

1. ¿Por qué estandarizar las respuestas?

Una API es un contrato entre consumidores y servidor. Cuando cada endpoint retorna estructuras diferentes, el cliente debe programar excepciones ad hoc, aumenta la complejidad y aparecen defectos. Una envoltura universal establece un lenguaje común para frontend, móvil y servicios externos.

Problema: inconsistencias habituales

```
GET /api/v1/estudiantes/1  -> {"id":1,"nombre":"Ana"}
GET /api/v1/asignaturas/    -> [{"id":1,"nombre":"Programacion"}]
POST /api/v1/matriculas/   -> {"error":"Cupo agotado"}
```

Los tres endpoints presentan contratos diferentes. Un cliente no puede obtener siempre el mensaje, los errores o los metadatos mediante una sola estrategia.

Solución: una envoltura común

```
{
  "success": true,
  "message": "Operación realizada correctamente.",
  "data": {},
  "errors": null,
  "meta": {}
}
```

Esta convención es una decisión de diseño propia del proyecto, no un requisito de HTTP o REST. Debe documentarse como norma arquitectónica y versionarse como cualquier contrato público.

2. Campos, tipos e invariantes

Campo	Tipo	Invariante
success	boolean	true para operación exitosa; false para respuesta de error
message	string	Texto claro para humanos, estable en idioma y estilo
data	objeto / lista / null	Payload en éxito; null en error
errors	objeto / lista / null	null en éxito; detalles estructurados en error
meta	objeto	Siempre presente, admite trazabilidad y paginación

Invariantes: no devolver listas u objetos planos en endpoints JSON cubiertos; no combinar success=true con un error HTTP; conservar tipos estables; y publicar toda modificación incompatible como evolución explícita del contrato.

Metadatos recomendados

```
"meta": {
  "request_id": "req_6f8342c1",
  "timestamp": "2026-10-13T16:00:00Z",
  "version": "v1"
}
```

El timestamp debe ser ISO 8601, preferentemente UTC. El identificador de solicitud debe ser único o suficientemente impredecible, asignado por infraestructura confiable o generado en el servidor.

3. Ejemplos de éxito

3.1 Recurso individual: HTTP 200

```
{
  "success": true,
  "message": "Matrícula obtenida correctamente.",
  "data": {"id": 45, "estudiante_id": 12, "estado": "aprobada"},
  "errors": null,
  "meta": {"request_id":"req_101", "timestamp":"2026-10-13T16:00:00Z", "version":"v1"}
}
```

3.2 Creación de recurso: HTTP 201

Al crear una matrícula, la API puede responder 201 Created y agregar una cabecera Location con la URI del recurso creado.

```
HTTP/1.1 201 Created
Location: /api/v1/matriculas/45
Content-Type: application/json

{"success":true,"message":"Matrícula creada.",
 "data":{"id":45,"estado":"pendiente"},"errors":null,
 "meta":{"request_id":"req_102","timestamp":"2026-10-13T16:01:00Z","version":"v1"}}
```

3.3 Listado paginado: HTTP 200

```
{
 "success": true,
 "message": "Matrículas obtenidas correctamente.",
 "data": [{"id":45}, {"id":46}],
 "errors": null,
 "meta": {
   "pagination": {"page":1,"per_page":10,"total_items":25,
                  "total_pages":3,"has_next":true,"has_previous":false},
   "request_id":"req_103", "timestamp":"2026-10-13T16:02:00Z",
   "version":"v1"
 }
}
```

El conteo total puede ser costoso en conjuntos masivos; si se adopta paginación por cursor, la estructura de pagination deberá evolucionar para representar cursores y disponibilidad de páginas sin inventar total_items.

4. Errores uniformes y clasificación

4.1 Validación: HTTP 400

```
{
  "success": false,
  "message": "Error de validación.",
  "data": null,
  "errors": {
    "code": "VALIDATION_ERROR",
    "details": [{"field":"correo","message":"Campo obligatorio.","rule":"required"}]
  },
  "meta": {"request_id":"req_104", "timestamp":"2026-10-13T16:03:00Z", "version":"v1"}
}
```

Según la política institucional, errores de validación podrían mapearse a 400 o 422; el proyecto debe elegir y documentar su convención de manera consistente.

4.2 Catálogo sugerido de errores

Código interno	HTTP habitual	Significado
VALIDATION_ERROR	400 / 422	Datos inválidos
AUTHENTICATION_REQUIRED	401	No autenticado
PERMISSION_DENIED	403	Sin autorización
RESOURCE_NOT_FOUND	404	Recurso inexistente o no visible
METHOD_NOT_ALLOWED	405	Método no admitido
BUSINESS_RULE_VIOLATION	409 / 422	Regla de dominio incumplida
CONFLICT	409	Conflicto de estado o concurrencia
RATE_LIMIT_EXCEEDED	429	Demasiadas solicitudes
INTERNAL_ERROR	500	Error inesperado
SERVICE_UNAVAILABLE	503	Servicio no disponible

Los mensajes destinados a usuarios no deben exponer SQL, rutas de archivos, valores secretos ni stack traces. Los detalles diagnósticos se registran en el sistema de observabilidad y se correlacionan mediante request_id.

5. Coherencia entre HTTP y success

Escenario	Estado HTTP	success
Consulta válida	200	true
Creación	201	true
Proceso aceptado	202	true
Validación incorrecta	400 / 422	false
Sin autenticación	401	false
Sin permisos	403	false
No encontrado	404	false
Conflicto de negocio	409	false
Error interno	500	false

Excepciones imprescindibles: HTTP 204 No Content y 304 Not Modified no llevan cuerpo de contenido; HEAD tampoco envía cuerpo de respuesta. Si se requiere envoltura JSON para una eliminación, puede utilizarse HTTP 200 con cuerpo en lugar de 204, siempre que sea una elección coherente. Descargas binarias, streaming, redirecciones y ciertos errores del proxy requieren

especificaciones por tipo de respuesta.

El contrato universal aplica a las representaciones JSON bajo control del backend; una caída anterior al procesamiento de la API no siempre puede envolverse. Los clientes también deben contemplar respuestas de infraestructura.

6. Diseño de los módulos compartidos

```
backend/
  core/
    responses.py       # constructores success/error
    exceptions.py      # handler global
    pagination.py      # estructura meta.pagination
    middleware.py      # request_id
    tests/             # pruebas del contrato
  matriculas/
    serializers.py
    views.py
    services.py
  config/
    settings.py
```

El contrato vive en un módulo reutilizable; no se replica en cada app. Las vistas resuelven casos de uso, los serializers validan datos y la capa común se ocupa de la representación JSON.

7. Implementación de respuestas en DRF

Ejemplo didáctico: funciones comunes con metadatos explícitos. El código completo debe ajustarse a la arquitectura y seguridad de la aplicación.

```
# core/responses.py
from datetime import datetime, timezone
from rest_framework.response import Response

API_VERSION = "v1"

def build_meta(request=None, **extra):
    meta = {
        "request_id": getattr(request, "request_id", "unknown"),
        "timestamp": datetime.now(timezone.utc).isoformat(),
        "version": API_VERSION,
    }
    meta.update(extra)
    return meta

def ok(data, message, *, request=None, status=200, **meta):
    return Response({
        "success": True, "message": message,
        "data": data, "errors": None,
        "meta": build_meta(request, **meta)
    }, status=status)

def fail(code, details, message, *, request=None, status=400):
    return Response({
        "success": False, "message": message,
        "data": None,
        "errors": {"code": code, "details": details},
        "meta": build_meta(request)
    }, status=status)
```

Ejemplo de vista

```
# matriculas/views.py
from rest_framework.views import APIView
from rest_framework.permissions import IsAuthenticated
from core.responses import ok
```

```
class MiMatriculaView(APIView):
    permission_classes = [IsAuthenticated]

    def get(self, request, pk):
        # La consulta debe filtrar por propietario y permisos.
        matricula = obtener_matricula_autorizada(request.user, pk)
        data = {"id": matricula.id, "estado": matricula.estado}
        return ok(data, "Matrícula obtenida correctamente.",
                  request=request)
```

No retornar manualmente JSON distintos para cada acción. Mantener un helper es práctico, pero un APIView puede devolver Response directamente si ya cumple exactamente el contrato.

8. Handler global de excepciones

DRF permite configurar EXCEPTION_HANDLER. Este handler procesa excepciones lanzadas; no normaliza por sí solo respuestas de error construidas y retornadas directamente. Es preciso combinarlo con convenciones en las vistas, pruebas y, si corresponde, una capa adicional cuidadosamente diseñada.

```
# core/exceptions.py
from rest_framework.views import exception_handler
from rest_framework.exceptions import ValidationError
from core.responses import fail

STATUS_CODES = {
    401: "AUTHENTICATION_REQUIRED",
    403: "PERMISSION_DENIED",
    404: "RESOURCE_NOT_FOUND",
    405: "METHOD_NOT_ALLOWED",
    429: "RATE_LIMIT_EXCEEDED",
}

def api_exception_handler(exc, context):
    response = exception_handler(exc, context)
    if response is None:
        # Dejar al gestor de errores 500 del proyecto registrar
        # y renderizar la respuesta genérica sin filtrar secretos.
        return None
    request = context.get("request")
    if isinstance(exc, ValidationError):
        details = [{"field": str(k), "message": str(v)}
                   for k, v in response.data.items()]
        return fail("VALIDATION_ERROR", details, "Error de validación.",
                    request=request, status=response.status_code)
    code = STATUS_CODES.get(response.status_code, "API_ERROR")
    detail = response.data.get("detail", "Solicitud no válida.")
    return fail(code, [{"message":str(detail)}],
                "No fue posible completar la solicitud.",
                request=request, status=response.status_code)

# config/settings.py
REST_FRAMEWORK = {
  "EXCEPTION_HANDLER": "core.exceptions.api_exception_handler",
  "DEFAULT_PAGINATION_CLASS": "core.pagination.UniversalPagination",
  "PAGE_SIZE": 10,
}
```

Atención: ValidationError puede contener listas o estructuras anidadas; el ejemplo simplifica el caso de campos en un diccionario. En producción se debe implementar una función recursiva para extraer errores anidados. Asimismo, los 500 requieren un handler adicional de Django/proxy que respete la política de seguridad y el tipo JSON negociado.

9. Paginación personalizada

```
# core/pagination.py
from rest_framework.pagination import PageNumberPagination
from core.responses import ok

class UniversalPagination(PageNumberPagination):
    page_size = 10
    page_size_query_param = "per_page"
    max_page_size = 100

    def get_paginated_response(self, data):
        pagination = {
            "page": self.page.number,
            "per_page": self.page.paginator.per_page,
            "total_items": self.page.paginator.count,
            "total_pages": self.page.paginator.num_pages,
            "has_next": self.page.has_next(),
            "has_previous": self.page.has_previous(),
        }
        return ok(data, "Recursos obtenidos correctamente.",
                  request=self.request, pagination=pagination)
```

Las vistas genéricas y ViewSets con paginación utilizarán automáticamente esta clase. Si se programa una APIView pura, será necesario paginar explícitamente. Definir ordenamiento estable para que las páginas sean reproducibles.

10. Trazabilidad: middleware e identificadores

```
# core/middleware.py
import uuid

class RequestIdMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        request.request_id = f"req_{uuid.uuid4().hex[:16]}"
        response = self.get_response(request)
        response["X-Request-ID"] = request.request_id
        return response
```

Instalar el middleware en MIDDLEWARE de Django. En entornos distribuidos, propagar identificadores de correlación solamente desde intermediarios confiables y aplicar límites de formato y tamaño. No registrar contraseñas, tokens de sesión o datos personales innecesarios.

Ciclo de trazabilidad

Solicitud HTTP → asignación request_id → vista/servicio → log estructurado → respuesta meta.request_id → diagnóstico del incidente. Se puede complementar con trace_id y span_id de OpenTelemetry, sin sustituir los códigos de error estables.

11. Definición OpenAPI del contrato

Documentar explícitamente la envoltura. El esquema siguiente es una base simplificada que puede ampliarse mediante oneOf para éxito/error y esquemas concretos por endpoint.

```
components:
  schemas:
    ApiMeta:
      type: object
      required: [request_id, timestamp, version]
      properties:
        request_id: {type: string}
        timestamp: {type: string, format: date-time}
        version: {type: string}
```

```
    ApiEnvelope:
      type: object
      required: [success, message, data, errors, meta]
      properties:
        success: {type: boolean}
        message: {type: string}
        data: {nullable: true}
        errors: {nullable: true}
        meta: {$ref: '#/components/schemas/ApiMeta'}
```

En OpenAPI 3.1/3.2 conviene representar tipos nulos mediante type: [object, "null"] o esquemas combinados, en lugar de depender de nullable (propiedad asociada a 3.0). Esquema mostrado como ilustración compacta: ajustar a la versión efectiva de OpenAPI antes de publicarlo.

12. Pruebas automatizadas del contrato

```
# core/tests/test_contract.py
import pytest

REQUIRED = {"success", "message", "data", "errors", "meta"}

def assert_contract(response):
    body = response.json()
    assert set(body) == REQUIRED
    assert type(body["success"]) is bool
    assert isinstance(body["message"], str)
    assert isinstance(body["meta"], dict)
    for key in ("request_id", "timestamp", "version"):
        assert key in body["meta"]
    if 200 <= response.status_code < 300:
        assert body["success"] is True
        assert body["errors"] is None
    else:
        assert body["success"] is False
        assert body["data"] is None
        assert body["errors"] is not None

@pytest.mark.django_db
def test_recurso_inexistente(api_client, usuario):
    api_client.force_authenticate(user=usuario)
    response = api_client.get('/api/v1/matriculas/999999/')
    assert response.status_code == 404
    assert_contract(response)
    assert response.json()["errors"]["code"] == "RESOURCE_NOT_FOUND"
```

La prueba presupone fixtures y rutas reales del proyecto. Se deben añadir casos para respuestas 200/201/202, errores 400/401/403/409/429/500, listados paginados, HEAD, 204/304 y archivos descargables, según el alcance publicado.

Prueba de paginación

```
def assert_pagination(response):
    body = response.json()
    p = body["meta"]["pagination"]
    assert isinstance(body["data"], list)
    assert p["page"] >= 1
    assert p["per_page"] >= 1
    assert p["total_items"] >= 0
    assert isinstance(p["has_next"], bool)
    assert isinstance(p["has_previous"], bool)
```

13. Laboratorio: API de matrículas

Situación: una API académica devuelve datos con formatos distintos según el módulo. El equipo debe migrarla hacia el contrato universal sin modificar las reglas de matrícula.

• Inspeccionar 5 endpoints existentes y registrar sus formas actuales de éxito/error.

• Implementar core.responses y seleccionar un único formato de errors.details.

• Configurar excepciones centralizadas y diseñar el tratamiento de errores 500.

• Integrar UniversalPagination con page y per_page.

• Agregar request_id, timestamp UTC y version en todas las respuestas JSON.

• Crear al menos 10 pruebas de contrato que cubran casos positivos y negativos.

• Actualizar OpenAPI y documentar excepciones 204, 304, HEAD y descargas.

• Ejecutar pruebas, presentar evidencia y registrar una decisión arquitectónica ADR.

Producto esperado

Repositorio con implementación y tests; especificación OpenAPI; README de instalación; matriz de escenarios; reporte de resultados y ADR-00X “Adopción de contrato universal de API”.

14. Actividades de aprendizaje y evaluación

Actividades

• Análisis: comparar respuestas de tres APIs y detectar inconsistencias.

• Diseño: construir un catálogo de errores para un módulo académico.

• Implementación: desarrollar el handler y la paginación estándar.

• Pruebas: demostrar que una respuesta 404 cumple estructura y código HTTP.

• Reflexión: justificar por qué no se debe envolver una respuesta HTTP 204.

Rúbrica 2 / 1 / 0

Criterio	2: logrado	1: parcial	0: no logrado
Estructura uniforme	Cinco campos correctos	Algunas inconsistencias	Sin estándar
HTTP y errores	Códigos y errores coherentes	Fallas aisladas	Uso incorrecto
Trazabilidad	Metadatos y registros completos	Implementación incompleta	Sin correlación
Paginación	Metadata exacta y verificada	Campos incompletos	No funciona
Pruebas	Escenarios positivos y negativos	Cobertura limitada	Sin evidencias
Documentación	Contrato y excepciones publicados	Documentación parcial	No documenta

Preguntas de autoevaluación

• ¿Por qué success no sustituye el código HTTP?

• ¿Qué diferencias existen entre un error de validación y un conflicto de negocio?

• ¿Qué módulo se encarga de los errores retornados directamente desde una vista?

• ¿Cómo asociar un error mostrado en frontend con un log del servidor?

• ¿En qué respuestas no es posible incluir el envelope JSON?

• ¿Cómo cambia la metadata si se usa paginación por cursor?

15. Lista de verificación de entrega

• [ ] Todos los endpoints JSON contemplados usan success/message/data/errors/meta.

• [ ] success, data y errors cumplen invariantes.

• [ ] Los errores son seguros y tienen código interno estable.

• [ ] Códigos HTTP correctos; excepciones del protocolo documentadas.

• [ ] Los listados incluyen meta.pagination.

• [ ] Las respuestas contienen request_id, timestamp y version.

• [ ] Existen pruebas automáticas para cada familia de respuestas.

• [ ] OpenAPI describe fielmente los ejemplos y esquemas.

• [ ] Observabilidad permite correlacionar errores sin exponer secretos.

16. Glosario y referencias

Envelope: envoltura estándar de una respuesta. Serializer: componente que transforma y valida datos. Handler: función centralizada que procesa excepciones. Request ID: identificador de una solicitud. Contrato: acuerdo verificable entre proveedor y consumidor. Metadata: datos auxiliares de la respuesta.

Referencias recomendadas: Django REST Framework, documentación de excepciones https://www.django-rest-framework.org/api-guide/exceptions/ ; paginación https://www.django-rest-framework.org/api-guide/pagination/ ; especificación HTTP Semantics RFC 9110 https://www.rfc-editor.org/rfc/rfc9110 ; OpenAPI Specification https://spec.openapis.org/oas/latest.html ; OWASP API Security Top 10 https://owasp.org/API-Security/.

Nota metodológica: los fragmentos de implementación son una plantilla de enseñanza, no un paquete listo para instalar. En producción es necesario integrar errores anidados, excepciones fuera de DRF, negociación de contenido, autorización, observabilidad, versionado y pruebas de extremo a extremo.

GUÍA DIDÁCTICA 13

Buenas prácticas de codificación

Código legible, mantenible, seguro y verificable para backend

Destinatarios: estudiantes de Computación y desarrolladores de backend en formación.

Caso transversal: servicio de inscripciones académicas desarrollado en Python, Django y Django REST Framework.

Propósito: aprender a identificar defectos de diseño y calidad, refactorizar código y aplicar herramientas automatizadas.

Resultados de aprendizaje: (1) escribir código expresivo y modular; (2) aplicar prácticas de validación, manejo de errores y seguridad; (3) verificar calidad con pruebas y herramientas; (4) argumentar mejoras en una revisión de código.

Orientación didáctica: cada apartado presenta concepto, criterio, ejemplo y actividad. Los fragmentos son ilustrativos y pueden necesitar adaptación al proyecto y su versión de dependencias.

1. ¿Qué significa escribir buen código?

Una buena implementación no solo produce el resultado esperado: permite comprender la intención, cambiar un requisito con bajo riesgo, detectar errores y colaborar en equipo. La calidad se observa en el comportamiento, la claridad y la capacidad de evolución.

Dimensión	Pregunta de revisión
Corrección	¿Respeta los requisitos y casos límite?
Legibilidad	¿Otro desarrollador puede entender su propósito?
Mantenibilidad	¿Cambiar una regla obliga a editar muchos archivos?
Seguridad	¿Valida entradas y protege información y permisos?
Verificabilidad	¿Se puede probar sin preparar toda la aplicación?
Eficiencia	¿Evita trabajo y consultas innecesarias?

Regla práctica

Optimizar primero la claridad y la corrección; mejorar rendimiento a partir de medidas. “Código limpio” no implica fragmentar cada instrucción ni introducir capas sin una necesidad verificable.

2. Convenciones y nombres expresivos

Mantener un estilo uniforme reduce el esfuerzo de lectura. En Python, PEP 8 recomienda cuatro espacios por nivel, nombres coherentes y consistencia interna; PEP 257 orienta sobre docstrings.

Elemento	Convención habitual	Ejemplo
Módulos y funciones	snake_case	crear_matricula
Variables	snake_case	cupos_disponibles
Clases	PascalCase	MatriculaService
Constantes	UPPER_SNAKE_CASE	MAX_INTENTOS
Booleanos	Expresar condición	esta_habilitado

Antes y después

```
# Poco expresivo
def f(a, b):
    return a - b > 0

# Expresa la intención
def tiene_cupos(capacidad: int, inscritos: int) -> bool:
    return capacidad > inscritos
```

Evitar: nombres como `data`, `tmp` o `process` cuando existe un concepto del dominio más específico. Los nombres de módulos, endpoints y propiedades deben formar un vocabulario común.

3. Funciones pequeñas y responsabilidad única

Una función debería resolver una tarea reconocible. Conviene separar validación, decisiones de negocio y efectos secundarios. El tamaño no es una regla absoluta: una función de 30 líneas clara puede ser mejor que diez funciones indescifrables.

```
# Mezcla validación, dominio y persistencia
def matricular(request):
    if not request.data.get("estudiante_id"):
        raise ValueError("Estudiante requerido")
    # ... valida prerrequisitos, crea registros, envía correo

# Separación conceptual
# Serializer: valida estructura de entrada
# Servicio: verifica elegibilidad y ejecuta caso de uso
# Repositorio/ORM: persiste dentro de una transacción
# Worker: envía notificación después del commit
```

Preguntas para refactorizar

- ¿La función necesita conocer detalles HTTP, SQL y correo al mismo tiempo?
- ¿Es posible darle un nombre preciso sin usar “y”?
- ¿Puede probarse su regla principal sin red ni base de datos?

4. Separación de responsabilidades y dependencias

Los controladores traducen HTTP; los servicios de aplicación orquestan casos de uso; el dominio contiene reglas; la infraestructura integra la base de datos y servicios externos. Evite acceder a `request` desde entidades del dominio o devolver códigos HTTP desde un modelo.

```
# Dominio: no depende de Django ni HTTP
class CuposAgotados(Exception):
    pass

def verificar_cupos(capacidad: int, inscritos: int) -> None:
    if inscritos >= capacidad:
        raise CuposAgotados("No quedan cupos")

# Aplicación: coordina operaciones
# API: transforma CuposAgotados -> respuesta HTTP 409
```

Relacionar estas decisiones con SOLID: responsabilidad única (SRP), extensión controlada (OCP), sustitución (LSP), interfaces pequeñas (ISP) e inversión de dependencias (DIP). No aplicar patrones por obligación.

5. Condiciones, casos límite y complejidad

Favorecer cláusulas de guarda y expresiones claras. Especificar tratamiento de `None`, colecciones vacías, duplicados, límites, concurrencia y estados inválidos.

```
def validar_periodo(periodo) -> None:
    if periodo is None:
        raise ValueError("El periodo es obligatorio")
    if not periodo.esta_abierto:
        raise PeriodoCerrado("No admite matrículas")
    if periodo.fecha_fin < periodo.fecha_inicio:
        raise ConfiguracionInvalida("Rango inconsistente")
```

No confundir “fallar rápido” con mostrar errores internos al usuario: la API debe convertir excepciones de dominio en mensajes seguros y estables.

6. Tipado, contratos y documentación útil

Las anotaciones de tipos hacen explícitas las expectativas y facilitan el análisis estático, pero no sustituyen la validación en tiempo de ejecución. Los docstrings explican decisiones no evidentes, efectos y restricciones; no deben repetir literalmente el código.

```
from dataclasses import dataclass

@dataclass(frozen=True)
class SolicitudMatricula:
    estudiante_id: int
    asignatura_id: int

def registrar_matricula(solicitud: SolicitudMatricula) -> int:
    """Registra la matrícula y devuelve su identificador.

    Lanza MatriculaDuplicada si ya existe una inscripción activa.
    """
    ...
```

7. Validación y codificación segura

Nunca confiar en entrada del cliente: validar tipo, longitud, rango, relaciones y permisos. Usar consultas parametrizadas u ORM. Evitar construir SQL interpolando cadenas, registrar secretos o exponer trazas internas.

```
# Vulnerable: no utilizar concatenación de valores externos
sql = f"SELECT * FROM estudiantes WHERE codigo = '{codigo}'"

# Correcto: ORM parametrizado
estudiante = Estudiante.objects.filter(codigo=codigo).first()

# Validación de estructura con DRF
class MatriculaSerializer(serializers.Serializer):
    estudiante_id = serializers.IntegerField(min_value=1)
    asignatura_id = serializers.IntegerField(min_value=1)
```

La validación de campos no reemplaza el permiso sobre el recurso ni la verificación de invariantes en el servicio de negocio. Las acciones sensibles se autorizan también a nivel de objeto.

8. Errores explícitos, contexto y trazabilidad

Capturar solo excepciones que se pueden resolver, traducir o enriquecer con contexto. Nunca usar `except Exception: pass`. Evitar devolver detalles del sistema a consumidores públicos.

```
try:
    resultado = servicio.registrar(solicitud)
except MatriculaDuplicada as exc:
    logger.info("matricula_duplicada", extra={
        "estudiante_id": solicitud.estudiante_id,
        "request_id": request_id,
    })
    raise ConflictError("La matrícula ya existe") from exc
```

Usar códigos de error estables, un identificador de correlación por solicitud y logs estructurados. Proteger datos personales en los registros.

9. Persistencia, transacciones y rendimiento

Evitar la consulta N+1 usando `select_related` y `prefetch_related` según la relación. Aplicar restricciones a nivel de base de datos, transacciones y controles de concurrencia cuando se asignan recursos escasos.

```
from django.db import transaction

@transaction.atomic
def asignar_cupo(asignatura_id: int):
    oferta = Oferta.objects.select_for_update().get(
        pk=asignatura_id
    )
    if oferta.ocupados >= oferta.capacidad:
        raise CuposAgotados()
    oferta.ocupados += 1
    oferta.save(update_fields=["ocupados"])
```

Este patrón requiere una base de datos y un modelo de concurrencia compatibles. En sistemas reales se debe comprobar además cómo se representa la matrícula, evitar doble asignación y probar solicitudes simultáneas.

10. Pruebas como parte del diseño

Una buena prueba comunica una regla y detecta una regresión. Usar nombres descriptivos, patrón Arrange–Act–Assert, pruebas de límites y rechazo de estados inválidos. No simular todo: los mocks sirven principalmente para aislar dependencias externas.

```
import pytest

@pytest.mark.parametrize(
    ("capacidad", "inscritos", "esperado"),
    [(20, 19, True), (20, 20, False), (0, 0, False)],
)
def test_disponibilidad_de_cupos(capacidad, inscritos, esperado):
    assert tiene_cupos(capacidad, inscritos) is esperado
```

Complementar pruebas unitarias con integración, API, seguridad y regresión. La cobertura porcentual es un indicador parcial, no un sustituto de escenarios relevantes.

11. Comentarios, documentación y deuda técnica

Un comentario útil explica por qué se toma una decisión, no lo que hace una línea obvia. Un TODO debe tener contexto, responsable o incidencia; no se deben acumular promesas vagas. Documentar decisiones significativas mediante ADR y actualizar OpenAPI cuando cambia el contrato.

```
# Motivo: los registros deben conservar la fecha institucional
# original aunque se actualice la zona horaria del servidor.
fecha_registro = normalizar_fecha_institucional(fecha_cliente)
```

12. Automatización de estilo y calidad

Automatizar el formato, el linting, las pruebas y las comprobaciones de seguridad desde la máquina del desarrollador y CI. Evitar debates repetitivos de formato durante code review.

```
# Instalación (entorno de desarrollo)
python -m pip install ruff pytest pytest-django

# Formato, análisis de estilo y pruebas
ruff format --check .
ruff check .
pytest -q
```

Ejemplo de configuración pyproject.toml

```
[tool.ruff]
line-length = 88

[tool.ruff.lint]
select = ["E4", "E7", "E9", "F", "I", "B", "UP"]

[tool.pytest.ini_options]
DJANGO_SETTINGS_MODULE = "config.settings.test"
python_files = ["test_*.py"]
```

Otras verificaciones posibles: mypy o pyright para tipos, escaneo de dependencias, secretos, migraciones y cumplimiento de pruebas de seguridad. Se recomienda mantener reglas graduales y justificadas.

13. Git, commits y revisión de código

Trabajar con cambios pequeños, fáciles de revisar y con una finalidad definida. El pull request debe explicar qué cambió, por qué, riesgos, pruebas ejecutadas y consideraciones de despliegue.

Revisión	Comprobar
Comportamiento	Requisitos, reglas de negocio y casos extremos
Diseño	Responsabilidades, dependencias y duplicación
Seguridad	Permisos, entradas, secretos y datos personales
Datos	Transacciones, integridad y migraciones

Revisión	Comprobar
Pruebas	Escenarios positivos, negativos y regresión
Operación	Logs, métricas y comportamiento ante fallos

Ejemplo de mensaje de commit

```
fix(matriculas): impedir inscripciones cuando no hay cupos

- agrega validación en el servicio de dominio
- protege el cambio con transacción
- incorpora prueba del último cupo disponible
```

14. Antipatrones frecuentes y alternativas

Antipatrón	Mejor alternativa
God class: una clase hace todo	Separar casos de uso y políticas de dominio
Números mágicos dispersos	Constantes o configuración con nombre
except Exception: pass	Captura específica y observabilidad
Duplicar lógica de validación	Regla única reutilizable y probada
Más abstracciones sin necesidad	Resolver el problema real con diseño simple
Consultas en bucles	Consultas por lote o relaciones precargadas
Credenciales en código	Gestión de secretos por entorno
Pruebas que dependen del orden	Datos aislados y preparación reproducible

15. Laboratorio integrador: refactorización de matrículas

Situación: existe un endpoint que recibe el ID de un estudiante, comprueba cupos, guarda la matrícula y envía un correo. Actualmente toda la lógica está en una vista extensa y varios errores se silencian.

Trabajo propuesto

- Identificar al menos seis problemas de legibilidad, diseño, seguridad o verificabilidad.
- Definir estructura por módulos (API, servicios, dominio y persistencia).
- Crear una función o política de dominio para evaluar cupos y prerrequisitos.
- Incorporar restricciones y una transacción para evitar sobreasignación de cupos.
- Traducir errores del dominio a respuestas HTTP coherentes sin filtrar detalles.
- Agregar pruebas unitarias, de integración, API y un escenario negativo de permisos.
- Ejecutar Ruff y pytest; documentar cambios y publicar un pull request con evidencia.

Evidencias a entregar

Repositorio o enlace al commit; descripción de decisiones; comparación antes/después; pruebas ejecutadas; resultados de herramientas; y breve reflexión sobre riesgos remanentes.

16. Rúbrica de evaluación (2 / 1 / 0)

Criterio	2 puntos	1 punto	0 puntos
Legibilidad	Nombres y flujo claros	Detalles poco claros	Difícil de comprender
Diseño modular	Separación consistente	Separación parcial	Responsabilidades mezcladas
Seguridad	Validaciones y permisos correctos	Controles incompletos	Riesgos evidentes
Pruebas	Cubre escenarios clave	Cobertura parcial	Sin pruebas útiles
Automatización	Herramientas y CI ejecutan	Ejecución incompleta	No se verificó
Documentación	Cambios y decisiones claros	Documentación parcial	Sin evidencia

Puntaje máximo: 12 puntos. La rúbrica puede complementarse con criterios de proyecto y retroalimentación cualitativa.

17. Autoevaluación y preguntas para debatir

- ¿Por qué una función corta no siempre es una función bien diseñada?
- ¿Qué diferencia existe entre validar un tipo de dato y aplicar una regla de negocio?
- ¿En qué casos una transacción y un índice mejoran aspectos distintos del sistema?
- ¿Por qué no debería registrarse el cuerpo completo de todas las solicitudes?
- ¿Cómo distinguir refactorización de cambio funcional?
- ¿Qué beneficio ofrece automatizar el estilo de código antes del pull request?
- ¿Qué pruebas faltan cuando solo verificamos el caso exitoso?

18. Lista de verificación para entregar una funcionalidad

- ☐ Los nombres reflejan conceptos del dominio.
- ☐ El endpoint no contiene lógica de negocio compleja.
- ☐ Las entradas, permisos y estados se validan.
- ☐ Las modificaciones de datos protegen la integridad.
- ☐ Los errores se traducen sin revelar información sensible.
- ☐ No existen secretos ni datos personales en logs.
- ☐ Se ejecutaron pruebas relevantes y verificaciones automáticas.

- □ La documentación y el contrato API están actualizados.
- □ El cambio es revisable y tiene una justificación clara.

19. Glosario y referencias

Refactorización: mejora de la estructura interna sin cambiar el comportamiento observable. Acoplamiento: nivel de dependencia entre partes. Cohesión: concentración de responsabilidades relacionadas. Linting: análisis automatizado de patrones de código. Invariante: condición que debe permanecer verdadera en un estado válido. Code review: revisión colaborativa de cambios.

Python Software Foundation. PEP 8 — Style Guide for Python Code. https://peps.python.org/pep-0008/

Python Software Foundation. PEP 257 — Docstring Conventions. https://peps.python.org/pep-0257/

Ruff. The Ruff Linter and Formatter. https://docs.astral.sh/ruff/

Django. Security in Django. https://docs.djangoproject.com/en/stable/topics/security/

Django REST Framework. Validators y Testing. https://www.django-rest-framework.org/

OWASP. Secure Coding Practices — Quick Reference Guide. https://owasp.org/www-project-secure-coding-practices-quick-reference-guide/

Martin Fowler. Refactoring (2nd edition). Addison-Wesley, 2018.

Nota docente: elegir primero una convención común, justificar las excepciones y evaluar siempre comportamiento, seguridad y claridad. Una herramienta automática informa sobre problemas posibles, pero no sustituye el juicio técnico.

SERIE BACKEND PROFESIONAL · GUÍA 12

Operación y mantenimiento de un backend

Backups, recuperación, auditoría, monitoreo y evolución

Propósito: que el estudiante sea capaz de mantener un backend confiable después de su despliegue, demostrar la recuperabilidad de los datos y planificar su evolución sin comprometer la operación.

Caso transversal: API institucional de matrículas académicas con Django REST Framework y PostgreSQL.

Resultados de aprendizaje: diseñar un plan de respaldos, ejecutar una restauración de ensayo, diseñar evidencias de auditoría, definir umbrales de monitoreo y planificar cambios compatibles.

Prerrequisitos: fundamentos de API REST, base de datos, seguridad, pruebas, observabilidad y DevOps.

Material docente · edición octubre de 2026 · ejemplos educativos para laboratorios controlados.

1. ¿Qué es la operación y mantenimiento?

La operación comprende las tareas que mantienen un servicio disponible y verificable: supervisión, respaldos, atención de incidentes, despliegues controlados y gestión de capacidad. El mantenimiento comprende corregir fallos, adaptar dependencias y evolucionar funcionalidades sin afectar contratos críticos.

Tipo	Pregunta	Ejemplo
Correctivo	¿Qué falló?	Corregir una matrícula duplicada
Preventivo	¿Qué podría fallar?	Probar restauración cada mes
Adaptativo	¿Qué cambió afuera?	Actualizar PostgreSQL compatible
Evolutivo	¿Qué necesidad nueva existe?	Agregar matrícula por equivalencia

Responsabilidades y artefactos operativos

• Propietario del servicio: aprueba objetivos de disponibilidad, prioridades y riesgos.

• Equipo de desarrollo: conserva cambios probados, migraciones seguras y documentación.

• Operaciones: administra alertas, copias, restauraciones y despliegues.

• Seguridad: revisa acceso privilegiado, auditoría, retención e incidentes.

• Artefactos: inventario, runbook, matriz RACI, plan de respaldo, bitácora de incidentes y registro de cambios.

2. Backups: conceptos y estrategias

Un backup es una copia destinada a permitir la recuperación. No basta con que exista un archivo: debe poder leerse, contener los datos correctos, estar protegido y poder restaurarse dentro de los objetivos del servicio.

Modalidad	Ventaja	Limitación
Lógico (pg_dump)	Portable y útil para restauraciones selectivas	Puede tardar en bases grandes
Físico/base backup	Recupera un clúster completo	Exige compatibilidad y procedimientos precisos
WAL + PITR	Permite recuperar hasta un instante	Requiere archivado continuo y pruebas
Snapshot	Rápido para volúmenes	Debe garantizar consistencia y no sustituye una política de backup

Regla práctica 3-2-1 (orientativa)

Mantener tres copias de datos (original más dos), en al menos dos medios o dominios de fallo, con una copia fuera del sitio. Como refuerzo: almacenamiento inmutable o desconectado, cifrado, separación de credenciales y pruebas periódicas.

Ejemplo: respaldo lógico de PostgreSQL

```
pg_dump -Fc -h db.internal -U backup_user \
  -d matriculas -f matriculas_2026-10-13.dump

# Comprobar que el archivo es legible
pg_restore --list matriculas_2026-10-13.dump | head
```

Precaución: el usuario de backup debe disponer de permisos suficientes sin utilizar innecesariamente privilegios de superusuario. Las credenciales no se escriben en scripts ni en el repositorio.

3. Política de respaldos

Elemento	Decisión de ejemplo (debe validarse)
Frecuencia	Copia lógica diaria + continuidad WAL para PITR
Retención	Diarias 14 días; semanales 8 semanas; mensuales 12 meses
Protección	Cifrado en tránsito y reposo; control de accesos; copia aislada
Verificación	Comprobar éxito del job, tamaño, integridad y restauración de ensayo
Responsable	Operaciones ejecuta; responsable del servicio valida
Cobertura	Datos, archivos subidos, configuración recuperable, claves y dependencias

La retención del ejemplo no es universal: se define conforme al riesgo, volumen de datos, obligaciones institucionales y requisitos de privacidad. Los respaldos pueden contener datos personales; protegerlos y eliminarlos según su ciclo de vida.

4. Recuperación y continuidad del servicio

RPO (Recovery Point Objective) es la máxima pérdida de datos tolerable expresada en tiempo. RTO (Recovery Time Objective) es el tiempo objetivo para restablecer el servicio. No son lo mismo que los tiempos realmente obtenidos: se deben medir en simulacros.

Escenario	RPO objetivo	RTO objetivo	Medida
Servidor de API inactivo	No aplica a datos si BD sigue disponible	30 min	Recrear contenedor y validar
Falla de disco de BD	15 min	2 horas	Restaurar base + WAL
Error humano en registros	15 min	2 horas	PITR a instancia aislada y reconciliar

Nota: estos son valores hipotéticos para discusión en clase, no garantías de una infraestructura real.

Procedimiento seguro de restauración: laboratorio

- Declarar el incidente, recopilar hora de ocurrencia, alcance y última copia verificada.
- Aislar un entorno de restauración; nunca sobrescribir producción para practicar.

• Restaurar desde el último backup válido; aplicar WAL hasta el punto requerido si procede.

• Verificar conteos, integridad referencial, usuarios, casos críticos y ausencia de corrupción.

• Ejecutar smoke tests de API, comprobar logs y medir tiempo real de recuperación.

• Documentar RPO/RTO observados y aprobar el cambio de tráfico bajo un procedimiento controlado.

Ejemplo de restauración en base de ensayo

```
createdb -h localhost -U postgres matriculas_restore
pg_restore -h localhost -U postgres \
  --no-owner -d matriculas_restore matriculas_2026-10-13.dump

# Validar registros (solo base de ensayo)
psql -d matriculas_restore -c "SELECT count(*) FROM matriculas;"
```

Una restauración puede necesitar roles, extensiones, archivos adjuntos o secretos que el dump no incluye. Registrar estas dependencias en el runbook.

5. Auditoría: quién hizo qué, cuándo y sobre qué

La auditoría busca evidenciar acciones relevantes y permitir reconstruir eventos. No sustituye logs técnicos: estos explican la ejecución, mientras que la auditoría conserva hechos de negocio o seguridad con contexto verificable.

Modelo mínimo de evento de auditoría

```
{
  "event_id": "evt_8f1...",
  "occurred_at": "2026-10-13T15:10:00Z",
  "actor_id": "usr_42",
  "action": "matricula.anular",
  "resource_type": "matricula",
  "resource_id": "mat_514",
  "outcome": "success",
  "request_id": "req_75a...",
  "reason_code": "SOLICITUD_ESTUDIANTE"
}
```

Controles de una auditoría confiable

• Generar eventos desde servicios de aplicación cuando se confirma la transacción, o usar patrón outbox para envío fiable.

• Restringir lectura y alteración de registros, agregar retención y protección contra manipulación.

• Sincronizar relojes, usar UTC y correlacionar eventos mediante request_id/trace_id.

• Evitar registrar contraseñas, tokens, identificaciones completas o cargas confidenciales.

• Auditar cambios administrativos, permisos, anulaciones, exportaciones y restauraciones.

6. Monitoreo en producción

El monitoreo observa indicadores conocidos y detecta anomalías. La observabilidad (Guía 09) permite investigar a fondo qué ocurre combinando logs, métricas y trazas. Para operación cotidiana se necesitan paneles, umbrales y personas responsables de responder.

Indicador	Pregunta operacional	Acción ante desviación
Disponibilidad	¿La API responde?	Verificar health/readiness y dependencias
Tasa de error 5xx	¿Fallan solicitudes?	Revisar despliegue reciente y trazas
Latencia p95	¿Responde a tiempo?	Revisar consultas, saturación y caché
Disco y conexiones	¿Hay riesgo de agotamiento?	Escalar capacidad y ajustar límites
Último backup correcto	¿Podríamos recuperarnos?	Investigar job y ejecutar copia alternativa
Lag de replicación	¿Se retrasa la réplica?	Revisar saturación y continuidad

SLO y alertas: ejemplo

SLO propuesto para el laboratorio: 99,9 % de solicitudes exitosas en 30 días (definir exclusiones y medición) y p95 menor a 400 ms en el endpoint de consulta. No son objetivos recomendados para todo sistema: deben negociarse con el negocio.

```
# PromQL ilustrativo: ratio de errores 5xx en 5 min
sum(rate(http_requests_total{status=~"5.."}[5m]))
/
sum(rate(http_requests_total[5m]))
```

Una alerta debe indicar condición, severidad, destinatario y enlace a un runbook. Evitar alertas con datos insuficientes, por ejemplo divisiones con denominador cero o picos instantáneos sin relevancia.

7. Gestión de incidentes y cambios

Ciclo de vida de un incidente

Detección → clasificación → contención → diagnóstico → recuperación → comunicación → retrospectiva. Documentar la línea de tiempo, el impacto real, los responsables y las acciones preventivas sin culpabilizar a personas.

Ejemplo: el último despliegue eleva los errores 500

- Comprobar los errores por versión desplegada y su impacto.
- Si hay evidencia suficiente, activar rollback de la aplicación.
- No revertir a ciegas una migración destructiva de base de datos.
- Informar estado y siguiente hito de actualización.
- Crear prueba de regresión y una acción preventiva verificable.

Tipos de mantenimiento y compatibilidad

Toda evolución debe pasar por análisis de impacto, requisitos, migraciones compatibles, pruebas, plan de despliegue y reversión. Para cambios de esquema con cero o baja interrupción, aplicar la estrategia expand → migrate → contract: incorporar estructuras compatibles, migrar clientes/datos y retirar elementos antiguos una vez confirmada la transición.

8. Ejemplo de evolución segura de una API

Nuevo requerimiento: almacenar tipo de matrícula (ordinaria, extraordinaria). Evitar agregar de golpe un campo NOT NULL que rompa registros y despliegues existentes.

• Fase 1: añadir campo nullable con migración aditiva.

• Fase 2: desplegar código que lee valores antiguos y escribe el nuevo campo.

• Fase 3: completar datos históricos con job idempotente y observar métricas.

• Fase 4: introducir validación obligatoria cuando todos los productores están preparados.

• Fase 5: retirar compatibilidad antigua tras fecha anunciada y pruebas.

```
# Django: primer paso, migración compatible
class Matricula(models.Model):
    tipo = models.CharField(
        max_length=20, null=True, blank=True
    )

# Posteriormente: backfill + validación + constraint
```

9. Runbooks y documentación operativa

Un runbook convierte una situación frecuente en instrucciones reproducibles y seguras. Debe indicar propósito, prerrequisitos, señales, pasos, verificación, rollback, permisos necesarios, escalamiento y versión del documento.

Runbook	Qué debe responder
API caída	¿Cómo comprobar si falla aplicación, proxy o base de datos?
Backup fallido	¿Cómo detectar causa, repetir y proteger datos?
Restauración	¿Qué restaurar, quién autoriza y cómo comprobar?
Cambio de versión	¿Cuándo pausar, desplegar, verificar y volver atrás?
Incidente de seguridad	¿Cómo preservar evidencia y escalar sin divulgar datos?

10. Laboratorio integrador: matrícula académica

Situación: el sistema gestiona matrículas y adjuntos; un error de despliegue produce tasas elevadas de fallos y existe riesgo de pérdida de datos. Los equipos reciben una copia anonimizada y deben simular procedimientos en un ambiente local.

Actividades guiadas

• A1. Identificar datos críticos, dependencias y responsables; proponer política de backup.

• A2. Ejecutar pg_dump y restaurarlo en una segunda base; registrar tamaño y duración.

• A3. Probar consultas que aseguren integridad y ausencia de filas huérfanas.

• A4. Diseñar eventos de auditoría para crear, aprobar y anular matrículas.

• A5. Definir cuatro indicadores y dos alertas útiles con runbook.

- A6. Simular un cambio de modelo compatible y planificar rollback.
- A7. Entregar informe con evidencias, limitaciones y riesgos residuales.

Productos esperados

- Plan de respaldos y recuperación, con RPO/RTO argumentados.
- Script y evidencia de restauración controlada, más resultados de verificaciones.
- Esquema de auditoría y cuatro eventos de ejemplo sin datos sensibles.
- Panel o consultas de métricas, reglas de alertas y dos runbooks.
- ADR corto justificando estrategia de evolución y documento postmortem ficticio.

11. Evaluación y comprobación

Criterio	2 puntos: logrado	1 punto: parcial	0 puntos: ausente
Backups	Política clara y copia verificable	Copia sin política o verificación	No implementado
Restauración	Restauración probada y validada	Restauración sin controles	No comprobada
Auditoría	Eventos, protección y correlación	Eventos incompletos	No registra
Monitoreo	SLI, alertas y runbook coherentes	Indicadores sin actuación	No definido
Evolución	Migración compatible y reversión viable	Cambio sin análisis completo	Cambio riesgoso

Puntaje orientativo: 10 puntos. El docente puede añadir evaluación de comunicación, trabajo en equipo y protección de datos.

Preguntas de autoevaluación

- 1. ¿Por qué un archivo de respaldo no garantiza recuperación?
- 2. ¿Qué diferencia hay entre RPO y RTO? Proponga ambos para matrículas.
- 3. ¿Cuándo conviene un dump lógico y cuándo PITR?
- 4. ¿En qué se diferencia auditoría de logging técnico?
- 5. ¿Qué datos no deberían almacenarse en un evento?
- 6. ¿Cómo evitar alertas que producen fatiga operativa?
- 7. ¿Por qué una migración destructiva complica el rollback?
- 8. ¿Qué verificaciones determinan si una restauración fue exitosa?

12. Lista de verificación antes de operar

- [ ] Inventario de servicios, dependencias y responsables actualizado.

- [ ] Backups automáticos cifrados con supervisión de ejecución.
- [ ] Restauración ensayada y RPO/RTO medidos.
- [ ] Accesos privilegiados mínimos y auditoría revisable.
- [ ] Dashboards, SLO, alertas y rutas de escalamiento.
- [ ] Runbooks probados y procedimiento de incidentes.
- [ ] Cambios con pruebas, migraciones compatibles y rollback previsto.
- [ ] Calendario de parches, dependencias, soporte y revisión de capacidad.

13. Glosario y referencias

RPO: pérdida temporal máxima de datos tolerable. RTO: tiempo objetivo para recuperar el servicio. PITR: restauración a un instante usando backup base y WAL. WAL: registro de escritura anticipada de PostgreSQL. Runbook: procedimiento operacional repetible. SLO: objetivo medible de confiabilidad. ADR: registro de decisión arquitectónica. Postmortem: análisis posterior a un incidente.

Referencias de estudio:

- PostgreSQL 18. Backup and Restore. https://www.postgresql.org/docs/18/backup.html
- PostgreSQL 18. Continuous Archiving and PITR. https://www.postgresql.org/docs/18/continuous-archiving.html
- Django. Deployment checklist. https://docs.djangoproject.com/en/stable/howto/deployment/checklist/
- NIST SP 800-34 Rev. 1. Contingency Planning Guide for Federal Information Systems. https://doi.org/10.6028/NIST.SP.800-34r1
- Google. Site Reliability Engineering Workbook. https://sre.google/workbook/
- OpenTelemetry. Documentation. https://opentelemetry.io/docs/

Nota didáctica: ejecutar comandos solo en sistemas de laboratorio autorizados. Antes de aplicar políticas a producción, revisar requisitos normativos y procedimientos institucionales. Esta guía no sustituye un plan de continuidad aprobado.

GUÍA DIDÁCTICA 11

DevOps para un Backend Profesional

CI/CD · Contenedores · Configuración por entornos · Despliegue

Material de estudio universitario | Desarrollo de software y operación de servicios web

Propósito de aprendizaje

Comprender cómo transformar una aplicación backend que funciona en un equipo de desarrollo en un servicio verificable, reproducible, seguro y desplegable. Se utiliza como caso transversal una API de matrículas académicas con Django REST Framework y PostgreSQL.

Resultados de aprendizaje

• Diferenciar integración, entrega y despliegue continuos mediante un flujo verificable.

• Construir y ejecutar una imagen de contenedor reproducible, con medidas básicas de seguridad.

• Configurar entornos de desarrollo, pruebas, staging y producción sin exponer secretos.

• Diseñar un pipeline CI/CD y aplicar una estrategia de despliegue, validación y rollback.

Mapa de la guía

1. Fundamentos · 2. Flujo DevOps · 3. Git y CI · 4. CD · 5. Docker · 6. Compose · 7. Entornos y secretos · 8. Despliegues · 9. Seguridad y observabilidad · 10. Laboratorio · 11. Evaluación · 12. Referencias.

Requisitos previos: conocimientos básicos de HTTP, Git, APIs REST, base de datos y pruebas automatizadas.

1. ¿Qué es DevOps?

DevOps es un enfoque cultural y técnico que busca reducir la separación entre desarrollo (Dev) y operaciones (Ops) mediante colaboración, automatización, retroalimentación continua y responsabilidad compartida. No es sinónimo de Docker ni de instalar un servidor.

Conceptos esenciales

Concepto	Significado	Ejemplo académico
CI	Integración continua: integrar y verificar cambios frecuentemente.	Ejecutar lint, pruebas y construcción al recibir una solicitud de cambios.
CD: entrega	Preparar automáticamente una versión lista para desplegar; liberación puede requerir aprobación.	Generar una imagen validada y publicar artefacto.
CD: despliegue	Publicar automáticamente los cambios aprobados según la política del equipo.	Desplegar a staging y luego producción con controles.
IaC	Infraestructura como código: configuración versionada y revisable.	Definir servicios de aplicación y base de datos mediante Compose.

Idea clave

CI no significa despliegue automático. CD puede significar continuous delivery (entrega) o continuous deployment (despliegue); se debe aclarar cuál se utiliza. Un pipeline fallido debe bloquear la promoción de la versión.

2. Del código al servicio: cadena de entrega

Desarrollador → commit / pull request → análisis estático → pruebas unitarias e integración → construcción de imagen → escaneo → artefacto etiquetado → staging → pruebas de humo → aprobación → producción → monitoreo.

Cada flecha representa una transferencia que puede fallar. Defina para cada etapa entradas, salida verificable, responsable y criterio de rechazo.

Prácticas operativas

• Versionar código, dependencias, migraciones y configuración no sensible.

• Aplicar cambios pequeños y revisiones por pull request.

• Usar artefactos inmutables (idealmente referenciados por digest) entre staging y producción.

• Registrar qué commit, imagen, versión de esquema y aprobación se desplegaron.

3. Control de versiones e integración continua

Una rama principal protegida, revisiones de código y verificaciones obligatorias reducen cambios no verificados. GitFlow es posible, pero no obligatorio: elija una estrategia de ramas según la cadencia de integración.

Pipeline mínimo

Fase	Qué verifica	Criterio de aceptación
Lint y formato	Consistencia y defectos frecuentes	Ruff sin errores
Tests	Reglas, API y componentes	pytest finaliza sin fallas
Migraciones	Esquema coherente	makemigrations --check --dry-run
Build	Empaquetado reproducible	docker build exitoso
Seguridad	Dependencias e imagen	Riesgos evaluados; bloquear críticos según política

Ejemplo: flujo CI de GitHub Actions

```
name: Backend CI
on:
  pull_request:
  push:
    branches: [main]
permissions:
  contents: read
jobs:
  verify:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: '3.12'
      - run: pip install -r requirements.txt
      - run: pip install ruff pytest pytest-django
      - run: ruff check .
      - run: pytest -q
```

Este ejemplo es ilustrativo: para una prueba de integración real con PostgreSQL, agregue un servicio de base de datos, variables de prueba, health checks y migraciones. En proyectos de producción, fije las acciones a versiones revisadas o SHA completos para disminuir riesgos de la cadena de suministro.

Actividad 1

Explique por qué las pruebas automáticas en un pull request no equivalen a desplegar a producción y proponga tres reglas de protección de rama.

4. Entrega y despliegue continuos (CD)

La entrega continua garantiza que existe una versión técnicamente lista para publicarse; el despliegue continuo automatiza además su puesta en servicio. Ambos requieren separar la verificación de la decisión de publicar.

Flujo de promoción recomendado

Commit identificado → imagen construida una sola vez → registro de imágenes → despliegue a staging → smoke tests → aprobación humana opcional → producción → verificación posterior → rollback si falla.

En GitHub Actions, los environments permiten limitar secretos, ramas y aprobaciones por ambiente. Las capacidades concretas dependen de la configuración y del plan del repositorio.

Etiquetado y procedencia

```
# Ejemplo conceptual, no credenciales reales
docker build -t registry.example.edu/matriculas:1.4.0 .
docker push registry.example.edu/matriculas:1.4.0
# Idealmente desplegar por digest sha256:...
```

Criterios de promoción

- Todas las verificaciones de CI concluyeron correctamente.
- La imagen fue analizada y su procedencia queda registrada.
- Las migraciones son compatibles con la versión anterior durante el despliegue.
- La configuración de staging representa los componentes críticos de producción.
- Existe procedimiento documentado para revertir código y recuperar datos si fuera necesario.

5. Contenedores: conceptos

Una imagen es una plantilla inmutable de capas; un contenedor es su instancia ejecutable. Dockerfile describe la construcción. Un registro almacena imágenes. Los volúmenes preservan datos fuera del ciclo de vida de un contenedor.

Elemento	Qué resuelve
Imagen	Empaquetado repetible de runtime y aplicación.
Contenedor	Proceso aislado con límites y configuración definidos.
Red	Comunicación controlada entre servicios.
Volumen	Persistencia de datos (requiere respaldos independientes).
Health check	Comprobación operacional; no sustituye monitoreo.

6. Dockerfile para Django: ejemplo didáctico

```
FROM python:3.12-slim
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
RUN useradd --create-home appuser
USER appuser
EXPOSE 8000
CMD ["gunicorn", "config.wsgi:application", \
     "--bind", "0.0.0.0:8000"]
```

La imagen debe considerar versiones fijadas de dependencias, manejo de archivos estáticos, un archivo .dockerignore, un proceso no privilegiado y dependencias sin secretos. En producción, evalúe builds multietapa y fijar la imagen base por digest.

Archivo .dockerignore

```
.git
.venv
__pycache__
*.pyc
.env
.env.*
*.sqlite3
coverage.xml
.pytest_cache
```

Construcción y ejecución local

```
docker build -t api-matriculas:dev .
docker run --rm -p 8000:8000 \
  --env-file .env.local api-matriculas:dev
```

Importante: un archivo .env.local jamás debe contener credenciales reales del entorno institucional ni subirse a Git. Docker por sí solo no cifra secretos ni convierte automáticamente una aplicación en segura.

Buenas prácticas de imagen

- Base mínima y actualizada; revisar CVE antes de publicar.
- Separar dependencias de desarrollo respecto de producción.
- Evitar instalar herramientas innecesarias dentro de la imagen final.
- No utilizar el servidor de desarrollo de Django para tráfico productivo.

7. Orquestación local con Docker Compose

Compose permite describir múltiples servicios y sus dependencias. Este ejemplo didáctico inicia una API Django y PostgreSQL con comprobación de disponibilidad de la base de datos.

```
services:
  db:
    image: postgres:17
    environment:
      POSTGRES_DB: matriculas
      POSTGRES_USER: matriculas
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U matriculas"]
      interval: 5s
      retries: 10
  api:
    build: .
    ports: ["8000:8000"]
    environment:
      DATABASE_URL: ${DATABASE_URL}
      DJANGO_DEBUG: "false"
    depends_on:
      db:
        condition: service_healthy
volumes:
  pgdata:
```

Para comenzar, defina DB_PASSWORD y DATABASE_URL en un archivo local fuera del control de versiones. En un despliegue real, utilice un mecanismo de secretos adecuado y evite incrustar la contraseña en una URL almacenada de forma insegura. No publique el puerto de PostgreSQL a Internet.

Comandos frecuentes

```
docker compose config
docker compose up --build -d
docker compose ps
docker compose logs --tail=100 api
docker compose down        # no elimina volúmenes por defecto
```

Precaución: docker compose down -v sí elimina los volúmenes definidos; puede ocasionar pérdida de datos. No lo ejecute sobre un entorno con información importante.

8. Configuración por entornos y doce factores

El mismo código puede desplegarse en distintos ambientes mediante configuración externa. No debe construirse una rama o imagen diferente para cada secreto de producción.

Entorno	Objetivo	Valores habituales
Local	Desarrollo rápido	DEBUG habilitado; servicios simulados; datos sintéticos.
Test	Ejecución automatizada	Base aislada; credenciales efímeras; resultados reproducibles.
Staging	Validación previa	DEBUG desactivado; infraestructura semejante a producción.
Producción	Operación real	HTTPS; secretos gestionados; backups; observabilidad.

Configuración de Django con variables de entorno

```
import os

def env_bool(name, default=False):
    return os.getenv(name, str(default)).lower() == "true"

DEBUG = env_bool("DJANGO_DEBUG", False)
SECRET_KEY = os.environ["DJANGO_SECRET_KEY"]
ALLOWED_HOSTS = os.environ.get(
    "DJANGO_ALLOWED_HOSTS", ""
).split(",")
SESSION_COOKIE_SECURE = not DEBUG
CSRF_COOKIE_SECURE = not DEBUG
```

En producción, valide dominios, HTTPS y configuración de proxy, nunca use claves por defecto, y ejecute python manage.py check --deploy contra la configuración real. La lectura de variables no sustituye controles de gestión y rotación de secretos.

Secretos: qué no hacer

- No colocar contraseñas, tokens o llaves privadas en Dockerfile, repositorio o logs.
- No copiar archivos .env reales dentro de las imágenes.
- No otorgar a todos los jobs acceso a secretos de producción.
- Preferir identidad federada/OIDC y credenciales temporales para acceso a nube cuando sea viable.

9. Estrategias de despliegue y rollback

Estrategia	Funcionamiento	Ventaja / riesgo
Recreate	Detener anterior e iniciar nueva versión.	Simple; puede causar interrupción.
Rolling	Reemplazar instancias gradualmente.	Menor interrupción; exige compatibilidad temporal.
Blue/green	Dos entornos; tráfico cambia entre ambos.	Rollback rápido del tráfico; coste adicional.
Canary	Enviar primero pequeña fracción del tráfico.	Control de riesgo; requiere métricas y ruteo.

Despliegue seguro en secuencia

• Comprobar backups y capacidad de recuperación; definir ventana y responsables.

• Aplicar migraciones compatibles (patrón expand/contract cuando corresponda).

• Publicar la imagen aprobada; verificar readiness y endpoints críticos.

• Monitorizar tasa de errores, latencia p95, saturación y logs.

• Si se supera el umbral de falla, revertir el tráfico o la imagen y ejecutar el plan de recuperación.

Rollback no es lo mismo que restaurar datos

Revertir una imagen no revierte automáticamente una migración destructiva ni recupera datos eliminados. Diseñe cambios de esquema compatibles hacia atrás; pruebe restauraciones antes de necesitar utilizarlas.

10. Seguridad y confiabilidad en DevOps

• Principio de mínimo privilegio para pipeline, runners y cuentas de despliegue.

• Escaneo de dependencias, imágenes y secretos; gestionar hallazgos con contexto.

• Políticas de retención de logs, artefactos y respaldos.

• Separar liveness (el proceso está vivo) de readiness (puede recibir tráfico).

• Definir SLI/SLO y alertas por impacto en usuarios, no solo por CPU.

Ejemplo: si una nueva versión aumenta respuestas 5xx y latencia p95 al consultar matrículas, la señal operativa debe activar investigación y, según política, rollback.

11. Laboratorio integrador: API de matrículas académicas

Escenario: el equipo mantiene una API REST para consultar e inscribir estudiantes. Se requiere trasladarla del equipo de un desarrollador a un servidor de staging con despliegue reproducible.

Producto mínimo del laboratorio

- Repositorio Git con README, requirements.txt, pruebas y migraciones.
- Dockerfile y .dockerignore revisados.
- compose.yaml con API y PostgreSQL, almacenamiento persistente y health check.
- Configuración local/staging mediante variables; archivo .env.example sin valores sensibles.
- Workflow CI: lint + pruebas + verificación de migraciones + build.
- Procedimiento de despliegue, smoke tests, logs, rollback y evidencia.

Prueba de humo propuesta

```
# Ejemplos de respuestas esperadas
GET /health/live     -> HTTP 200
GET /health/ready    -> HTTP 200 si BD disponible
GET /api/v1/cursos/  -> HTTP 200, contrato esperado
POST /api/v1/matriculas/ -> HTTP 401 sin autenticación
```

Los endpoints de salud son propuestos para el ejercicio: deben implementarse en el proyecto y protegerse frente a exposición de datos internos.

Experimento de falla controlada

1) Detenga PostgreSQL en staging. 2) Observe readiness, logs y respuestas de la API. 3) Reinicie el servicio. 4) Registre tiempo de recuperación y las condiciones que deben activar una alerta. 5) Explique qué parte pudo y cuál no pudo recuperar el contenedor.

Preguntas para discusión

- ¿Qué ocurre cuando una migración elimina una columna utilizada por la versión anterior?
- ¿Por qué conviene desplegar el mismo digest de imagen en staging y producción?
- ¿Qué información es imprescindible para auditar un despliegue?

12. Evaluación y rúbrica

Criterio	2 puntos	1 punto	0 puntos
Pipeline CI	Automático, reproducible, bloquea fallas.	Parcial o sin algunos controles.	Ausente o no funciona.
Contenedores	Imagen segura y Compose funcional.	Funciona con defectos o riesgos.	No se ejecuta.
Entornos y secretos	Separación correcta, sin filtraciones.	Separación incompleta.	Secretos expuestos.
Despliegue	Staging, smoke y rollback documentados.	Documentación incompleta.	Sin estrategia viable.
Evidencia	Logs, ejecuciones y conclusiones verificables.	Evidencia parcial.	Sin evidencia.

Total máximo: 10 puntos. Recomendada evaluación individual del entendimiento y grupal de los artefactos compartidos.

Autoevaluación

- Explique con sus palabras la diferencia entre integración, entrega y despliegue continuos.
- ¿Qué problema resuelve una imagen inmutable? ¿Qué NO resuelve?
- ¿Por qué las variables de entorno requieren gestión segura de secretos?
- ¿En qué caso elegiría blue/green frente a recreate?
- ¿Qué indicadores usaría para detectar una versión defectuosa?
- ¿Qué sucede con datos persistentes al recrear un contenedor sin volumen?

Lista de comprobación antes de producción

- Pruebas y escaneos aprobados; artefacto identificable.
- Configuración de producción revisada con check --deploy.
- Certificados TLS, dominios y variables validadas.
- Migraciones compatibles, respaldo y restauración probados.
- Monitoreo, alertas, responsables y rollback verificados.

13. Glosario y referencias

Término	Definición breve
Pipeline	Secuencia automatizada de validación y entrega.
Runner	Agente que ejecuta los trabajos definidos en CI/CD.
Artefacto	Resultado versionado de una etapa de construcción.
Registry	Repositorio de imágenes de contenedores.
Staging	Entorno previo a producción para validaciones.
Rollback	Retorno operativo a una versión anterior.
Readiness	Señal que indica capacidad de atender solicitudes.
IaC	Definición declarativa y versionada de infraestructura.

Documentación oficial recomendada

• Docker Docs. Compose in production: https://docs.docker.com/compose/how-tos/production/

• Docker Docs. Building images: https://docs.docker.com/build/

• GitHub Docs. Workflows: https://docs.github.com/en/actions

• GitHub Docs. Deployment environments: https://docs.github.com/en/actions/concepts/workflows-and-actions/deployment-environments

• Django Docs. Deployment checklist: https://docs.djangoproject.com/en/stable/howto/deployment/checklist/

• The Twelve-Factor App: https://12factor.net/

• OWASP. CI/CD Security Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/CI_CD_Security_Cheat_Sheet.html

Nota didáctica: las versiones del ejemplo son ilustrativas y deben revisarse según el soporte vigente, dependencias del proyecto y política institucional. Nunca copie credenciales reales en ejercicios o repositorios.

SERIE BACKEND PROFESIONAL · GUÍA 10

Rendimiento del backend

Caché · Paginación · Consultas optimizadas · Tareas asíncronas

Guía didáctica para estudiantes universitarios de Computación. Documento de estudio autónomo con fundamentos, decisiones de diseño, ejemplos Python/Django/PostgreSQL, laboratorio, preguntas y rúbrica.

Caso transversal: plataforma de gestión académica con estudiantes, asignaturas, matrículas y generación de reportes.

Objetivos: identificar cuellos de botella; seleccionar estrategias de caché y paginación; diagnosticar consultas; diseñar trabajos asíncronos confiables; y comprobar mejoras mediante evidencia.

Requisitos previos: HTTP y API REST, bases de datos relacionales, Django ORM y conceptos básicos de pruebas.

1. ¿Qué significa rendimiento?

El rendimiento es la capacidad de un sistema para atender una carga de trabajo con tiempos de respuesta y recursos aceptables. No equivale solamente a “hacer el código más rápido”: incluye capacidad, estabilidad, consumo y experiencia del usuario.

Término	Definición didáctica	Ejemplo
Latencia	Tiempo de una solicitud individual	GET /matriculas tarda 240 ms
Throughput	Solicitudes u operaciones por segundo	150 solicitudes/segundo
Concurrencia	Operaciones activas simultáneamente	40 solicitudes en curso
p95 / p99	Percentil de tiempos observado	p95 450 ms: 95 % terminan ≤450 ms
Saturación	Recurso al límite que acumula espera	Pool de conexiones agotado

La regla principal: medir antes de optimizar. Definir escenario de carga, volumen de datos, entorno, percentiles, tasa de errores y línea base. Una media favorable puede ocultar solicitudes extremadamente lentas.

2. Método de mejora basada en evidencia

Secuencia: (1) formular hipótesis → (2) medir línea base → (3) localizar cuello de botella → (4) introducir un cambio aislado → (5) repetir medición equivalente → (6) documentar impacto y efectos secundarios.

```
Escenario: 100 usuarios concurrentes, 50 000 matriculas.
Indicadores a registrar: p50, p95, p99, solicitudes/s,
% 4xx/5xx, CPU, memoria, consultas/request, conexiones DB.
IMPORTANTE: los objetivos numéricos se acuerdan para cada sistema.
```

3. Caché: reutilizar trabajo sin perder corrección

Una caché guarda resultados o representaciones temporalmente para evitar cálculos, lecturas o transferencias repetidas. Es adecuada cuando el coste de obtener un dato supera el coste y riesgo de almacenarlo. Se debe diseñar el vencimiento y la invalidación.

3.1 Tipos y patrones

Modalidad	Uso	Riesgo / control
Caché HTTP	ETag, Cache-Control, 304	Datos privados: reglas estrictas
Caché de aplicación	Resultados de servicios y catálogos	Claves y TTL con alcance correcto
Redis	Caché compartida entre instancias	Memoria, caídas y expiración
Cache-aside	Consultar caché; si falla consultar DB	Invalidez tras actualizar
Invalidación por evento	Eliminar clave al modificar datos	Eventos perdidos, carreras

No cachear sin control matrículas individualizadas ni datos sensibles en cachés compartidas. En cache keys incluir los permisos/ámbito de organización y filtros pertinentes; jamás una clave global para una respuesta personalizada.

```
# settings.py (ejemplo didactico; Redis debe estar instalado)
CACHES = {
    "default": {
        "BACKEND": "django.core.cache.backends.redis.RedisCache",
        "LOCATION": "redis://127.0.0.1:6379/1",
    }
}

from django.core.cache import cache

def catalogo_periodos():
    clave = "catalogo:periodos:v1"
    resultado = cache.get(clave)
    if resultado is None:
        resultado = list(Periodo.objects.values("id", "nombre"))
        cache.set(clave, resultado, timeout=300)
    return resultado
```

Discusión: ¿qué sucede cuando el personal actualiza un periodo? Debe invalidarse la clave (o cambiar su versión) después de confirmar la transacción. Un TTL limita la obsolescencia, pero no reemplaza siempre la invalidación explícita.

4. Paginación: controlar tamaño y costo

Paginación significa dividir una colección en segmentos de tamaño limitado. Reduce la transferencia de datos y el trabajo de serialización; también previene respuestas enormes.

Estrategia	Ventajas	Limitaciones
Page number / OFFSET	Sencilla, permite saltar a página	OFFSET grande puede encarecer lectura; cambios causan saltos
Limit/offset	Flexible y fácil de comprender	Requiere límites máximos y orden estable

Estrategia	Ventajas	Limitaciones
Cursor / keyset	Adecuada para listados grandes o cambiantes	Requiere ordenamiento estable y claves apropiadas

```
# DRF: paginacion con cursor (ejemplo)
from rest_framework.pagination import CursorPagination

class MatriculaCursorPagination(CursorPagination):
    page_size = 25
    ordering = "-created_at"  # conviene ordenar tambien por id

# Ejemplo de keyset SQL con clave de desempate:
SELECT id, created_at, estado
FROM matricula
WHERE (created_at, id) < (:fecha, :ultimo_id)
ORDER BY created_at DESC, id DESC
LIMIT 25;
```

Un cursor correcto requiere un orden total estable: si varias filas tienen la misma fecha, el identificador resuelve empates. Considerar índices compatibles con el filtro y el orden. Validar que el tamaño máximo de página no pueda ser arbitrariamente alto.

5. Optimización de consultas SQL y ORM

Un endpoint puede ser lento por ejecutar demasiadas consultas, leer filas innecesarias o usar planes costosos. La optimización responsable comienza con el patrón de acceso real.

5.1 Problema N+1

```
# Ineficiente: una consulta de matriculas + una por estudiante
matriculas = Matricula.objects.all()
for m in matriculas:
    print(m.estudiante.nombre)

# Mejor: relacion ForeignKey mediante JOIN
matriculas = Matricula.objects.select_related("estudiante")
for m in matriculas:
    print(m.estudiante.nombre)

# Para colecciones many-to-many / relaciones reversas:
Asignatura.objects.prefetch_related("matriculas")
```

select_related suele resolver relaciones simples mediante JOIN; prefetch_related recupera relaciones múltiples en consultas adicionales controladas. No aplicar ambos indiscriminadamente: comparar número de consultas, memoria y plan.

5.2 Consultas selectivas, índices y planes

```
-- Indice orientado a consultas frecuentes por periodo/estado
CREATE INDEX idx_matricula_periodo_estado
    ON matricula (periodo_id, estado);

-- Observar plan REAL en un entorno de prueba
EXPLAIN (ANALYZE, BUFFERS)
SELECT id, estudiante_id, estado
FROM matricula
WHERE periodo_id = 12 AND estado = 'APROBADA';
```

La salida muestra scans, joins, filas estimadas y reales, uso de buffers y tiempos. Un índice no siempre es mejor que lectura secuencial: depende de selectividad, distribución y tamaño de datos. Cada índice también encarece escrituras y ocupa espacio. EXPLAIN ANALYZE ejecuta la sentencia: extremar precaución en modificaciones de datos.

5.3 Checklist de consulta

Comprobar filtros, ordenamiento, columnas seleccionadas, relaciones precargadas, índices apropiados, cardinalidad de JOIN, estadísticas del optimizador, consultas repetidas y límites. Inspeccionar las consultas reales del ORM, no solamente su apariencia en Python.

6. Procesamiento asíncrono y tareas de fondo

Las tareas que tardan segundos o minutos —como un reporte masivo de matrículas— normalmente no deben bloquear la conexión HTTP. La arquitectura típica separa API, broker/cola, worker y almacenamiento de resultados.

```
CLIENTE --POST /reportes--> API --encola--> BROKER
   |                          |                |
   |<--202 Accepted + job_id-|                v
   |                                       WORKER
   |--GET /jobs/{id}--------------------------|
   |<-- estado/progreso/resultado ------------|
```

El HTTP 202 Accepted comunica que el procesamiento se ha aceptado, no que haya terminado correctamente. El cliente consulta un recurso de estado o recibe notificación cuando el trabajo finaliza.

6.1 Celery con Django: flujo ilustrativo

```
# tasks.py
from celery import shared_task

@shared_task(bind=True, autoretry_for=(TimeoutError,),
             retry_backoff=True, retry_kwargs={"max_retries": 3})
def generar_reporte(self, periodo_id):
    # Reconsultar la DB; no transmitir datos sensibles en la cola
    # Produccion: idempotencia, permisos y almacenamiento duradero
    return crear_reporte(periodo_id)

# views.py, dentro de un POST autenticado/autorizado:
job = generar_reporte.delay(periodo_id)
return Response({"job_id": job.id, "estado": "pendiente"},
                status=202)
```

Precauciones: una cola puede entregar la misma tarea más de una vez. Diseñar operaciones idempotentes; limitar reintentos y aplicar backoff; usar dead-letter queue donde proceda; establecer timeouts, límites de concurrencia, observabilidad, permisos de acceso al estado y retención de resultados. Encolar tras el commit para evitar leer datos todavía no confirmados.

6.2 ¿Async HTTP es igual que una cola?

No. El soporte async/await permite esperar operaciones I/O sin bloquear el hilo de ejecución correspondiente; no garantiza persistencia, reintentos ni seguimiento duradero de trabajos. Para tareas largas o críticas, un worker y una cola suelen ofrecer garantías operativas más adecuadas.

7. Optimización de extremo a extremo

Analizar capas separadamente: red y tamaño JSON; autenticación y serialización; lógica de dominio; ORM/SQL; caché; servicios externos; y saturación de procesos/conexiones. Evitar optimizar solo un componente cuando el cuello de botella se ubica en otro.

Síntoma	Hipótesis inicial	Evidencia útil
p95 crece con carga	Conexiones o workers saturados	Pool, cola, utilización
Muchas consultas por request	N+1 / acceso repetido	Query count y SQL
Listado muy pesado	Sin paginar o serialización costosa	Tamaño respuesta, duración
Reporte tarda 40 s	Trabajo síncrono	Traces, CPU, tiempos
Datos obsoletos	TTL / invalidación defectuosa	Eventos, claves, hit ratio

8. Laboratorio guiado: matrículas académicas

Escenario: el endpoint GET /api/v1/matriculas devuelve miles de registros; tarda mucho bajo carga; la descarga de un reporte bloquea solicitudes. Trabajo en grupos de 3–5 estudiantes.

Fase A — Diagnóstico: crear datos de prueba; capturar latencia p50/p95, tamaño de payload, consultas por request, uso de CPU/memoria y plan de consulta. Guardar capturas y comandos reproducibles.

Fase B — Cambios: introducir paginación con orden estable; controlar máximo page_size; solucionar N+1 con select_related; crear un índice solo si el plan lo respalda; aplicar caché a catálogo de asignaturas con invalidación. Registrar hipótesis por cambio.

Fase C — Asincronía: crear POST /reportes que devuelve 202 y job_id; consultar GET /jobs/{id}; implementar estados PENDIENTE, EN_PROCESO, COMPLETADO, FALLIDO; probar reintentos e idempotencia.

Fase D — Verificación: repetir idéntico escenario de carga. Comparar métricas y confirmar funcionalidad, integridad, permisos, orden estable y ausencia de datos desactualizados.

Métrica	Antes	Después	Interpretación
p95 (ms)	____	____	¿Mejoró en la misma carga?
SQL/request	____	____	¿Disminuyó N+1?
Solicitudes/s	____	____	¿Mayor capacidad?
Errores (%)	____	____	¿Se mantuvo confiabilidad?
Duración reporte (s)	____	____	¿Se liberó HTTP?

9. Ejercicios de aprendizaje

• Explique la diferencia entre latencia, throughput y concurrencia con un ejemplo de API.

• Diseñe una clave de caché segura para el catálogo de asignaturas por institución e idioma. ¿Cuándo la invalida?

• Justifique el uso de keyset frente a OFFSET para un listado de 500 000 matrículas.

• Encuentre el patrón N+1 en un serializer que expone estudiante y asignaturas. Proponga corrección.

• Interprete un plan EXPLAIN y argumente por qué conviene o no crear un índice compuesto.

• Diseñe estados, respuestas HTTP, reintentos y errores de un trabajo de generación de certificados.

• Proponga cómo medir p95 y tasa de errores antes/después sin sesgos de prueba.

10. Rúbrica de evaluación (escala 2 / 1 / 0)

Criterio	2 - Logrado	1 - Parcial	0 - No logrado
Diagnóstico	Métricas y evidencia reproducible	Datos incompletos	Sin medición
Paginación	Orden y límites correctos	Funciona con defectos	No implementa
Consultas	Reduce costo con evidencia	Optimiza sin comparar	Sin mejoras
Caché	TTL, claves e invalidación seguros	Invalidación incompleta	Insegura o ausente
Jobs	202, estados, idempotencia y errores	Faltan controles	Sin flujo válido
Informe	Comparación, análisis y conclusiones	Comparación parcial	Sin evidencia

Puntaje máximo: 12 puntos. Se recomienda exigir pruebas de regresión y preservación de permisos antes de aceptar una mejora.

11. Autoevaluación: respuestas orientativas

¿Mayor hit rate siempre equivale a mejor sistema? No: depende de frescura, privacidad y efectos de invalidación. ¿Un índice siempre acelera consultas? No: aumenta mantenimiento y puede no ser selectivo. ¿202 significa trabajo terminado? No. ¿Async elimina la necesidad de workers? No en procesos duraderos. ¿Cómo demostrar optimización? Midiendo bajo condiciones comparables.

12. Glosario y referencias

TTL: tiempo de vigencia de una entrada. Cache stampede: numerosas solicitudes recomputan simultáneamente una entrada caducada. N+1: consulta adicional por cada elemento listado. Keyset: paginación basada en clave de orden. Worker: proceso que ejecuta trabajos de una cola. Idempotencia: repetir una operación sin multiplicar sus efectos previstos.

• Django: https://docs.djangoproject.com/en/stable/topics/cache/

• Django QuerySet API: https://docs.djangoproject.com/en/stable/ref/models/querysets/

• Django REST Framework, Pagination: https://www.django-rest-framework.org/api-guide/pagination/

• PostgreSQL, Using EXPLAIN: https://www.postgresql.org/docs/current/using-explain.html

• Celery, User Guide: https://docs.celeryq.dev/en/stable/userguide/

• Redis documentation: https://redis.io/docs/latest/

• RFC 9110 HTTP Semantics: https://www.rfc-editor.org/rfc/rfc9110

Nota: el código ilustra patrones y requiere adaptación (autenticación, políticas de permisos, configuración de infraestructura y pruebas) para despliegues reales.

SERIE BACKEND PROFESIONAL

GUÍA DIDÁCTICA 09

Observabilidad

Logs estructurados · Métricas · Trazas · Alertas

Propósito: comprender cómo conocer el estado interno de un backend a partir de sus señales, detectar degradaciones, investigar errores y responder a incidentes con evidencia.

Destinatarios: estudiantes universitarios de Computación. Caso transversal: sistema de matrículas académicas construido con Python, Django REST Framework y PostgreSQL.

Resultados de aprendizaje: 1) distinguir logs, métricas y trazas; 2) instrumentar una operación HTTP; 3) definir indicadores y alertas accionables; 4) relacionar una incidencia con su causa probable sin exponer datos personales.

Versión: octubre de 2026 · Material para estudio, laboratorio y evaluación.

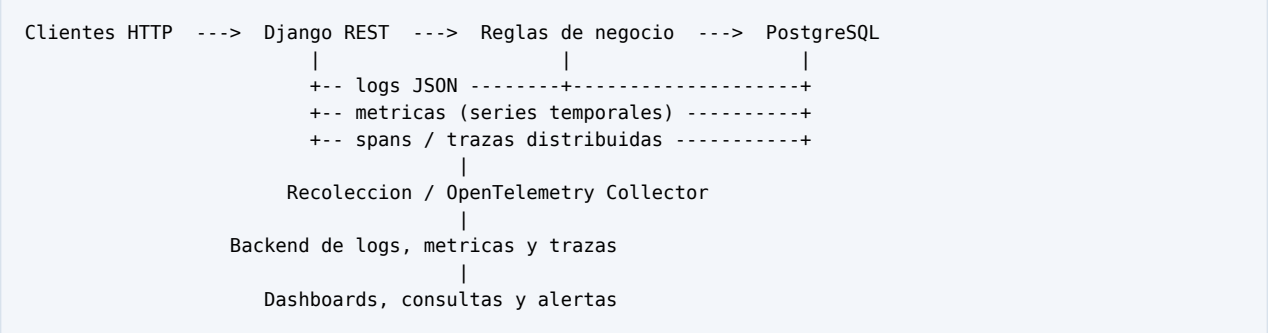
1. Introducción: monitoreo y observabilidad

Monitoreo consiste en observar indicadores conocidos y verificar condiciones esperadas. Observabilidad es la capacidad de inferir el estado interno del sistema a partir de señales externas, incluso para preguntas no previstas inicialmente.

Ejemplo: una API responde lentamente. Monitoreo informa «p95 de latencia alto»; observabilidad permite seguir la solicitud y descubrir que una consulta SQL bloqueada consume la mayor parte del tiempo.

Concepto	Pregunta que responde	Ejemplo
Logs	¿Qué evento ocurrió?	«Matrícula rechazada: sin cupos», con request_id
Métricas	¿Cuánto, con qué frecuencia y tendencia?	Tasa de errores 5xx y latencia p95
Trazas	¿Por dónde pasó esta solicitud?	API → servicio → PostgreSQL
Alertas	¿Hay que intervenir ahora?	Error rate sostenido y runbook asociado

Flujo conceptual de telemetría



La arquitectura concreta depende del volumen, el presupuesto y los requisitos de privacidad. En un monolito pequeño, instrumentar lo esencial antes de incorporar más componentes suele ser suficiente.

2. Logs estructurados

Un log registra un evento discreto. En vez de crear mensajes ambiguos en texto libre, es preferible emitir registros JSON con campos estables para consultar, filtrar y correlacionar.

Campos recomendados

Campo	Función
timestamp	Fecha/hora en UTC, preferiblemente ISO 8601
level	DEBUG, INFO, WARNING, ERROR, CRITICAL
service.name / environment	Origen del evento y entorno
event / message	Código estable de evento y descripción legible
request_id	Identificador de una solicitud dentro del backend
trace_id / span_id	Correlación con trazas distribuidas
duration_ms	Duración si el evento mide una operación

```
{
  "timestamp": "2026-10-13T14:00:00Z",
  "level": "WARNING",
  "service": "matriculas-api",
  "event": "matricula.cupo_agotado",
  "request_id": "req-7c42",
  "trace_id": "4bf92f3577b34da6a3ce929d0e0e4736",
  "course_id": 42,
  "message": "Solicitud rechazada por cupo agotado"
}
```

Reglas de higiene

- Nunca registrar contraseñas, JWT, cookies, tokens, números de identificación ni cuerpos completos con datos sensibles.
- Usar INFO para eventos normales relevantes; WARNING para situaciones recuperables; ERROR para fallos que requieren diagnóstico.
- Evitar crear un log por cada iteración de bucle; definir retención, redacción y acceso por roles.

3. Implementar logs en Python y Django

La biblioteca estándar logging permite registrar eventos. Para producción, configure un formateador JSON mantenido y añada metadatos de contexto mediante middleware o filtros de logging.

```
# services/matriculas.py
import logging
logger = logging.getLogger("matriculas")

def solicitar_matricula(estudiante_id, asignatura_id, cupos):
    if cupos <= 0:
        logger.warning(
            "matricula.cupo_agotado",
            extra={"asignatura_id": asignatura_id},
        )
        raise CupoAgotado("No existen cupos disponibles")
    logger.info("matricula.solicitud_validada")
    return {"estado": "pendiente"}
```

Precisión: el atributo `extra` agrega campos al `LogRecord`, pero el formato JSON requiere configuración adicional. No se debe asumir que un logger estándar imprime JSON automáticamente.

Contexto por solicitud

```
# Ejemplo conceptual de middleware
import uuid

class RequestIdMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        request.request_id = str(uuid.uuid4())
        response = self.get_response(request)
        response["X-Request-ID"] = request.request_id
        return response
```

El middleware muestra cómo generar un identificador; para incluirlo de forma segura en todos los logs conviene usar un context variable y limpiar su valor al finalizar cada solicitud. No reutilizar identificadores arbitrarios del cliente sin validarlos.

4. Métricas: medir el comportamiento

Las métricas agregan mediciones numéricas a lo largo del tiempo. Permiten identificar tendencias sin tener que consultar cada solicitud. Las dimensiones o etiquetas ayudan a segmentar resultados, pero no deben tener cardinalidad descontrolada.

Tipo	Significado	Ejemplo
Counter	Valor acumulativo creciente	Total de solicitudes HTTP
Gauge	Valor que sube o baja	Solicitudes activas
Histogram	Distribución de observaciones	Duraciones de solicitudes por intervalos

Indicadores básicos de un backend

- Tráfico: peticiones por segundo, distinguiendo rutas y métodos.
- Errores: porcentaje de respuestas 5xx; los 4xx de negocio pueden analizarse aparte.
- Latencia: p50, p95 y p99, explicando que p95 es el valor por debajo del cual cae aproximadamente el 95 % de observaciones.
- Saturación: uso de CPU, memoria, conexiones DB, longitud de colas y workers ocupados.

Ejemplo de métricas Prometheus

```
# Series ilustrativas en formato de exposicion Prometheus
http_requests_total{method="POST",route="/api/v1/matriculas",status="201"} 180
http_requests_total{method="POST",route="/api/v1/matriculas",status="500"} 4
http_request_duration_seconds_bucket{route="/api/v1/matriculas",le="0.5"} 160
```

En producción, use una biblioteca de instrumentación para gestionar contadores, buckets y metadatos correctamente. No etiquetar por `user_id`, `request_id` o URI con identificadores únicos: cada valor genera nuevas series.

5. Consultas PromQL y calidad de métricas

Prometheus almacena series temporales. PromQL permite convertir los contadores acumulados en tasas y proporciones útiles. Las siguientes expresiones son ejemplos para adaptar al nombre real de las métricas.

Solicitudes por segundo

```
sum(rate(http_requests_total[5m]))
```

Porcentaje de errores 5xx

```
100 * sum(rate(http_requests_total{status=~"5.."}[5m]))
      / clamp_min(sum(rate(http_requests_total[5m])), 0.001)
```

Latencia p95 estimada por histograma clásico

```
histogram_quantile(0.95,
  sum by (le) (rate(http_request_duration_seconds_bucket[5m]))
)
```

Al calcular p95 agregado con histogramas clásicos, debe preservarse la etiqueta `le`. Para histogramas nativos la consulta cambia. No calcular un p95 global promediando percentiles de distintos servidores.

Calidad de instrumentación

- Mantener nombres y unidades consistentes; segundos para duración y contadores para eventos.
- Etiquetar por plantillas de ruta, no por rutas concretas con IDs: `/matriculas/{id}`.
- Evitar que la tasa de errores parezca 0 cuando no hay tráfico; monitorizar ausencia de datos por separado.
- Definir qué se considera éxito desde la perspectiva del usuario, no solo desde HTTP.

6. Trazas distribuidas: seguir una solicitud

Una traza representa el recorrido lógico de una operación. Está compuesta por spans, que describen pasos con inicio, fin, atributos y relaciones padre–hijo. El contexto se propaga entre servicios, por ejemplo mediante encabezados W3C Trace Context.

```
Traza: POST /api/v1/matriculas                 total 480 ms
  +-- HTTP / Django                          480 ms
       +-- Validacion autenticacion           24 ms
       +-- Servicio matricular               435 ms
            +-- Consulta cupos PostgreSQL    300 ms  <-- cuello de botella
            +-- Escritura matricula           42 ms
            +-- Publicar evento               18 ms
```

Instrumentación manual conceptual con OpenTelemetry

```
from opentelemetry import trace

tracer = trace.get_tracer("matriculas.service")

def procesar_matricula(asignatura_id):
    with tracer.start_as_current_span("matricula.procesar") as span:
        span.set_attribute("matricula.asignatura_id", asignatura_id)
        validar_cupos(asignatura_id)
        return guardar_matricula(asignatura_id)
```

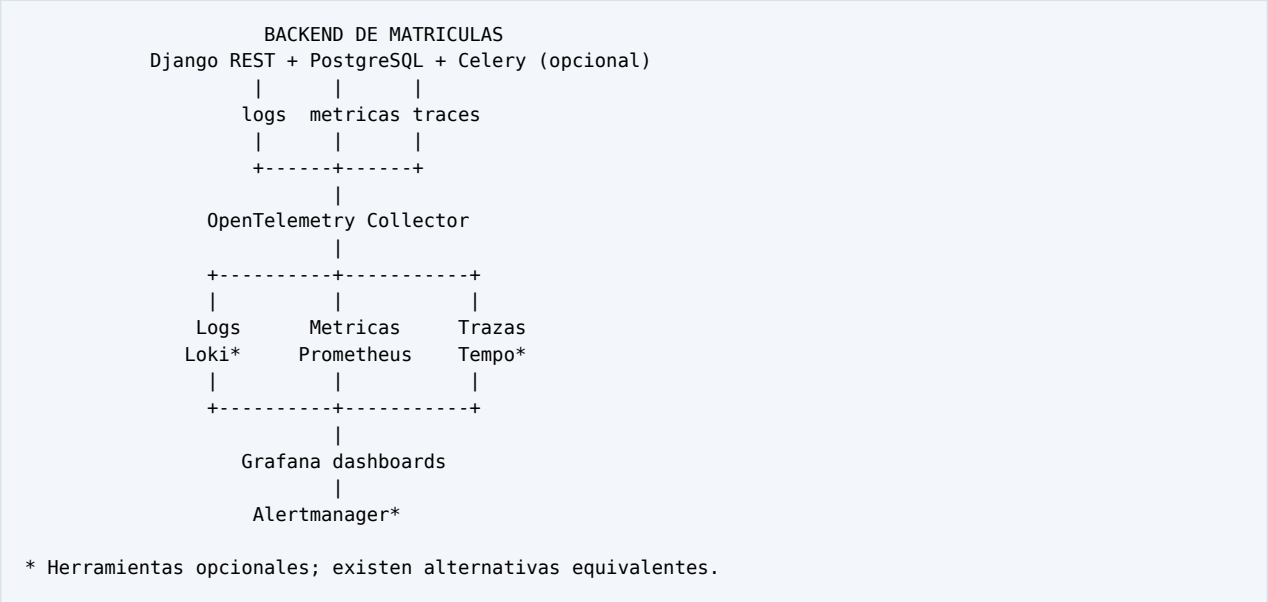
La exportación requiere configurar SDK, proveedor de trazas, exportador y, si aplica, un Collector. Use instrumentación automática de Django y PostgreSQL para reducir trabajo manual y evitar spans duplicados.

Muestreo y privacidad

No todas las trazas necesitan almacenarse: el sampling ayuda a controlar costes. No registrar datos personales en atributos de spans. Mantenga `trace_id` consistente entre logs y trazas para navegar del síntoma al detalle.

7. Arquitectura de observabilidad de referencia

Una arquitectura habitual separa producción, recolección, almacenamiento y visualización. OpenTelemetry unifica la instrumentación, pero no reemplaza por sí solo a un backend de almacenamiento y consultas.



Componentes y responsabilidades

Componente	Responsabilidad
Instrumentación	Emitir señales del backend
Collector	Recibir, procesar, filtrar y exportar telemetría
Backend de métricas	Almacenar y consultar series temporales
Backend de logs	Almacenar y consultar eventos estructurados
Backend de trazas	Buscar spans y recorridos completos
Dashboards y alertas	Visibilidad operativa y notificación accionable

8. Alertas: de una señal a una acción

Una alerta debe indicar un problema que merece intervención, no simplemente una anomalía estadística. La documentación de Prometheus recomienda alertar por síntomas cercanos al impacto en usuarios, acompañados por paneles que ayuden al diagnóstico.

Severidad	Criterio orientativo	Respuesta
Crítica	La API no permite completar matrículas	Notificación inmediata y procedimiento de incidente
Alta	Error 5xx sostenido o latencia que incumple SLO	Investigar y mitigar con prioridad
Informativa	Crecimiento de consumo sin impacto inmediato	Revisar en horario de mantenimiento

Ejemplo de regla Prometheus (ajustar umbrales)

```
groups:
  - name: backend_matriculas
    rules:
      - alert: TasaAltaErroresMatriculas
        expr: |
          sum(rate(http_requests_total{route="/api/v1/matriculas",
               status=~"5.."}[5m]))
          / clamp_min(sum(rate(http_requests_total{
               route="/api/v1/matriculas"}[5m])), 0.001) > 0.05
        for: 10m
        labels:
          severity: warning
        annotations:
          summary: "Errores 5xx superiores al 5 %"
          description: "Revisar dashboard y trazas de matrículas"
```

Importante: el 5 % es un umbral didáctico; no es universal. En un sistema con poco tráfico, complemente con un número mínimo de solicitudes y pruebas sintéticas para evitar falsas alarmas.

9. SLI, SLO, SLA y presupuestos de error

SLI es la medida de un aspecto del servicio (por ejemplo, proporción de solicitudes válidas exitosas). SLO es el objetivo de esa medida, en un período. SLA es un compromiso formal que puede incluir consecuencias contractuales.

Caso numérico: disponibilidad del módulo de matrículas

Suponga 100 000 solicitudes elegibles durante 30 días y 99 850 respuestas satisfactorias. La proporción de éxito es 99,85 %. Si el SLO es 99,9 %, el servicio incumple el objetivo en ese período.

```
SLI = solicitudes satisfactorias / solicitudes elegibles
SLI = 99 850 / 100 000 = 0,9985 = 99,85 %

SLO = 99,9 % -> presupuesto de error = 0,1 %
Errores permitidos sobre 100 000 solicitudes = 100
Errores observados = 150 -> excedente = 50
```

La población elegible debe especificarse: puede excluir rechazos esperados por reglas de negocio y peticiones inválidas. No debe excluir fallos reales para maquillar el SLI.

Burn rate

La tasa de consumo del presupuesto mide a qué velocidad se gasta el margen de error permitido. Un burn rate elevado sostenido en varias ventanas temporales puede justificar alertas más fiables que un único umbral puntual.

10. Laboratorio guiado: una matrícula lenta

Escenario: estudiantes reportan que crear una matrícula tarda más de tres segundos. El objetivo es instrumentar, reproducir, interpretar y proponer mejoras verificables.

Preparación

- Crear un endpoint `POST /api/v1/matriculas` en un proyecto Django de prueba con datos ficticios.
- Activar logs estructurados; añadir `request_id` y correlación `trace_id`.
- Instrumentar duración HTTP y consultas SQL; configurar exportación hacia un entorno local.
- Definir métricas para solicitudes totales, errores y latencia, con etiquetas de baja cardinalidad.

Secuencia de trabajo

Paso	Acción	Evidencia
1	Ejecutar 30 solicitudes en el entorno de pruebas	Tabla de tiempos y códigos
2	Consultar tasa de error y p95	Captura de panel con ventana definida
3	Localizar una traza con latencia alta	trace_id y spans relevantes
4	Cruzar traza y log usando correlación	Evento, ID y marca temporal
5	Optimizar la consulta o la regla costosa	Cambio en código y explicación
6	Repetir bajo condiciones comparables	Comparación antes/después

Entrega: README del laboratorio, capturas de métricas y trazas ficticias, archivo de consulta PromQL, regla de alerta y breve análisis causal. No incorporar credenciales ni datos reales de estudiantes.

11. Ejercicios de aplicación y autoevaluación

Ejercicios

- E1. Transforme «Error matriculando a Ana» en un log JSON respetuoso de la privacidad y con identificadores de correlación.
- E2. Diseñe tres métricas para la operación de crear matrículas. Justifique el tipo, la unidad y las etiquetas.
- E3. Dibuje una traza API → servicio de dominio → PostgreSQL y explique dónde buscaría un bloqueo de base de datos.
- E4. Redacte una alerta que detecte degradación persistente y un runbook de cinco pasos.
- E5. Calcule un SLI cuando 9 950 de 10 000 solicitudes elegibles fueron satisfactorias; compárelo con un SLO de 99,7 %.

Preguntas para discutir

- ¿Por qué logs, métricas y trazas no son intercambiables?
- ¿Por qué un identificador de estudiante no debe ser etiqueta de una métrica?
- ¿Cómo distinguir un rechazo por falta de cupo de un error 500 real?
- ¿Qué aporta un request_id si existe trace_id?
- ¿Qué riesgos introduce almacenar el cuerpo completo de solicitudes en los logs?

Orientaciones de solución

E1: registrar código de evento, tiempo, nivel, request_id y trace_id sin nombre. E2: contador HTTP, histograma de duración y gauge de solicitudes activas. E3: buscar span SQL prolongado y evidencias de bloqueos. E4: usar umbral, duración, impacto y procedimiento. E5: 99,5 %; está por debajo del objetivo 99,7 %.

12. Rúbrica de evaluación y lista de verificación

Criterio	2 — Logrado	1 — Parcial	0 — No logrado
Logs	JSON coherente, correlación y privacidad	Campos incompletos	Sin estructura o expone datos
Métricas	Instrumentos y etiquetas correctos	Instrumentación parcial	Métricas no interpretables
Trazas	Spans y causa evidenciados	Traza sin diagnóstico	No hay correlación
Alertas	Regla accionable y runbook	Solo umbral sin acción	No define respuesta
Análisis	Compara antes/después con evidencia	Comparación incompleta	Sin verificación

Puntuación máxima: 10 puntos. Puede convertirse a la escala institucional conforme a los criterios definidos por el docente.

Checklist de salida a producción

- Logs estructurados y centralizados, sin datos sensibles.
- Métricas de tráfico, errores, latencia y saturación.
- Trazas correlacionadas en rutas críticas.
- Etiquetas de métricas con cardinalidad controlada.
- Paneles con ventanas de tiempo y unidades claras.
- Alertas basadas en impacto, con responsables y runbooks.
- Retención, acceso y coste de telemetría definidos.
- Comprobación periódica de que la propia telemetría funciona.

13. Glosario

Término	Definición breve
Observabilidad	Inferir el estado del sistema mediante señales observables.
Log	Registro de un evento con contexto.
Métrica	Valor medible agregado a través del tiempo.
Traza	Conjunto de spans vinculados de una operación.
Span	Operación temporizada dentro de una traza.
Correlación	Vínculo entre telemetría de una misma operación.
Cardinalidad	Cantidad de combinaciones posibles de etiquetas.
SLI / SLO	Indicador medido / objetivo de ese indicador.
Runbook	Procedimiento operativo para resolver un incidente.
Burn rate	Velocidad de consumo del presupuesto de error.

Referencias y lecturas recomendadas

https://opentelemetry.io/docs/concepts/

https://opentelemetry.io/docs/concepts/semantic-conventions/

https://prometheus.io/docs/practices/alerting/

https://prometheus.io/docs/prometheus/latest/querying/basics/

https://sre.google/workbook/alerting-on-slos/

https://docs.djangoproject.com/en/stable/topics/logging/

Las configuraciones presentadas son pedagógicas y deben ajustarse a las versiones instaladas, protección de datos, políticas de seguridad y necesidades reales de disponibilidad.

SERIE BACKEND PROFESIONAL · GUÍA DIDÁCTICA 08

Documentación de un backend profesional

OpenAPI / Swagger · Arquitectura C4 · Registros de decisiones (ADR) · Guías técnicas

Guía de estudio para estudiantes universitarios de Computación

Propósito: aprender a producir documentación útil, verificable y mantenible durante todo el ciclo de vida de una API, no solo al terminar el desarrollo.

Caso transversal: Sistema de Gestión Académica con servicios de estudiantes, asignaturas y matrículas.
Tecnologías de referencia: Python, Django REST Framework, PostgreSQL, OpenAPI y Quarto.

Resultados de aprendizaje

- Diseñar una especificación OpenAPI que describa contratos, errores, autenticación y ejemplos.
- Distinguir y elaborar los niveles C4 con públicos y límites claros.
- Registrar decisiones relevantes mediante ADR y justificar sus consecuencias.
- Construir documentación de desarrollo, despliegue, operación y contribución reproducible.

Octubre de 2026 · Edición académica

1. ¿Qué es la documentación técnica?

Es el conjunto de artefactos que explican qué hace un sistema, cómo está construido, cómo se usa, cómo se modifica, cómo se despliega y por qué se tomaron decisiones. Una documentación de calidad es precisa, localizable, vigente, reproducible y adecuada a su audiencia.

Artefacto	Pregunta que responde	Destinatario
Referencia API	¿Cómo consumo el servicio?	Frontend, integradores
C4	¿Dónde están las partes y sus relaciones?	Arquitectos, desarrolladores
ADR	¿Por qué elegimos esta solución?	Equipo y responsables
README / guías	¿Cómo inicio, pruebo y despliego?	Nuevos integrantes y operaciones
Runbook	¿Qué hago ante una incidencia?	Operación y soporte

Documentación como código (Docs as Code)

Mantener los documentos junto al repositorio, con revisiones mediante pull requests, control de versiones y comprobaciones automáticas. No basta con generar páginas: los cambios de comportamiento deben actualizar ejemplos, contratos, diagramas y ADR cuando corresponda.

Ejemplo de regla de equipo

Una modificación de endpoint no se considera terminada sin actualización de OpenAPI, pruebas del contrato y notas de compatibilidad. Si cambia un límite entre módulos, se revisa el C4 y se registra un ADR si la decisión es significativa.

2. OpenAPI y Swagger: conceptos y diferencias

OpenAPI Specification (OAS) define un formato legible por máquinas para describir APIs HTTP. Incluye rutas, operaciones, parámetros, cuerpos, respuestas, esquemas y mecanismos de seguridad. Swagger es el nombre de herramientas de un ecosistema (por ejemplo, Swagger UI y Swagger Editor) que pueden visualizar o trabajar con una definición OpenAPI; no son sinónimos exactos.

Componentes principales de una especificación

• openapi: versión del lenguaje de especificación, distinta de la versión comercial de tu API.

• info: título, versión del contrato y descripción.

• servers: bases URL conocidas para los entornos.

• paths: operaciones y respuestas por recurso.

• components/schemas: estructuras reutilizables de datos.

• security: mecanismos exigidos, con excepciones documentadas.

Una distinción clave

La versión 3.1.x o 3.2.x de OpenAPI indica qué sintaxis soporta el archivo; una ruta /api/v1/ representa la versión funcional de una API. Son mecanismos diferentes.

3. Contrato HTTP: escribir antes de implementar

Un contrato es un acuerdo verificable entre proveedor y consumidores: método y ruta, tipos de entrada, restricciones, respuesta, errores, seguridad, comportamiento de paginación y compatibilidad. Debe incluir ejemplos representativos tanto exitosos como fallidos.

```
GET /api/v1/matriculas?estado=aprobada&page=1
Authorization: Bearer <token>
Accept: application/json

HTTP/1.1 200 OK
Content-Type: application/json

{
  "data": [{"id": 120, "estado": "aprobada"}],
  "meta": {"page": 1, "total": 1}
}
```

El parámetro estado filtra; page controla paginación; el token identifica al consumidor. El contrato debe aclarar si se permite consultar únicamente matrículas propias.

4. Especificación OpenAPI: ejemplo trabajado

El siguiente fragmento es introductorio y muestra una operación con parámetros, seguridad, esquema y respuestas. Se usa OpenAPI 3.1.0 por amplia compatibilidad de herramientas; al adoptar otra versión, verifica soporte en el generador y visor elegidos.

```
openapi: 3.1.0
info:
  title: API de Matriculas
  version: 1.0.0
servers:
  - url: https://api.ejemplo.edu/api/v1
paths:
  /matriculas/{id}:
    get:
      summary: Consultar matricula
      operationId: consultarMatricula
      security:
        - bearerAuth: []
      parameters:
        - name: id
          in: path
          required: true
          schema: {type: integer, minimum: 1}
      responses:
        "200":
          description: Matricula encontrada
          content:
            application/json:
              schema:
                $ref: "#/components/schemas/Matricula"
        "403": {description: Acceso prohibido}
        "404": {description: Recurso no encontrado}
components:
  securitySchemes:
    bearerAuth: {type: http, scheme: bearer}
  schemas:
    Matricula:
      type: object
      required: [id, estado]
      properties:

        id: {type: integer}
        estado:
          type: string
          enum: [borrador, pendiente, aprobada, rechazada]
```

¿Cómo comprobar el archivo?

- Validar su sintaxis y referencias con un linter compatible con la versión elegida.
- Abrirlo en un visor OpenAPI y comprobar ejemplos, tipos y seguridad.
- Realizar pruebas de contrato para verificar que el backend devuelve lo especificado.
- Publicar el documento junto a su versión de API, sin exponer secretos o datos reales.

5. Documentación automática en Django REST Framework

Una opción habitual es drf-spectacular, que extrae gran parte del esquema desde vistas y serializers. La generación automática debe completarse con descripciones, ejemplos, respuestas de error y comprobaciones manuales: el código no siempre revela toda la semántica del negocio.

```
# Fragmento orientativo de settings.py
INSTALLED_APPS += ["drf_spectacular"]
REST_FRAMEWORK = {
    "DEFAULT_SCHEMA_CLASS":
        "drf_spectacular.openapi.AutoSchema"
}
SPECTACULAR_SETTINGS = {
    "TITLE": "API Academica",
    "VERSION": "1.0.0",
}

# urls.py
from drf_spectacular.views import (
    SpectacularAPIView, SpectacularSwaggerView
)
urlpatterns += [
    path("api/schema/", SpectacularAPIView.as_view()),
    path("api/docs/", SpectacularSwaggerView.as_view(
        url_name="schema")),
]
```

Precaución: habilitar la documentación interactiva según la política de exposición de cada ambiente; Swagger UI no sustituye controles de autorización.

6. Arquitectura C4: explicar el sistema por niveles

El modelo C4 comunica arquitectura mediante vistas con distinto grado de detalle. Los niveles principales son Contexto, Contenedores, Componentes y Código. No se exige dibujar todos en cada proyecto: escoger las vistas que ayuden a responder preguntas concretas. Contenedor en C4 es una aplicación o almacén de datos desplegable/ejecutable, no necesariamente un contenedor Docker.

Nivel	Foco	Ejemplo académico
C1 · Contexto	Personas, sistema y dependencias externas	Estudiantes, docentes, SGA, servicio identidad
C2 · Contenedores	Aplicaciones y almacenes de datos	Frontend web, API Django, PostgreSQL
C3 · Componentes	Componentes dentro de un contenedor	MatriculasController, MatriculaService, Repository
C4 · Código	Clases e implementación seleccionada	Matricula, PoliticaCupos, repositorio

C1. Diagrama de contexto

Dibuja una caja para “Sistema de Gestión Académica”; fuera de ella, estudiante y docente como personas y un servicio institucional de identidad como sistema externo. Todas las flechas llevan verbos: “consulta”, “gestiona”, “autentica”. Evita dibujar tablas de base de datos aquí.

C2. Diagrama de contenedores

Dentro del sistema: aplicación web Angular (interfaz), API Django REST (reglas y endpoints) y PostgreSQL (persistencia). Expresa protocolos y tecnologías en los enlaces: “HTTPS/JSON” y “SQL/TCP”. Representa al proveedor de identidad fuera del límite del sistema.

C3. Diagrama de componentes

Amplía el contenedor API: adaptador HTTP, servicio de aplicación “InscribirEstudiante”, política de cupos, repositorio y cliente del servicio de identidad. El énfasis es qué responsabilidad tiene cada componente y sus dependencias, no el detalle de cada método.

C4. Vista de código

Reserva diagramas de clases o secuencia para partes difíciles; por ejemplo, la reserva atómica del último cupo. Mostrar todas las clases de Django suele aportar ruido en vez de claridad.

7. Ejemplo de C4 como texto (Structurizr DSL)

```
workspace "Gestion academica" "Arquitectura" {
  model {
    estudiante = person "Estudiante"
    sga = softwareSystem "Sistema academico" {
      web = container "Web" "Interfaz" "Angular"
      api = container "API" "Reglas" "Django REST"
      db = container "Base de datos" "Persistencia"
        "PostgreSQL"
      web -> api "Consume" "HTTPS/JSON"
      api -> db "Lee y escribe" "SQL"
    }
    estudiante -> sga "Gestiona sus matriculas"
  }
  views {
    systemContext sga "Contexto" { include *; autoLayout lr }
    container sga "Contenedores" { include *; autoLayout lr }
  }
}
```

El DSL anterior es un punto de partida didáctico. Comprueba la sintaxis con la versión de Structurizr que utilice el curso; si se incorpora una plataforma de identidad, debe añadirse como sistema externo al modelo.

8. Diagramas: reglas de comunicación

• Cada vista declara título, alcance, nivel, leyenda, fecha o versión y audiencia.

• Mantén nombres consistentes entre API, repositorio y diagramas.

• Una relación debe expresar dirección, propósito y, cuando agrega valor, protocolo.

• Distingue sistemas propios, personas, terceros y almacenes de datos.

• Actualiza diagramas cuando cambie la arquitectura, no por cambios cosméticos.

Errores de modelado frecuentes

Poner un microservicio, una clase y una tabla al mismo nivel; confundir un “contenedor” lógico de C4 con Docker; utilizar flechas sin etiqueta; dibujar detalles técnicos excesivos en C1; o mantener una imagen desconectada del código real.

9. ADR: registrar decisiones, no únicamente resultados

Un Architecture Decision Record documenta una decisión importante, el problema que la motivó y sus consecuencias. La plantilla concisa de Michael Nygard incluye título, estado, contexto, decisión y consecuencias. Se recomienda numerar los registros y conservar su historia; cuando se reemplaza una decisión, un ADR posterior puede marcar al anterior como superado.

ADR-0001: adoptar un monolito modular

Estado: Aceptado · Fecha: 2026-10-13 · Responsables: Equipo backend.

Contexto: el proyecto tiene un equipo pequeño, requiere entrega rápida y todavía no ha demostrado necesidad de despliegue independiente por módulo. Se consideran un monolito modular y microservicios.

Decisión: construir una API Django en un monolito modular con límites explícitos para identidad, matrículas y catálogos.

Consecuencias favorables: menos complejidad operativa, despliegue más simple y transacciones locales. Costos: mayor disciplina para evitar dependencias indebidas y menor independencia del despliegue entre módulos. Revisión: reconsiderar ante necesidades comprobables de escalado o autonomía.

Plantilla reutilizable

```
# ADR-XXXX: [decision en una frase]
Fecha: AAAA-MM-DD
Estado: Propuesto | Aceptado | Rechazado | Superado

## Contexto
[Problema, restricciones y alternativas]
## Decision
[Eleccion concreta y alcance]
## Consecuencias
[Beneficios, costos, riesgos y compromisos]
## Seguimiento
[Senales que obligan a revisar la decision]
```

10. Cuándo crear un ADR

Úsalo para elecciones significativas y difíciles de revertir: estrategia de autenticación, persistencia, consistencia de eventos, partición de módulos o patrón de despliegue. No es necesario crear un ADR para cada variable, función o ajuste menor. El objetivo es dejar trazable el porqué.

11. Guías técnicas que todo backend necesita

Documento	Contenido mínimo
README.md	Propósito, arquitectura breve, requisitos, inicio rápido, enlaces
CONTRIBUTING.md	Ramas, estilo, pruebas, revisiones, commits
Guía de desarrollo	Variables, entorno, comandos, semillas y depuración
Guía de despliegue	Entornos, pasos, migraciones, rollback y controles
Runbook de operación	Monitoreo, alertas, incidencias, recuperación
Guía de seguridad	Autorización, secretos, reporte de incidentes
Registro de cambios	Cambios funcionales y rupturas de compatibilidad

Ejemplo: inicio rápido comprobable

```
git clone https://ejemplo.invalid/academico-api.git
cd academico-api
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env
python manage.py migrate
python manage.py runserver
pytest
```

Las URL y comandos son ilustrativos: antes de publicar, sustituye el repositorio de ejemplo y verifica que el procedimiento funcione desde un entorno limpio. No publiques un .env con secretos reales.

12. Operación, seguridad y mantenimiento documental

- Describir variables de entorno por nombre, significado y obligatoriedad, nunca valores secretos.
- Documentar migraciones destructivas y procedimientos de restauración validados.
- Explicar cómo consultar logs por request_id o trace_id sin registrar datos sensibles.
- Establecer responsables, versión, fecha de revisión y enlaces a código y pruebas.
- Mantener documentación de errores, límites, cuotas, expiración de credenciales y rate limiting.

Indicadores de calidad documental

Cobertura de endpoints documentados; porcentaje de ejemplos verificados; antigüedad de diagramas; ADR con estado actualizado; tiempo que un integrante nuevo tarda en poner el sistema en marcha; frecuencia de errores de documentación detectados por consumidores.

13. Documentación y desarrollo guiado por especificaciones

En SDD, la documentación se convierte en insumo ejecutable y verificable. Un flujo razonable es: requisitos → escenarios → diseño C4 y ADR → contrato OpenAPI → implementación → pruebas de contrato → publicación. Esto no significa congelar las decisiones; cualquier cambio debe registrar su impacto sobre especificación, código y pruebas.

14. Laboratorio integrador: módulo de matrículas

Situación: una institución requiere que los estudiantes consulten e ingresen solicitudes de matrícula y que el personal autorizado las apruebe. El equipo debe documentar el módulo antes de implementarlo completamente.

Entregables exigidos

• Un README con instrucciones de ejecución reproducibles y enlaces al resto de artefactos.

• Un archivo openapi.yaml con GET /matriculas/{id}, POST /matriculas y sus respuestas 201, 400, 401, 403, 409.

• Diagramas C4 C1 y C2; C3 para el módulo de matrículas; C4 solo si añade valor.

• Un ADR sobre consistencia de cupos y otro sobre la organización modular.

• Una guía de operación para manejar caída de base de datos y fallo de migración.

• Una comprobación automática del esquema y al menos dos pruebas de contrato.

Actividad 1. Identificar vacíos

El equipo entrega un Swagger UI operativo, pero no explica permisos ni errores y tampoco indica cómo ejecutar migraciones. ¿Qué artefactos faltan? ¿Qué riesgos introduce esta omisión? Propón una lista de verificación mínima.

Actividad 2. Evaluar un diagrama

Un supuesto C1 muestra clases de Django, tablas SQL y rutas HTTP. Identifica el nivel correcto de esos elementos y redibuja la vista de contexto para un estudiante que no conoce el sistema.

Actividad 3. Redactar decisiones

Compara dos estrategias: reservar el cupo dentro de una transacción PostgreSQL frente a hacerlo mediante una cola asíncrona. Elabora un ADR que considere consistencia, latencia, operación y complejidad.

Actividad 4. Verificar un contrato

Dado un endpoint POST /matriculas que devuelve 200 en lugar de 201 y omite los errores 409, explica si el problema está en la API, en la documentación o en ambas, y crea un caso de prueba automatizado.

15. Rúbrica de evaluación (2 / 1 / 0)

Criterio	2 · Logrado	1 · Parcial	0 · No logrado
OpenAPI	Contrato completo y validado	Faltan casos o esquemas	Ausente o inválido
C4	Niveles coherentes y claros	Mezcla algunos niveles	Sin límites o relaciones
ADR	Decisión y costos justificados	Solo describe elección	No registra el porqué
Guías	Reproducibles y seguras	Pasos incompletos	No permiten ejecutar
Coherencia	Docs, código y tests alineados	Diferencias puntuales	Incompatibilidades graves

Calificación: máximo 10 puntos (cinco criterios × dos puntos). El docente puede añadir evidencias, ponderaciones y retroalimentación de acuerdo con el resultado de aprendizaje.

16. Autoevaluación y soluciones orientativas

Pregunta: ¿Swagger y OpenAPI son lo mismo?
Orientación: No. OpenAPI es una especificación; Swagger agrupa herramientas relacionadas.

Pregunta: ¿Qué significa “contenedor” en C4?
Orientación: Una aplicación o almacén de datos ejecutable o desplegable; no necesariamente Docker.

Pregunta: ¿Qué ventaja tiene un ADR?
Orientación: Conserva el contexto, los compromisos y las consecuencias de una decisión.

Pregunta: ¿Es suficiente generar OpenAPI a partir del código?
Orientación: No siempre: deben verificarse reglas, autorización, ejemplos y errores.

Pregunta: ¿Por qué documentar rollback?
Orientación: Porque los despliegues y migraciones pueden fallar y requieren recuperación segura.

17. Lista de control antes de publicar

- ☐ Contrato validado y ejemplos representativos correctos
- ☐ Códigos HTTP, autenticación, permisos y errores explícitos
- ☐ Diagramas C4 con límites, títulos y relaciones legibles
- ☐ ADRs numerados, fechados y con estado y consecuencias
- ☐ README y pasos de instalación probados en entorno limpio
- ☐ Sin contraseñas, tokens ni información personal en ejemplos
- ☐ Pruebas y documentación actualizadas en el mismo cambio
- ☐ Versión, mantenedor y procedimiento de actualización visibles

18. Glosario básico

- Contrato API: acuerdo de comportamiento entre servicio y consumidor.
- Esquema: descripción formal de estructura, tipos y restricciones.
- OpenAPI: especificación para describir interfaces HTTP.
- Swagger UI: interfaz visual para consultar APIs descritas con OpenAPI.
- C4: enfoque de diagramación de arquitectura por niveles de detalle.
- ADR: registro de una decisión arquitectónica y su contexto.
- Runbook: instrucciones operativas para procedimientos e incidentes.
- Docs as Code: documentación revisada y automatizada junto al código.

19. Referencias para profundizar

• OpenAPI Initiative. OpenAPI Specification. https://spec.openapis.org/oas/

• Swagger. Documentación de herramientas. https://swagger.io/tools/

• Simon Brown. C4 model, diagramas. https://c4model.com/diagrams

• Michael Nygard. Plantilla de Architecture Decision Record. https://architecture-decision-record.github.io/templates/decision-record-template-by-michael-nygard/

• Django REST Framework. Documenting your API. https://www.django-rest-framework.org/topics/documenting-your-api/

• drf-spectacular. Documentación. https://drf-spectacular.readthedocs.io/

• Structurizr. Documentación de DSL. https://docs.structurizr.com/dsl

• Diátaxis. Framework de documentación. https://diataxis.fr/

Nota: los fragmentos son ejemplos educativos; deben adaptarse, probarse y revisarse antes de utilizarlos en producción.

Pruebas de un Backend Profesional

Pruebas unitarias, de integración, de API, de seguridad y de regresión

Propósito. Comprender cómo diseñar, ejecutar y automatizar pruebas que demuestren el comportamiento esperado de un backend, detecten defectos y reduzcan el riesgo de modificarlo. Se usa como caso transversal una API de matrículas académicas implementada con Python, pytest, Django y Django REST Framework.

Resultados de aprendizaje. Al finalizar, el estudiante podrá: (1) clasificar pruebas por alcance y objetivo; (2) diseñar casos positivos, negativos y de frontera; (3) crear fixtures y pruebas parametrizadas; (4) probar API, permisos y transacciones; (5) configurar una suite reproducible en integración continua; y (6) interpretar cobertura y fallos con criterio técnico.

Requisitos previos. Python básico, HTTP/REST, SQL, principios SOLID, reglas de negocio y manejo elemental de Git. Los ejemplos son educativos y requieren ajustar nombres de módulos, URLs y modelos en un proyecto real.

1. ¿Qué significa probar software?

Una prueba de software ejecuta un sistema o parte de él bajo condiciones definidas y compara su comportamiento con un resultado esperado. Probar no significa “demostrar que nunca habrá errores”: significa obtener evidencia controlada de que requisitos, reglas y contratos se cumplen.

Ejemplo: si un curso tiene 30 cupos y ya posee 30 matrículas activas, un intento adicional debe rechazarse sin crear una matrícula. La prueba comprueba tanto la respuesta como el estado persistido.

1.1 Vocabulario esencial

Término	Definición y ejemplo
Caso de prueba	Conjunto de precondiciones, entradas, pasos y resultados esperados.
Aserción	Comparación automática: assert resultado == esperado.
Fixture	Datos y entorno de prueba reutilizables (usuario, curso, cliente HTTP).
Mock / stub	Sustituto controlado de dependencia externa, como servicio de correo.
Suite	Colección de pruebas ejecutables en conjunto.
Regresión	Defecto que reaparece o que introduce una modificación en una funcionalidad existente.

1.2 Patrón AAA: Arrange, Act, Assert

```
# Arrange: configurar datos y condiciones
cupos = 30
ocupados = 30
# Act: ejecutar la conducta
hay_cupo = ocupados < cupos
# Assert: comprobar resultado esperado
assert hay_cupo is False
```

Una prueba clara puede leerse como una especificación: “dado este contexto, cuando ocurre esta acción, entonces sucede este resultado”.

2. Pirámide de pruebas y estrategias de cobertura

Las pruebas rápidas y aisladas suelen ser numerosas; las pruebas con base de datos, servidor y servicios externos son más costosas. La conocida pirámide es una heurística, no una cuota fija: el equilibrio depende del riesgo, la arquitectura y el producto.

Capa	Qué verifica	Costo relativo
Unitarias	Reglas y funciones aisladas	Bajo
Integración	Interacción con ORM, BD o servicios	Medio
API / extremo a extremo	Rutas, serialización, permisos y persistencia	Medio / alto

Ortogonalmente, seguridad, regresión y rendimiento describen propósitos que pueden aplicarse en varias capas. Una prueba de autorización puede ser unitaria o ejecutarse contra la API real.

2.1 Elegir qué probar primero

- Priorizar reglas críticas: cupos, prerrequisitos, pagos, cambios de estado y permisos.
- Cubrir fronteras: 0, 1, límite exacto, límite + 1; valores vacíos, nulos y malformados.
- Incluir rutas de error: dependencia caída, objeto inexistente, conflicto y entrada inválida.
- Verificar efectos secundarios: datos creados, eventos emitidos, correos no enviados por error.

3. Pruebas unitarias

Una prueba unitaria evalúa una unidad de comportamiento —función, método o regla— sin depender de la red ni de una base de datos. Permite detectar rápidamente errores de negocio y facilita refactorizar.

3.1 Regla del dominio: matrícula permitida

```
# dominio/matriculas.py
class ReglaMatriculaError(Exception):
    pass

def validar_cupo(capacidad: int, activos: int) -> None:
    if capacidad <= 0:
        raise ReglaMatriculaError("CAPACIDAD_INVALIDA")
    if activos >= capacidad:
        raise ReglaMatriculaError("CUPO_AGOTADO")
```

3.2 Prueba positiva, negativa y de frontera

```
# tests/unit/test_cupos.py
import pytest
from dominio.matriculas import validar_cupo, ReglaMatriculaError

def test_acepta_si_hay_un_cupo():
    validar_cupo(capacidad=30, activos=29)

@pytest.mark.parametrize("activos", [30, 31])
def test_rechaza_cupo_lleno_o_excedido(activos):
    with pytest.raises(ReglaMatriculaError, match="CUPO_AGOTADO"):
        validar_cupo(capacidad=30, activos=activos)

def test_rechaza_capacidad_cero():
    with pytest.raises(ReglaMatriculaError, match="CAPACIDAD_INVALIDA"):
        validar_cupo(capacidad=0, activos=0)
```

Lectura: `parametrize` ejecuta un caso con varios valores. `raises` comprueba una excepción esperada. Conviene contrastar también el mensaje o código estable del error, no solo su clase.

3.3 Dobles de prueba y aislamiento

```
from unittest.mock import Mock

def notificar_matricula(mailer, correo):
    mailer.enviar(correo, "Matrícula confirmada")

def test_notificacion_invoca_proveedor():
    mailer = Mock()
    notificar_matricula(mailer, "alumno@ejemplo.edu")
    mailer.enviar.assert_called_once_with(
        "alumno@ejemplo.edu", "Matrícula confirmada"
    )
```

Advertencia: no simular todas las dependencias. Un exceso de mocks prueba detalles de implementación en lugar de comportamiento observable.

4. Pruebas de integración

Comprueban si componentes reales colaboran correctamente: servicio de aplicación, repositorio ORM, migraciones, restricciones SQL y transacciones. Necesitan una base de datos de prueba aislada y datos reproducibles.

4.1 Integración con Django ORM

```
# Ejemplo: adaptar nombres de modelos al proyecto.
import pytest
from django.db import IntegrityError, transaction
from academia.models import Asignatura

@pytest.mark.django_db
def test_codigo_de_asignatura_es_unico():
    Asignatura.objects.create(codigo="COM101", nombre="Programación")
    with pytest.raises(IntegrityError):
        with transaction.atomic():
            Asignatura.objects.create(
                codigo="COM101", nombre="Otra asignatura"
            )
```

El modelo debe declarar una restricción única real sobre `codigo`. La segunda creación debe fallar por integridad y no dejar datos inconsistentes.

4.2 Transacciones, aislamiento y concurrencia

Para cupos, una comprobación “leer cantidad y luego insertar” no basta bajo solicitudes simultáneas. Debe probarse la estrategia transaccional (por ejemplo, bloqueo de la fila del curso, actualización condicional o restricción que preserve el invariante). Las pruebas concurrentes necesitan conexiones y transacciones reales, usualmente con PostgreSQL, no solo SQLite en memoria.

- Preparar dos solicitudes para el último cupo; al finalizar debe haber exactamente una matrícula nueva.
- Comprobar que un fallo a mitad de operación revierte todas las escrituras involucradas.
- Ejecutar pruebas críticas con el mismo motor de BD que producción.

5. Pruebas de API y contrato HTTP

Validan al consumidor externo: ruta, método, parámetros, esquema JSON, código HTTP, encabezados y efectos en persistencia. No basta con que el endpoint responda 200: debe respetar el contrato acordado.

5.1 Prueba HTTP con APIClient

```
# Adaptar /api/v1/matriculas/ y modelos al proyecto.
import pytest
from rest_framework.test import APIClient

@pytest.mark.django_db
def test_usuario_anonimo_no_crea_matricula():
    client = APIClient()
    response = client.post("/api/v1/matriculas/", {
        "estudiante_id": 12, "asignatura_id": 8
    }, format="json")
    assert response.status_code in (401, 403)
```

El contrato de autenticación determina si se devuelve 401 o 403; la prueba definitiva debe exigir el estado especificado, no aceptar ambos indistintamente. El ejemplo admite los dos únicamente para ilustrar configuraciones posibles.

5.2 Matriz de escenarios de API

Escenario	HTTP esperado	Comprobación adicional
Creación válida	201 Created	Recurso persistido y Location si está definido
Datos inválidos	400 o 422 según contrato	Error por campo, no se creó el recurso
Sin autenticación	401 (según esquema)	Sin modificación de datos
Sin permiso	403 Forbidden	No expone contenido protegido
Objeto inexistente	404 Not Found	Respuesta estandarizada
Sin cupos	409 Conflict	No incrementa matrícula
Exceso de peticiones	429 Too Many Requests	Control definido de rate limit

5.3 Pruebas de esquema y versionado

Comparar respuestas reales con OpenAPI: nombres y tipos de campos, requisitos obligatorios, códigos documentados y compatibilidad de clientes. Un cambio de `nombre` a `nombre_completo` puede romper consumidores aun cuando las pruebas unitarias pasen.

```
# Ejemplo simplificado de contrato JSON
respuesta = {"id": 15, "estado": "aprobada"}
assert isinstance(respuesta["id"], int)
assert respuesta["estado"] in {"pendiente", "aprobada", "anulada"}
# En producción: usar validación contra JSON Schema/OpenAPI.
```

6. Pruebas de seguridad

Las pruebas de seguridad buscan evidencias de que el backend protege identidad, permisos, datos y recursos frente a usos no previstos. Se recomienda combinar revisiones de código, herramientas automatizadas y pruebas manuales autorizadas. Deben ejecutarse en un entorno de pruebas y con datos ficticios.

6.1 Autorización horizontal: objeto ajeno

```
@pytest.mark.django_db
def test_estudiante_no_ve_matricula_ajena(
    usuario_a, matricula_de_b
):
    client = APIClient()
    client.force_authenticate(user=usuario_a)
    url = f"/api/v1/matriculas/{matricula_de_b.pk}/"
    respuesta = client.get(url)
    assert respuesta.status_code in (403, 404)
```

En el sistema real se debe escoger un resultado contractualmente definido (403 o 404), exigirlo y verificar que el cuerpo no revele información de otro estudiante. Esta prueba detecta una clase frecuente de fallos de autorización por objeto (BOLA).

6.2 Controles que conviene verificar

Control	Pregunta de prueba
Autenticación	¿Se rechazan credenciales incorrectas y sesiones inválidas?
Autorización	¿Cada rol puede operar solo sobre sus recursos permitidos?
Validación	¿Se rechazan tipos, tamaños y campos no autorizados?
Masificación	¿Un usuario puede cambiar campos reservados como rol o estado?
Rate limiting	¿Los límites evitan abuso sin afectar flujos normales?
Errores seguros	¿No aparecen contraseñas, secretos ni trazas internas?
Auditoría	¿Se registra la acción sin guardar tokens ni datos sensibles?

Las categorías y pruebas de riesgo deben ajustarse a OWASP API Security Top 10 y a los requisitos del proyecto. No se deben probar sistemas ajenos sin autorización.

7. Pruebas de regresión

Una prueba de regresión verifica que un comportamiento que ya funcionaba sigue funcionando después de un cambio. Se recomienda agregar un caso automatizado para reproducir cada defecto corregido, antes de aplicar la solución.

7.1 Ciclo de corrección guiado por prueba

1. Reproducir el defecto mediante una prueba que falla. 2. Implementar la corrección mínima. 3. Confirmar que la nueva prueba pasa. 4. Ejecutar la suite completa. 5. Conservar la prueba para detectar futuras reapariciones.

Ejemplo: un usuario podía anular una matrícula ya finalizada. Se agrega un test que exige 409 para esa transición; luego se corrige la regla de estados y se conserva el test.

7.2 Diferencia entre smoke, regresión y aceptación

Tipo	Alcance
Smoke	Conjunto pequeño para saber si el despliegue es operable.
Regresión	Conjunto para descubrir efectos de cambios sobre comportamiento existente.
Aceptación	Verifica criterios de negocio previamente acordados.

8. Diseño de casos y calidad de las pruebas

8.1 Particiones de equivalencia y análisis de límites

Si el periodo admite entre 1 y 30 créditos, se prueban grupos representativos: valores válidos, por debajo y por encima del rango; además 0, 1, 30 y 31. Los valores fronterizos concentran defectos frecuentes.

```
import pytest

@pytest.mark.parametrize("creditos,es_valido", [
    (0, False), (1, True), (15, True),
    (30, True), (31, False)
])
def test_rango_creditos(creditos, es_valido):
    assert (1 <= creditos <= 30) is es_valido
```

8.2 Propiedades de una buena prueba

- Determinista: mismo código y entradas producen el mismo resultado.
- Independiente: no depende del orden de ejecución ni comparte estado sin control.
- Legible: nombre y aserciones describen una regla o conducta observable.
- Rápida cuando corresponde: las unitarias no arrancan innecesariamente la aplicación.
- Relevante: verifica resultados y efectos, no los detalles internos accidentales.

8.3 Antipatrones

Evitar pruebas que siempre pasan, múltiples comportamientos sin relación en un mismo test, tiempos de espera arbitrarios, conexión con servicios de producción, fixtures gigantes y aserciones demasiado permisivas.

9. Cobertura y métricas

Cobertura de líneas indica qué líneas se ejecutaron durante las pruebas; no demuestra que se hayan verificado todos los comportamientos. El 80 % puede ser una meta formativa inicial, nunca una garantía de ausencia de defectos.

```
# Instalación (en un entorno virtual)
python -m pip install pytest pytest-django pytest-cov
# Ejecutar suite y generar cobertura
pytest -q --cov=academia --cov-report=term-missing
# Generar reporte navegable
pytest --cov=academia --cov-report=html
```

Interpretación: una rama de autorización crítica no cubierta puede ser más grave que muchas líneas triviales sin pruebas. Complementar cobertura con revisión de casos de frontera, mutación de pruebas cuando sea viable y análisis de riesgos.

10. Organización del proyecto y fixtures

```
backend/
  academia/
    domain/
    services/
    models.py
    views.py
  tests/
    conftest.py
    unit/test_reglas.py
    integration/test_repositorios.py
    api/test_matriculas.py
    security/test_autorizacion.py
    regression/test_errores_corregidos.py
  pytest.ini
```

10.1 Configuración orientativa

```
# pytest.ini
[pytest]
DJANGO_SETTINGS_MODULE = proyecto.settings_test
python_files = test_*.py
addopts = -ra --strict-markers
markers =
    slow: pruebas más costosas
    security: pruebas de controles de seguridad
```

La configuración `settings_test` debe ser creada para el proyecto: BD independiente, servicios externos simulados y secretos ficticios. Nunca apuntar pruebas destructivas a producción.

10.2 Fixture reutilizable

```
# tests/conftest.py
import pytest
from rest_framework.test import APIClient

@pytest.fixture
def api_client():
    return APIClient()
```

11. Automatización en integración continua

La integración continua debe ejecutar validaciones por cada cambio propuesto y evitar fusionar código con pruebas críticas fallidas. Un flujo recomendable incluye calidad estática, migraciones verificadas, pruebas, cobertura y revisión de artefactos.

```
# .github/workflows/tests.yml (ejemplo conceptual)
name: backend-tests
on: [push, pull_request]
jobs:
  pytest:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: '3.12'
      - run: pip install -r requirements-dev.txt
      - run: pytest -q
```

En proyectos reales, revisar versiones vigentes de acciones, fijar permisos mínimos, provisionar una BD de prueba y evitar credenciales reales. El ejemplo solo demuestra la estructura del flujo.

12. Laboratorio integrador: API de matrículas

Situación: la universidad requiere que un estudiante solicite matrícula en una asignatura. El servicio acepta estudiantes habilitados, respeta cupos y prerrequisitos, evita duplicados y exige permisos apropiados.

12.1 Reglas de aceptación

- R1. Un estudiante habilitado puede solicitar matrícula si existen cupos.
- R2. No es posible matricularse dos veces en la misma asignatura y periodo.
- R3. Si no hay cupo, se devuelve un error de negocio consistente, sin inserción.
- R4. Un estudiante no puede consultar ni modificar la matrícula de otro.
- R5. Las operaciones concurrentes no pueden exceder la capacidad.

12.2 Plan de pruebas que debe entregar el estudiante

Identificador	Nivel	Caso	Resultado esperado
UT-01	Unitario	capacidad 20, activos 19	Permite
UT-02	Unitario	capacidad 20, activos 20	Rechaza
IT-01	Integración	matrícula duplicada	Restricción/negocio impide duplicado
API-01	API	solicitud válida	201 y recurso creado
API-02	API	sin cupo	409 y datos intactos
SEC-01	Seguridad	acceso a matrícula ajena	403 o 404 según contrato
CON-01	Integración	dos solicitudes al último cupo	Solo una confirma
REG-01	Regresión	anulación de matrícula finalizada	Conflicto sin cambio de estado

12.3 Evidencias requeridas

- Código fuente y estructura de carpetas de pruebas.
- Salida de pytest con número de tests correctos y fallidos.
- Reporte de cobertura interpretado críticamente.
- Matriz de pruebas con requisito, datos, resultado esperado y resultado real.
- Explicación de al menos un defecto encontrado y su corrección con prueba de regresión.

13. Actividades de aprendizaje

Actividad A — Identificación (individual)

Para diez escenarios proporcionados por el docente, clasificar el nivel de prueba y justificar si requiere mocks, BD, autenticación o red. Entregar tabla con razón técnica.

Actividad B — Implementación (parejas)

Construir cinco pruebas unitarias parametrizadas para cupos y prerrequisitos, con casos positivos, negativos y de frontera. Ejecutar pytest y explicar las aserciones.

Actividad C — Integración y seguridad (equipos)

Implementar pruebas contra la BD y una API DRF para evitar duplicados y acceso horizontal indebido. Probar que, tras un error, el estado de los datos no cambia.

Actividad D — Regresión y CI

Introducir deliberadamente un fallo en una regla de estados, crear primero una prueba que lo reproduzca, corregirlo y añadir el flujo de CI. Documentar el antes y después.

14. Rúbrica de evaluación (2 / 1 / 0)

Criterio	2 — Logrado	1 — Parcial	0 — No logrado
Diseño	Casos cubren reglas, errores y límites	Faltan escenarios relevantes	No relaciona pruebas con requisitos
Unitarias	Aisladas, claras y deterministas	Dependencias o aserciones débiles	No ejecutan o no verifican
Integración/API	Verifica BD, HTTP y efectos	Solo valida parte del flujo	No contrasta resultados
Seguridad	Comprueba permisos por objeto y errores	Pruebas incompletas	No prueba acceso indebido
Regresión	Reproduce fallo y previene reaparición	Caso poco robusto	Sin prueba de regresión
Evidencias	Suite, reporte y explicación completos	Documentación incompleta	Sin evidencias reproducibles

Calificación sugerida: 12 puntos máximos; puede transformarse proporcionalmente a la escala de la asignatura. La rúbrica debe comunicarse antes de iniciar el laboratorio.

15. Autoevaluación

- ¿Por qué una cobertura del 100 % no demuestra que un backend sea correcto?
- ¿Qué diferencia hay entre una prueba unitaria y una de integración?
- ¿Cuándo es apropiado usar un mock y cuándo podría ocultar errores?
- ¿Qué debe comprobar una prueba HTTP además del código de estado?
- ¿Cómo se verifica un acceso horizontal indebido entre estudiantes?
- ¿Qué cambia al probar una carrera por el último cupo con PostgreSQL?
- ¿Qué defecto nuevo conviene convertir inmediatamente en una prueba de regresión?
- ¿Cómo se garantiza el aislamiento de los datos durante la ejecución de pytest?

16. Glosario y fuentes de consulta

Fixture: preparación reutilizable del contexto. Mock: doble programable que registra interacciones. Stub: doble que devuelve respuestas predeterminadas. CI: integración continua. Regresión: reaparición o introducción de un defecto tras un cambio. Contrato: comportamiento externo acordado. Cobertura: proporción instrumentada de código ejercitado por pruebas.

Fuentes oficiales y de consulta:

https://docs.pytest.org/en/stable/

https://pytest.org/en/latest/how-to/fixtures.html

https://pytest.org/en/latest/how-to/parametrize.html

https://www.django-rest-framework.org/api-guide/testing/

https://docs.djangoproject.com/en/stable/topics/testing/

https://owasp.org/www-project-api-security/

https://owasp.org/www-project-web-security-testing-guide/

https://coverage.readthedocs.io/

Nota didáctica. Los fragmentos son ilustrativos: los nombres de clases, rutas y modelos deben adaptarse al proyecto. La calificación de respuestas HTTP y el alcance de las pruebas deben derivarse del contrato API acordado.

SERIE BACKEND PROFESIONAL | GUÍA DIDÁCTICA 06

Gestión de errores en aplicaciones backend

Excepciones · Respuestas estandarizadas · Trazabilidad

Documento de estudio para estudiantes de Computación. Incluye fundamentos, criterios de diseño, ejemplos ejecutables de referencia, pruebas y una práctica integradora de gestión académica.

Ficha académica	Descripción
Propósito	Diseñar errores comprensibles para clientes, seguros para usuarios y rastreables para el equipo técnico.
Prerrequisitos	HTTP y REST; JSON; Python; nociones de Django REST Framework.
Resultado de aprendizaje	Implementa un catálogo de errores, respuestas Problem Details, manejo centralizado y correlación de registros.
Escenario	Módulo de matrícula académica: cupos, estudiantes, prerrequisitos y servicios externos.

Idea central: un error bien diseñado no es simplemente un mensaje: es parte del contrato de la API y de la capacidad operativa del sistema.

Convención de ejemplos: los fragmentos son pedagógicos y deben ajustarse a la estructura real del proyecto, pruebas y requisitos institucionales.

1. Objetivos y mapa conceptual

• Identificar la diferencia entre un error esperado de dominio y un fallo técnico no previsto.

• Clasificar los errores y asignarles estados HTTP coherentes.

• Diseñar respuestas interoperables bajo RFC 9457.

• Centralizar el tratamiento de excepciones sin ocultar fallos.

• Correlacionar una petición con logs, métricas y trazas.

• Verificar errores, seguridad y observabilidad mediante pruebas.

Cadena de gestión de errores

Etapa	Responsabilidad
Cliente	Envía la solicitud y recibe un contrato de error estable.
Controlador / API	Valida formato, autentica, autoriza y transforma excepciones conocidas en respuestas HTTP.
Aplicación y dominio	Detectan conflictos y reglas incumplidas; emiten errores semánticos.
Infraestructura	Reporta fallos de base de datos, redes, colas y servicios externos.
Observabilidad	Registra el evento técnico con contexto seguro y correlación.

Ejemplo: «El grupo ya no tiene cupos» es un conflicto esperado, no un fallo inesperado del servidor. «No se pudo conectar a PostgreSQL» sí es un fallo de infraestructura; el cliente recibe una respuesta genérica mientras el servidor conserva la evidencia técnica.

2. Conceptos esenciales

Concepto	Definición y ejemplo
Error	Condición por la que una operación no puede completarse como se esperaba.
Excepción	Mecanismo del lenguaje para interrumpir el flujo y comunicar una condición: raise ValueError(...).
Error de dominio	Restricción de negocio incumplida: cupo agotado o prerrequisito pendiente.
Fallo técnico	Dependencia caída, fallo de programación, tiempo agotado.
Propagación	Una excepción asciende hacia capas superiores hasta que se gestione.
Captura	Bloque except que maneja específicamente una excepción.
Trazabilidad	Posibilidad de relacionar una solicitud con registros y eventos internos.

3. Taxonomía de errores

Diferencias por origen y respuesta

Categoría	Ejemplo	Respuesta habitual
Datos inválidos	Campo semestre ausente o fuera de rango	400 o 422, según contrato
No autenticado	Token ausente o inválido	401 + WWW-Authenticate si aplica
Sin autorización	Usuario intenta anular matrícula ajena	403 o 404 por política de divulgación
No encontrado	Identificador inexistente	404
Conflicto de estado	Cupo agotado; matrícula ya confirmada	409
Límite de frecuencia	Demasiadas solicitudes	429 + posible Retry-After
Dependencia no disponible	Servicio externo temporalmente caído	503 + posible Retry-After
Fallo inesperado	Error de programación sin controlar	500; mensaje público genérico

Importante: 400 y 422 no son intercambiables por capricho: defina en el contrato una política consistente. Un error sintáctico suele asociarse a 400; 422 puede representar contenido bien formado que no puede procesarse según sus instrucciones.

Diferenciar error de usuario y fallo del sistema

• El cliente debe saber si puede corregir su entrada, autenticarse, dejar de repetir la operación o reintentar más tarde.

• El servidor debe conservar contexto suficiente para diagnosticar la causa sin exponer información confidencial.

• Un mismo código HTTP puede admitir diversos códigos de negocio: CUPOS_AGOTADOS, MATRICULA_DUPLICADA.

4. Excepciones de Python: uso correcto

```
class ReglaNegocioError(Exception):
    """Condición prevista dentro del dominio."""

class CuposAgotados(ReglaNegocioError):
    pass

def verificar_cupos(disponibles: int) -> None:
    if disponibles <= 0:
        raise CuposAgotados("No existen cupos disponibles")

try:
    verificar_cupos(0)
except CuposAgotados as exc:
    print(f"Regla de negocio incumplida: {exc}")
```

Antipatrón: no capture indiscriminadamente `Exception` para devolver 200 con `success:false`; el cliente, los monitores y la infraestructura interpretarían una falsa operación exitosa.

5. Diseño de un catálogo institucional de errores

Un catálogo documenta un identificador de error, su significado, estado HTTP, condiciones y remedios. Evita que cada módulo invente una respuesta diferente.

Código	HTTP	Cuándo sucede	Acción cliente
MATRICULA_DUPLICADA	409	Inscripción vigente ya registrada	Consultar matrícula actual
CUPOS_AGOTADOS	409	No quedan cupos	Elegir otro grupo
PRERREQUISITO_PENDIENTE	422	No cumple condición académica	Revisar prerequisitos
ESTUDIANTE_NO_ENCONTRADO	404	Entidad no existe o no es visible	Verificar recurso
DEPENDENCIA_NO_DISPONIBLE	503	Servicio requerido inaccesible	Reintentar según política

Buenas prácticas

• No utilice el mensaje visible como identificador de programa; use un código estable y documentado.

• Defina un dueño y una versión de cada error; evite cambiar su significado silenciosamente.

• Incluya ejemplos en OpenAPI, con cuerpos y encabezados.

• No filtre información sobre existencia de recursos que el usuario no tiene derecho a conocer.

6. Respuestas estandarizadas: RFC 9457

El estándar Problem Details for HTTP APIs define un objeto JSON común para representar errores con el tipo de contenido `application/problem+json`. Sustituye a RFC 7807.

Miembro	Significado
type	URI que identifica el tipo de problema; about:blank si no hay tipo específico.
title	Resumen breve y estable del tipo de problema.
status	Estado HTTP expresado numéricamente.
detail	Explicación específica de esta ocurrencia.
instance	URI que identifica esta ocurrencia (opcional).

```
HTTP/1.1 409 Conflict
Content-Type: application/problem+json
X-Request-ID: 2c61112f-1f7b-4fd1-ae25-214273b3c886

{
  "type": "https://api.ejemplo.edu/problems/cupos-agotados",
  "title": "Cupos agotados",
  "status": 409,
  "detail": "El grupo 23 no dispone de cupos.",
  "instance": "/api/v1/matriculas/solicitudes/783",
  "code": "CUPOS_AGOTADOS",
  "request_id": "2c61112f-1f7b-4fd1-ae25-214273b3c886"
}
```

Nota: `code` y `request_id` son miembros de extensión, no miembros base del RFC. El dominio api.ejemplo.edu es ilustrativo.

7. Del dominio al contrato HTTP

La entidad de dominio no debería depender de clases HTTP. Un adaptador de API traduce excepciones semánticas a respuestas públicas.

```
# dominio/errores.py
class ErrorDominio(Exception):
    codigo = "REGLA_INCUMPLIDA"
    mensaje_publico = "No se pudo completar la operación"

class CuposAgotados(ErrorDominio):
    codigo = "CUPOS_AGOTADOS"
    mensaje_publico = "El grupo no dispone de cupos"

# aplicacion/matriculas.py
def matricular(estudiante_id, grupo, repositorio):
    if grupo.cupos_disponibles == 0:
        raise CuposAgotados()
    return repositorio.inscribir(estudiante_id, grupo.id)
```

La capa API decidirá el estado 409. En un proyecto real, la verificación de cupos debe realizarse dentro de una transacción segura ante concurrencia; la comprobación mostrada es solo conceptual.

8. Manejo centralizado con Django REST Framework

DRF incluye un `exception_handler` configurable. Gestiona sus excepciones API, Http404 y PermissionDenied; otras excepciones pueden no producir una respuesta directa desde el manejador.

```
# api/exceptions.py
from rest_framework.views import exception_handler
from rest_framework.response import Response
from dominio.errores import CuposAgotados


def manejar_excepcion(exc, context):
    if isinstance(exc, CuposAgotados):
        request = context.get("request")
        return Response({
            "type": "https://api.ejemplo.edu/problems/cupos-agotados",
            "title": "Cupos agotados", "status": 409,
            "detail": exc.mensaje_publico, "code": exc.codigo,
            "request_id": getattr(request, "request_id", None),
        }, status=409, content_type="application/problem+json")

    return exception_handler(exc, context)
```

```
# settings.py
REST_FRAMEWORK = {
    "EXCEPTION_HANDLER": "api.exceptions.manejar_excepcion"
}
```

Alcance: el fragmento personaliza un error de dominio. Para estandarizar también los errores de DRF, hay que transformar sus respuestas, respetar encabezados relevantes y configurar un manejo seguro de 500 fuera del flujo de excepciones conocido. No basta con instalar el manejador.

9. Errores de validación: campos y estructura

Las fallas de validación deben explicar al cliente qué campo incumple una regla, sin revelar secretos ni lógica interna excesiva.

```
HTTP/1.1 400 Bad Request
Content-Type: application/problem+json

{
  "type": "https://api.ejemplo.edu/problems/validacion",
  "title": "Datos inválidos",
  "status": 400,
  "detail": "Revise los campos indicados.",
  "code": "VALIDACION_FALLIDA",
  "errors": [
    {"field": "periodo", "code": "required",
     "message": "Este campo es obligatorio"},
    {"field": "creditos", "code": "min_value",
     "message": "Debe ser un número positivo"}
  ]
}
```

El arreglo `errors` es una extensión de la aplicación. Para clientes multilingües, resulta conveniente conservar códigos estables y traducir mensajes visibles.

10. Seguridad de los mensajes de error

Mostrar al cliente	Mantener solo en servidor
Código HTTP y error público	Stack trace y ruta de archivos
Mensaje breve, reparable	Cadenas de conexión y secretos
Identificador de correlación	SQL detallado y datos internos
Reintento recomendado	Tokens y cabeceras de autenticación

- Deshabilitar DEBUG en producción.
- No incluir valores de contraseñas, tokens, identificadores sensibles ni cargas completas en logs.
- Sanitizar entradas y reducir datos de contexto a lo estrictamente necesario.
- No convertir errores 500 en mensajes descriptivos de infraestructura para el cliente.

11. Políticas de reintentos y resiliencia

No todos los errores justifican un reintento. Repetir un POST no idempotente puede duplicar matrículas o pagos.

Estado	Comportamiento prudente
400 / 401 / 403 / 404 / 422	Corregir solicitud, credenciales o permisos; no insistir sin cambios.

Estado	Comportamiento prudente
409	Reconsultar estado y resolver el conflicto; no reintentar a ciegas.
429	Esperar conforme a Retry-After si existe; limitar frecuencia.
503 / 504	Reintento acotado con backoff y jitter si la operación es segura o idempotente.

12. Trazabilidad: identificar una petición de extremo a extremo

Una solicitud puede atravesar gateway, API, servicios y base de datos. La correlación permite reconstruir su recorrido sin compartir el mensaje técnico completo con el usuario.

Identificadores clave

Identificador	Uso
request_id	ID por solicitud; puede devolverse al cliente para soporte.
trace_id	Identifica una traza que comprende varios servicios y operaciones.
span_id	Identifica una operación concreta dentro de la traza.
job_id	Identifica un trabajo asíncrono; no sustituye a trace_id.

Flujo de ejemplo: Cliente → API Gateway → Django REST Framework → Servicio de matrículas → PostgreSQL. Todos los eventos relacionados deben poder consultarse por identificadores o contexto propagado.

W3C Trace Context y OpenTelemetry

La propagación distribuida utiliza habitualmente el encabezado `traceparent`. OpenTelemetry ayuda a instrumentar logs, métricas y trazas; la creación de IDs y su propagación deben verificarse en la infraestructura real.

```
GET /api/v1/matriculas/783 HTTP/1.1
traceparent: 00-4bf92f3577b34da6a3ce929d0e0e4736-
  00f067aa0ba902b7-01

# Ejemplo conceptual de registro estructurado
{
  "level": "ERROR",
  "event": "matricula.persistencia_fallida",
  "request_id": "req-7a1d",
  "trace_id": "4bf92f3577b34da6a3ce929d0e0e4736",
  "module": "matriculas",
  "duration_ms": 207
}
```

La segunda línea del encabezado es continuación visual; en HTTP real el valor completo se envía en una sola línea.

13. Logs estructurados y niveles

Nivel	Ejemplo y criterio
DEBUG	Detalles de diagnóstico; desactivar o restringir en producción.
INFO	Inicio/fin de operaciones relevantes sin información confidencial.
WARNING	Error previsto o degradación recuperable.
ERROR	Operación fallida que requiere análisis.
CRITICAL	Fallo grave que compromete el servicio o datos.

14. Métricas, alertas y diagnósticos

• Indicadores: tasa de errores por ruta y clase HTTP; latencia p95; fallos de dependencias; excepciones no controladas.

• Alertas: actuar ante incremento sostenido de 5xx, errores de conexión o saturación; ajustar umbrales a la línea base.

• Trazas: distinguir tiempo de API, consultas y servicios externos.

• Cardinalidad: no usar IDs de estudiante como etiquetas de métricas; eleva costes y exposición.

Ejemplo de registro con logging estándar

```
import logging
logger = logging.getLogger(__name__)

def registrar_fallo(request_id, codigo):
    logger.warning(
        "regla_de_negocio_incumplida",
        extra={"request_id": request_id,
               "error_code": codigo, "module": "matriculas"}
    )
```

Nota: el formateador del logger debe configurarse para emitir los campos de extra en JSON. El ejemplo por sí solo no produce necesariamente un log estructurado.

15. Manejo de errores en operaciones asíncronas

En procesos largos, la API puede responder 202 Accepted con un job_id. El error ocurre en el worker y se consulta mediante un estado persistido del trabajo.

```
POST /api/v1/reportes -> 202 Accepted
{"job_id":"job-583", "status":"pending"}

GET /api/v1/jobs/job-583 -> 200 OK
{
  "job_id":"job-583",
  "status":"failed",
  "error": {
    "code":"GENERACION_REPORTE_FALLIDA",
    "message":"El reporte no pudo generarse"
  }
}
```

El GET del trabajo puede devolver HTTP 200 aunque su estado interno sea failed: la consulta del recurso job se completó, pero el proceso descrito por ese recurso falló. No se deben confundir ambas capas.

16. Antipatrones frecuentes

• Retornar 200 OK para cualquier excepción.

• Capturar Exception en todas las capas y ocultar el problema.

• Exponer trazas de pila y contraseñas en JSON.

• Usar 500 para toda validación o conflicto de negocio.

- Registrar solo mensajes planos sin IDs de correlación.
- Reintentar indefinidamente sin límites ni idempotencia.

17. Laboratorio: matrícula académica segura ante errores

Objetivo: ampliar la API de matrículas implementada en las guías anteriores para ofrecer errores estandarizados y observables.

Caso de trabajo

Un estudiante solicita inscribirse en un grupo. La operación puede fallar porque el grupo no existe, ya está matriculado, incumple prerrequisitos, no hay cupos o la base de datos no responde.

Actividades secuenciadas

Paso 1. Crear un catálogo de cinco errores, con código de negocio, HTTP y acción sugerida.

Paso 2. Implementar excepciones de dominio independientes de DRF.

Paso 3. Configurar un manejador HTTP central que emita application/problem+json.

Paso 4. Configurar IDs de petición y registro técnico seguro (sin tokens ni datos personales).

Paso 5. Escribir casos de prueba: un éxito, tres errores esperados y un fallo inesperado.

Paso 6. Documentar los esquemas y ejemplos en OpenAPI y verificar manualmente con Postman.

Pruebas orientativas con pytest + DRF

```
def test_cupos_agotados_devuelve_409(api_client, grupo_lleno):
    respuesta = api_client.post(
        "/api/v1/matriculas",
        {"grupo_id": grupo_lleno.id}, format="json"
    )
    assert respuesta.status_code == 409
    assert respuesta["Content-Type"].startswith(
        "application/problem+json"
    )
    assert respuesta.data["code"] == "CUPOS_AGOTADOS"
```

Este ejemplo presupone fixtures, autenticación y un endpoint implementado; no es una prueba autónoma. Complementar con pruebas de 401, 403, 404, 400, 500 y de no divulgación.

18. Casos para razonar

- ¿Por qué cupos agotados no debería generar HTTP 500?
- ¿Qué diferencia hay entre un 403 y un 404 por razones de privacidad?
- ¿Qué campos del error son contrato público y cuáles pertenecen únicamente al log?
- ¿Cuándo un cliente puede reintentar un 503 sin duplicar operaciones?
- ¿Por qué job.status=failed no implica que GET /jobs/{id} retorne 500?

19. Rúbrica de evaluación 2 / 1 / 0

Criterio	2: Logrado	1: Parcial	0: No logrado
Clasificación	Distingue reglas y fallos técnicos	Presenta algunas confusiones	No distingue
Contrato	RFC 9457 y códigos coherentes	Formato o estados incompletos	Sin estandarización
Implementación	Excepciones centralizadas	Manejo duplicado parcialmente	Sin manejo adecuado
Seguridad	No expone datos técnicos	Algunos riesgos de exposición	Filtra datos sensibles
Trazabilidad	Correlación útil y logs seguros	Correlación incompleta	Sin evidencia
Pruebas	Prueba positivos y negativos	Casos insuficientes	No existen pruebas

Puntuación máxima: 12 puntos. Se puede transformar a escala institucional sin modificar los criterios. Se sugiere solicitar evidencia de ejecución de tests y capturas de las respuestas.

20. Lista de control antes de publicar una API

• Todos los errores documentados tienen estado HTTP y código de negocio coherentes.

• Existe un formato público consistente y no se filtran datos privados.

• Los errores inesperados devuelven 500 genérico y generan una evidencia de diagnóstico interna.

• La correlación funciona entre API y dependencias instrumentadas.

• Se documentan los casos 4xx / 5xx más relevantes en OpenAPI.

• Las pruebas garantizan que los permisos no se convierten en filtraciones.

• Las reglas de reintento e idempotencia están especificadas.

21. Bibliografía y lecturas complementarias

• IETF. RFC 9457 (2023). Problem Details for HTTP APIs. https://www.rfc-editor.org/rfc/rfc9457.html

• IETF. RFC 9110 (2022). HTTP Semantics. https://www.rfc-editor.org/rfc/rfc9110.html

• Django REST Framework. Exceptions. https://www.django-rest-framework.org/api-guide/exceptions/

• W3C. Trace Context. https://www.w3.org/TR/trace-context/

• OpenTelemetry. Documentation. https://opentelemetry.io/docs/

• OWASP. Error Handling Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Error_Handling_Cheat_Sheet.html

Cierre: errores previsibles deben ser útiles para el consumidor; fallos imprevistos deben ser seguros para el usuario y diagnosticables para operación.

SERIE BACKEND PROFESIONAL | GUÍA 05

Reglas de negocio

Servicios de dominio, validaciones y gestión de estados

Guía didáctica universitaria con conceptos, antiejemplos, diseño modular, Python, Django REST Framework, pruebas y laboratorio integrador.

Caso transversal: sistema de matrícula académica. Un estudiante solicita matrícula en una asignatura; el sistema verifica prerrequisitos, cupos, períodos y estados de la solicitud.

Resultados de aprendizaje

- Diferenciar reglas de negocio, validaciones de entrada y restricciones de persistencia.
- Diseñar entidades, objetos de valor y servicios de dominio sin acoplarlos al transporte HTTP.
- Modelar un ciclo de vida mediante transiciones e invariantes verificables.
- Implementar casos de uso transaccionales y pruebas que protejan decisiones críticas.

Prerequisitos: nociones de clases y funciones en Python, HTTP/REST y bases de datos relacionales.

1. ¿Qué son las reglas de negocio?

Son condiciones, políticas y decisiones propias del dominio que determinan qué operaciones son válidas y cómo debe comportarse el sistema. Responden al qué se permite y por qué, no a cómo se dibuja una pantalla o se almacena una fila.

Ejemplo cotidiano

Una solicitud de matrícula solo puede aprobarse cuando el período está abierto, el estudiante cumple los prerrequisitos, existe cupo y no tiene ya una matrícula activa equivalente. Una API puede recibir un JSON perfectamente formado y, aun así, la operación debe rechazarse por motivos de negocio.

Categoría	Ejemplo	Responsable habitual
Validación de entrada	El identificador de asignatura es un entero positivo.	Serializer / esquema de entrada
Regla de negocio	No matricular si el período está cerrado.	Dominio / caso de uso
Integridad de datos	No permitir matrícula duplicada para alumno y oferta.	Restricción UNIQUE en BD
Política de seguridad	El docente no puede aprobar solicitudes de otra unidad.	Autorización / permisos

Lenguaje ubicuo

Utiliza términos estables compartidos entre docentes, estudiantes y desarrolladores: oferta académica, solicitud, cupo, matriculado, anulado. Evita nombrar conceptos de negocio según detalles técnicos como fila_estado_2.

Ejercicio de clasificación

Clasifica las siguientes afirmaciones en entrada, dominio, persistencia o seguridad: «correo con sintaxis válida», «máximo 30 cupos», «solo coordinación autoriza una excepción» y «la clave de una oferta debe ser única».

2. Dónde ubicar cada responsabilidad

En un backend organizado por capas, cada decisión debe tener un hogar claro. El objetivo es evitar duplicaciones y dependencias accidentales.

Capa	Responsabilidad	No debería hacer
API / presentación	Parsea HTTP, autentica, valida formato y serializa la respuesta.	Implementar directamente todos los cálculos y reglas.
Aplicación	Orquesta el caso de uso y la transacción, llama al dominio y puertos.	Depender de detalles de la interfaz de usuario.
Dominio	Protege invariantes, políticas y transiciones de entidades.	Importar request, Response o ORM por necesidad.
Infraestructura	Implementa repositorios, correo, reloj, BD y mensajería.	Decidir arbitrariamente reglas académicas.

Flujo de una petición

```
POST /api/v1/matriculas
  -> MatriculaViewSet (HTTP / permiso)
  -> SolicitarMatricula (caso de uso)
  -> PoliticaMatricula / Matricula (dominio)
  -> RepositorioMatriculas (persistencia)
  -> Respuesta HTTP
```

¿Servicio de dominio o servicio de aplicación?

• Servicio de dominio: expresa una decisión de negocio que no pertenece naturalmente a una sola entidad; por ejemplo, la elegibilidad basada en múltiples condiciones.

• Servicio de aplicación: coordina la operación: obtiene recursos, solicita una decisión, persiste resultados y publica un evento.

• Entidad: protege el estado y sus transiciones: una matrícula anulada no puede pasar directamente a aprobada.

3. Entidades, objetos de valor e invariantes

Una entidad conserva identidad a través del tiempo; un objeto de valor representa un concepto mediante sus atributos y suele ser inmutable. Una invariante es una regla que debe mantenerse verdadera en todo estado válido.

Ejemplos de invariantes

• El número de ocupantes de una oferta no supera su capacidad.

• Una solicitud anulada no se aprueba.

• Un estudiante no tiene dos matrículas activas de la misma oferta.

• Una matrícula aprobada debe referenciar una oferta existente.

```
from dataclasses import dataclass

@dataclass(frozen=True)
class Cupo:
    capacidad: int
    ocupados: int

    def __post_init__(self):
        if self.capacidad < 0 or not (0 <= self.ocupados <= self.capacidad):
            raise ValueError("Cupo inconsistente")

    @property
    def disponibles(self) -> int:
        return self.capacidad - self.ocupados
```

Observa: el constructor impide crear valores inválidos. Esto no sustituye el bloqueo o control de concurrencia cuando dos peticiones compiten por el último cupo.

Evita el modelo anémico

Si todas las entidades son simples contenedores de campos y todas las reglas están en los controladores, el código se vuelve difícil de mantener. Sitúa el comportamiento donde pueda preservar invariantes, sin forzar toda decisión a una clase.

4. Validaciones: cuatro niveles complementarios

Nivel	Qué verifica	Ejemplo
Sintáctico	Formato y tipo	`periodo_id` es entero; fecha ISO 8601.
Semántico	Significado y rango	La fecha final no antecede a la inicial.
Negocio	Políticas institucionales	No aprobar sin cumplir prerrequisitos.
Persistencia	Integridad bajo concurrencia	UNIQUE, FOREIGN KEY, CHECK y transacciones.

Ejemplo Django REST Framework

```
from rest_framework import serializers

class SolicitudMatriculaSerializer(serializers.Serializer):
    oferta_id = serializers.IntegerField(min_value=1)
    observacion = serializers.CharField(
        required=False, allow_blank=True, max_length=300
    )
```

El serializer comprueba estructura y límites. No debería convertirse en el único lugar donde se exige que una oferta esté abierta: esa regla también debe protegerse cuando se ejecutan procesos masivos, comandos o tareas asíncronas.

Errores de dominio expresivos

```
class ReglaDeNegocioError(Exception):
    codigo = "regla_negocio"

class PeriodoCerrado(ReglaDeNegocioError):
    codigo = "periodo_cerrado"

class SinCupo(ReglaDeNegocioError):
    codigo = "sin_cupo"
```

La traducción a HTTP corresponde al borde de la aplicación. Por ejemplo, una solicitud conflictiva puede devolver 409, con un código de error estable y un mensaje seguro.

5. Servicios de dominio

Un servicio de dominio reúne una decisión de negocio cuando esta requiere información de distintas entidades o no encaja con claridad en una entidad individual. Debe tener entradas explícitas y resultados comprobables.

```
from dataclasses import dataclass

@dataclass(frozen=True)
class ContextoMatricula:
    periodo_abierto: bool
    cumple_prerrequisitos: bool
    tiene_matricula_activa: bool
    cupos_disponibles: int

class PoliticaMatricula:
    def validar(self, contexto: ContextoMatricula) -> None:
        if not contexto.periodo_abierto:
            raise PeriodoCerrado("El periodo no admite matriculas")
        if not contexto.cumple_prerrequisitos:
            raise ReglaDeNegocioError("Faltan prerrequisitos")
        if contexto.tiene_matricula_activa:
            raise ReglaDeNegocioError("Matricula duplicada")
        if contexto.cupos_disponibles <= 0:
            raise SinCupo("No hay cupos disponibles")
```

Qué gana el diseño

- Las reglas quedan visibles y pueden probarse sin levantar el servidor HTTP.
- La misma política se reutiliza desde API, panel administrativo o proceso por lotes.
- La política puede evolucionar sin alterar la estructura del request.

Cuidado con las reglas configurables

Por ejemplo, el máximo de créditos por ciclo puede cambiar según reglamento, carrera y período. No conviene dispersarlo como números mágicos. Define una política versionada o configurable y registra su vigencia.

6. Gestión de estados: máquinas de estados

Los estados describen situaciones válidas del proceso. Las transiciones definen cambios permitidos y sus precondiciones. Una máquina de estados explícita evita que cualquier actualización directa cambie el campo estado.

Ciclo de vida de la solicitud

```
BORRADOR --enviar--> PENDIENTE --aprobar--> APROBADA
                         |                     |
                         +--rechazar--> RECHAZADA
                         |
                         +--anular----> ANULADA

APROBADA --anular (si procede)--> ANULADA
```

Estas transiciones son un ejemplo didáctico: una institución puede definir otros estados o condiciones. No deben confundirse con disposiciones normativas reales.

Origen	Acción	Destino	Guarda o condición
BORRADOR	enviar	PENDIENTE	Datos completos
PENDIENTE	aprobar	APROBADA	Elegibilidad y cupo confirmado
PENDIENTE	rechazar	RECHAZADA	Motivo registrado
PENDIENTE	anular	ANULADA	Usuario autorizado
APROBADA	anular	ANULADA	Normativa y período lo permiten

Preguntas de diseño

- ¿Qué estados son finales y cuáles permiten reversión?
- ¿Qué actor puede ejecutar cada transición?
- ¿Qué evento de negocio se registra al cambiar el estado?
- ¿Cómo se manejan dos transiciones simultáneas de la misma solicitud?

7. Implementación de transiciones seguras

```
from enum import Enum
from dataclasses import dataclass

class Estado(str, Enum):
    BORRADOR = "borrador"
    PENDIENTE = "pendiente"
    APROBADA = "aprobada"
    RECHAZADA = "rechazada"
    ANULADA = "anulada"

TRANSICIONES = {
    (Estado.BORRADOR, "enviar"): Estado.PENDIENTE,
    (Estado.PENDIENTE, "aprobar"): Estado.APROBADA,
    (Estado.PENDIENTE, "rechazar"): Estado.RECHAZADA,
    (Estado.PENDIENTE, "anular"): Estado.ANULADA,
    (Estado.APROBADA, "anular"): Estado.ANULADA,
}

@dataclass
class Matricula:
    estado: Estado

    def transicionar(self, accion: str) -> None:
        destino = TRANSICIONES.get((self.estado, accion))
        if destino is None:
            raise ReglaDeNegocioError("Transicion no permitida")
        self.estado = destino
```

Importante: esta versión se enfoca en la validez del grafo de estados. La autorización, las condiciones particulares de la transición y la persistencia atómica deben implementarse en las capas correspondientes.

Antiejemplo

```
# Evitar: cualquier cliente puede imponer cualquier estado
solicitud.estado = request.data["estado"]
solicitud.save()
```

En su lugar, expón acciones expresivas como aprobar o anular y comprueba permisos y guardas antes del cambio.

8. Caso de uso y transacciones

El caso de uso coordina repositorios y servicios, mientras la base de datos garantiza atomicidad. Verificar cupos y luego incrementar ocupación sin coordinación puede causar sobreventa.

Patrón transaccional con Django

```
from django.db import transaction

@transaction.atomic
def aprobar_solicitud(solicitud_id: int):
    solicitud = (Solicitud.objects.select_for_update()
                 .get(pk=solicitud_id))
    oferta = (Oferta.objects.select_for_update()
              .get(pk=solicitud.oferta_id))

    if solicitud.estado != "pendiente":
        raise ReglaDeNegocioError("Estado no aprobable")
    if oferta.ocupados >= oferta.capacidad:
        raise SinCupo("Oferta sin cupos")

    # Comprobar autorizacion y restantes guardas de dominio
    solicitud.estado = "aprobada"
    oferta.ocupados += 1
    oferta.save(update_fields=["ocupados"])
    solicitud.save(update_fields=["estado"])
```

Alcance: ejemplo simplificado para PostgreSQL; en sistemas reales hay que comprobar permisos fuera y dentro del flujo apropiado, establecer un orden consistente de bloqueos, proteger duplicados con restricciones y diseñar la reversión del cupo al anular. `select_for_update()` no brinda las mismas garantías en SQLite.

Eventos y efectos secundarios

Si después de aprobar debes enviar correo o publicar un mensaje, evita un efecto irreversible antes del commit. Usa `transaction.on_commit()` para efectos simples; para entrega confiable entre sistemas considera el transactional outbox.

9. Contratos de los casos de uso y API

El contrato debe expresar una intención: aprobar matrícula en lugar de permitir actualizaciones indiscriminadas del estado.

```
POST /api/v1/matriculas/125/aprobar
Authorization: Bearer <token>
If-Match: "version-4"

HTTP/1.1 200 OK
ETag: "version-5"
Content-Type: application/json

{"data": {"id": 125, "estado": "aprobada"}}
```

La cabecera ETag es opcional: debe implementarse realmente una versión del recurso y validación condicional para que `If-Match` tenga efecto. Un servidor puede devolver 412 Precondition Failed al no coincidir.

Situación	Código HTTP razonable	Comentario
Solicitud aprobada	200 OK	Devuelve representación
Entrada mal formada	400 Bad Request	Error de estructura
Sin autenticación	401 Unauthorized	No existe identidad válida
Sin permiso	403 Forbidden	El actor carece de autorización
Matrícula inexistente	404 Not Found	Recurso no localizado
Transición inválida / sin cupo	409 Conflict	Choque con estado actual
ETag desactualizado	412 Precondition Failed	Concurrencia condicional

Idempotencia

Aprobar dos veces no debería descontar el cupo dos veces. Define la respuesta para reintentos y, si corresponde, usa claves de idempotencia en operaciones con efectos no repetibles.

10. Pruebas de reglas de negocio

La pirámide de pruebas comienza por unidades del dominio, sigue con integración de base de datos y culmina con contratos HTTP y casos completos.

```
import pytest

def test_no_aprueba_periodo_cerrado():
    contexto = ContextoMatricula(
        periodo_abierto=False,
        cumple_prerrequisitos=True,
        tiene_matricula_activa=False,
        cupos_disponibles=1,
    )
    with pytest.raises(PeriodoCerrado):
        PoliticaMatricula().validar(contexto)

def test_no_permite_transicion_ilegal():
    m = Matricula(estado=Estado.RECHAZADA)
    with pytest.raises(ReglaDeNegocioError):
        m.transicionar("aprobar")
```

Matriz de casos que debe cubrir el laboratorio

Caso	Esperado
Período cerrado	No se crea ni aprueba matrícula
Sin prerrequisitos	Rechazo con código de negocio
Cupo = 0	Rechazo; ocupados no aumenta
Matrícula activa duplicada	Se mantiene unicidad
Estado rechazado + aprobar	Transición no permitida
Dos solicitudes por último cupo	Solo una aprobación consistente
Usuario sin permiso	403, sin cambios persistidos
Reintento de la misma aprobación	No consume otro cupo

11. Taller integrador: matrícula académica

Escenario. Diseña un módulo que reciba solicitudes de matrícula y permita aprobarlas o rechazarlas según condiciones institucionales simuladas.

Actividades secuenciales

• Actividad 1 — Descubrimiento: redacta 10 reglas en lenguaje claro e identifica su fuente o supuesto.

• Actividad 2 — Modelo: identifica entidades (Estudiante, Oferta, Solicitud), objetos de valor e invariantes.

• Actividad 3 — Estados: dibuja la máquina y elabora una tabla de transiciones con guardas y actores.

• Actividad 4 — Código: implementa PolíticaMatricula, Matricula.transicionar y el caso de uso AprobarSolicitud.

• Actividad 5 — Persistencia: implementa restricciones e integra `transaction.atomic` y bloqueo apropiado.

• Actividad 6 — API: define `POST /matriculas/{id}/aprobar` y una respuesta de error estructurada.

• Actividad 7 — Pruebas: automatiza escenarios válidos, inválidos, concurrentes y reintentos.

Entregables

• Diagrama de clases simplificado + máquina de estados + matriz de reglas.

• Código de dominio, aplicación, infraestructura y API en carpetas diferenciadas.

• Suite de pruebas y reporte breve de resultados.

• README con decisiones técnicas, supuestos, instrucciones y limitaciones.

Definición de terminado

Una función está terminada cuando protege sus invariantes aun sin interfaz web, documenta sus transiciones, verifica autorización, es transaccional cuando afecta múltiples filas y presenta pruebas automatizadas de resultados y errores.

12. Evaluación y autoevaluación

Rúbrica analítica: 2 / 1 / 0

Criterio	2 — Logrado	1 — Parcial	0 — No logrado
Modelado	Reglas, entidades e invariantes coherentes	Conceptos con omisiones	Sin distinción clara
Separación	Dominio independiente de HTTP/ORM	Dependencias parciales	Lógica mezclada
Estados	Transiciones y guardas completas	Faltan guardas	Estado editable libremente
Integridad	Transacciones y restricciones apropiadas	Protección incompleta	Riesgo de inconsistencias
Pruebas	Cubre casos límite, errores y reintentos	Solo camino feliz	No presenta evidencia

Preguntas para discusión

- ¿Por qué una validación de serializer no garantiza por sí sola una regla institucional?
- ¿Cuándo una operación debería pertenecer a una entidad y cuándo a un servicio de dominio?
- ¿Qué cambia si dos usuarios intentan ocupar el último cupo a la vez?
- ¿Qué diferencia existe entre transición inválida y acceso no autorizado?
- ¿Qué evidencias permiten demostrar que una regla sigue vigente después de refactorizar?

Respuestas orientativas

1) Porque otros puntos de entrada pueden omitir el serializer. 2) En la entidad, si protege su estado; en un servicio, cuando integra una decisión transversal. 3) Se necesita control de concurrencia y restricción en BD. 4) La primera viola el proceso; la segunda, los permisos. 5) Pruebas, contratos, trazabilidad de requisitos y revisión de cambios.

13. Antipatrones y recomendaciones

Antipatrón	Consecuencia	Alternativa
Controlador gigante	Reglas acopladas a HTTP	Caso de uso y dominio separados
Estado editable por PATCH	Se saltan guardas	Acciones explícitas
Constantes mágicas	Políticas ocultas	Configuración versionada
Validar solo en frontend	Bypass sencillo	Validar en backend y BD
Solo pruebas felices	Errores no detectados	Casos negativos y concurrencia
Eventos antes del commit	Notificaciones inconsistentes	on_commit o outbox

Relación con SDD

En Spec-Driven Development conviene describir cada regla con identificador, origen, condición, resultado esperado, excepciones y casos de prueba. Por ejemplo:

```
RN-MAT-04: No superar capacidad de la oferta
Dado: oferta con capacidad = ocupados
Cuando: se intenta aprobar una solicitud pendiente
Entonces: se rechaza con codigo "sin_cupo"
Y: no cambian solicitud ni contador de ocupados
Pruebas: dominio + integracion + concurrencia
```

Bibliografía de referencia

• Evans, E. (2003). Domain-Driven Design: Tackling Complexity in the Heart of Software. Addison-Wesley.

• Fowler, M. (2002). Patterns of Enterprise Application Architecture. Addison-Wesley.

• Vernon, V. (2013). Implementing Domain-Driven Design. Addison-Wesley.

• Documentación Django: Transactions — docs.djangoproject.com/en/stable/topics/db/transactions/

• Documentación Django REST Framework: Serializers / Permissions — www.django-rest-framework.org/

• PostgreSQL Documentation: Transaction Isolation — www.postgresql.org/docs/current/transaction-iso.html

BACKEND PROFESIONAL

Guía didáctica 04
Seguridad del backend

Autenticación · Autorización · Validación · Protección OWASP

Material de estudio para estudiantes universitarios de Computación. Incluye conceptos, comparaciones, ejemplos comentados en Django REST Framework, laboratorio guiado, evaluación y referencias técnicas.

Contexto transversal	Sistema de gestión académica
Tecnologías utilizadas	Python · Django · Django REST Framework · PostgreSQL
Enfoque	Defensa en profundidad, mínimo privilegio y comprobación con pruebas
Fecha	Octubre de 2026

Objetivo de aprendizaje. Diseñar, justificar, implementar y probar controles de seguridad en un backend, diferenciando identificación, autenticación, autorización y validación.

1. ¿Qué significa asegurar un backend?

La seguridad del backend consiste en proteger los datos y los procesos del sistema frente a accesos indebidos, modificaciones no autorizadas, pérdida de disponibilidad y divulgación de información. No es un middleware aislado: atraviesa API, dominio, persistencia, configuración y operación.

Propiedad	Pregunta que orienta el diseño
Confidencialidad	¿Quién puede consultar estos datos?
Integridad	¿Quién puede modificarlos y qué restricciones deben cumplirse?
Disponibilidad	¿Cómo evitamos que un usuario o un proceso agote los recursos?
Autenticidad	¿Cómo comprobamos la identidad del sujeto o del servicio?
Trazabilidad	¿Podemos investigar cambios sensibles sin exponer secretos?

Defensa en profundidad

Una solicitud atraviesa varios controles complementarios. Si falla uno, los restantes deben limitar el daño. No debe confiarse en que el frontend o un único middleware protejan los datos.

```
Cliente -> HTTPS -> proxy/rate limit -> autenticacion
        -> permisos de endpoint -> permisos de objeto
        -> validacion -> reglas de negocio -> base de datos
        -> respuesta minima + auditoria
```

Modelo de amenazas sencillo

- Activo: expedientes de matrícula y calificaciones.
- Amenaza: un estudiante altera el identificador de otra matrícula para leerla.
- Control: consulta filtrada por propietario y/o autorización por objeto.
- Verificación: prueba automática con dos usuarios y registros distintos.

2. Identificación, autenticación y autorización

Concepto	Qué pregunta responde	Ejemplo
Identificación	¿Quién dices ser?	Correo institucional o nombre de usuario
Autenticación	¿Puedes demostrarlo?	Contraseña, passkey o segundo factor
Autorización	¿Qué puedes hacer?	Docente registra notas de su curso
Auditoría	¿Qué ocurrió?	Registro de cambio de calificación

Situación: iniciar sesión correctamente no autoriza a cambiar la nota de cualquier estudiante. La identidad autenticada es la entrada para evaluar políticas específicas sobre la acción y el recurso.

Factores de autenticación

- Conocimiento: algo que sabes, como una contraseña.
- Posesión: algo que tienes, como un autenticador o clave criptográfica.
- Inherencia: algo que eres, como biometría utilizada de forma apropiada.

La autenticación multifactor usa factores independientes. Dos contraseñas no constituyen dos factores. Las passkeys resistentes al phishing pueden mejorar la protección cuando el entorno las admite.

3. Sesiones, tokens y contraseñas

Una sesión tradicional utiliza una cookie vinculada a estado del servidor; un token de acceso es una credencial que el servidor verifica. Ambas opciones requieren diseño cuidadoso: expiración, revocación, protección contra robo y transporte por HTTPS.

Mecanismo	Fortaleza	Riesgo / decisión
Cookie de sesión	Integración natural con Django	HttpOnly, Secure, SameSite y defensa CSRF
Token bearer	Útil para APIs y clientes diversos	Quien lo posea puede usarlo; reducir vida y exposición
OIDC / OAuth 2.0	Identidad federada / autorización delegada	Validar emisor, audiencia, firma y flujos permitidos

Contraseñas: buenas prácticas

- Nunca guardar contraseñas en texto plano ni cifrarlas como si fuesen datos recuperables.
- Usar funciones de hash especializadas y sal, por ejemplo Argon2id según el contexto y soporte.
- Aplicar limitación de intentos, recuperación segura, MFA según riesgo y mensajes que eviten enumerar usuarios.
- No registrar contraseñas ni tokens en logs.

```
# Django: ejemplo conceptual de uso de API de contrasenas
from django.contrib.auth import get_user_model
User = get_user_model()
usuario = User.objects.create_user(
    username="ana", password="frase-secreta-larga")
# Django almacena un hash, no la contrasena original.
```

Importante: el ejemplo usa una contraseña ficticia para enseñanza; en producción no se incrustan credenciales en el código.

4. Autorización: roles, atributos y objetos

Un backend puede combinar RBAC (roles), ABAC (atributos de usuario, recurso y contexto) y reglas de acceso a nivel de objeto. Los roles simplifican la administración, pero no sustituyen el análisis del recurso solicitado.

Sujeto	Acción	Regla ilustrativa
Estudiante	Ver matrícula	Solo matrículas de su titular
Docente	Registrar calificación	Solo asignaturas y paralelos asignados
Dirección	Consultar reporte agregado	Según ámbito institucional aprobado
Administrador técnico	Gestionar usuarios	No implica leer notas o expedientes sensibles

Ejemplo DRF: restringir un queryset

```
from rest_framework.permissions import IsAuthenticated
from rest_framework.viewsets import ReadOnlyModelViewSet

class MatriculaViewSet(ReadOnlyModelViewSet):
    permission_classes = [IsAuthenticated]
    serializer_class = MatriculaSerializer

    def get_queryset(self):
        # El modelo debe relacionar matricula con usuario titular
        return Matricula.objects.filter(
            estudiante__usuario=self.request.user)
```

El filtro impide que un estudiante consulte matrículas ajenas mediante identificadores alterados. Si existen otros roles, debe implementarse una política explícita y comprobable que defina sus alcances, no eliminar simplemente el filtro.

BOLA e IDOR

La autorización de objetos rota (BOLA, OWASP API1:2023) aparece cuando una API usa un ID proporcionado por el cliente sin comprobar si ese sujeto puede acceder al objeto. Los UUID no sustituyen el permiso.

5. Autorización de acciones y propiedades

Además del recurso, hay que comprobar la operación y los campos autorizados. No basta con permitir acceso a una matrícula: modificar su estado o una calificación puede requerir privilegios diferentes.

```
from rest_framework import serializers

class MatriculaCrearSerializer(serializers.ModelSerializer):
    class Meta:
        model = Matricula
        # Lista permitida: el cliente NO decide el propietario
        fields = ["asignatura", "periodo"]

    def create(self, validated_data):
        estudiante = self.context["request"].user.estudiante
        return Matricula.objects.create(
            estudiante=estudiante, **validated_data)
```

Control adicional: verificar que la asignatura esté ofertada, que existan cupos y que el estudiante cumpla prerrequisitos; estas reglas pertenecen al dominio, no solo al serializer.

Matriz de permisos

Recurso / acción	Estudiante	Docente	Dirección
Matrícula: consultar propia	Sí	No por defecto	Según ámbito
Nota: consultar	Solo propia	Solo asignados	Según finalidad
Nota: actualizar	No	Solo asignados	Según procedimiento
Usuarios: administrar	No	No	Rol específico

En sistemas reales, estas reglas deben revisarse con responsables funcionales, documentarse y probarse con casos positivos y negativos.

6. Validación de entradas y de reglas

Validar significa comprobar que el dato tiene formato, tamaño, tipo, rango y significado admitidos. La validación debe ocurrir en servidor incluso si el frontend ya la realiza.

Nivel	Ejemplo
Sintáctico	Una fecha cumple formato ISO 8601
Estructural	Un JSON incluye campos esperados, sin exceso de propiedades
Semántico	Fecha de finalización posterior a fecha de inicio
Dominio	No admitir una segunda matrícula incompatible
Autorización	El usuario puede modificar ese registro

```
from rest_framework import serializers

class SolicitudMatriculaSerializer(serializers.Serializer):
    asignatura_id = serializers.IntegerField(min_value=1)
    observacion = serializers.CharField(
        max_length=300, required=False, allow_blank=True)

    def validate(self, attrs):
        # Consultar las reglas reales del dominio en un servicio
        if not hay_cupos(attrs["asignatura_id"]):
            raise serializers.ValidationError(
                {"asignatura_id": "No existen cupos"})
        return attrs
```

Nota: hay_cupos representa una función del dominio a implementar; por sí sola no evita sobrecupos concurrentes. Se requieren transacciones y controles en base de datos.

Consultas SQL y archivos

- Usar ORM o parámetros enlazados, no concatenar cadenas con valores enviados por usuarios.
- Validar extensión, tipo real, tamaño y contenido de archivos; almacenar fuera de rutas ejecutables.
- Aplicar límites de tamaño a solicitudes JSON, archivos y paginación.

7. OWASP API Security Top 10 (2023)

OWASP identifica diez categorías de riesgos relevantes para APIs. La edición API Security Top 10 de 2023 se presenta como referencia específica de APIs, distinta de la lista general de aplicaciones web.

ID	Riesgo	Controles ilustrativos
API1	Broken Object Level Authorization	Permisos por registro, pruebas multiusuario
API2	Broken Authentication	MFA, sesiones seguras, tokens y rate limits
API3	Broken Object Property Level Authorization	Listas de campos, DTO y permisos de propiedad
API4	Unrestricted Resource Consumption	Límites, cuotas, paginación, timeouts
API5	Broken Function Level Authorization	Permisos por operación y rol
API6	Unrestricted Access to Sensitive Business Flows	Controles de abuso en procesos sensibles
API7	Server Side Request Forgery	Restringir destinos, egress y redirecciones
API8	Security Misconfiguration	Configuración segura, secretos y parches
API9	Improper Inventory Management	Inventario, versionado y retirar endpoints
API10	Unsafe Consumption of APIs	Validar respuestas de proveedores y aislar fallos

El Top 10 sirve para priorizar escenarios, pero no equivale a una lista completa de controles. Para verificar requisitos sistemáticos puede utilizarse OWASP ASVS y documentación de los frameworks.

8. Amenazas frecuentes: ejemplos razonados

Caso A: lectura indebida de expedientes

Un cliente consulta GET /api/v1/matriculas/830 y cambia el ID a 831. Si la API devuelve un expediente ajeno, hay una falla de permisos por objeto. Mitigación: filtrar por ámbito autorizado y probar con dos cuentas.

Caso B: modificación masiva de propiedades

Un cliente incluye {"rol":"admin"} o {"nota_final":10} en un JSON aunque el formulario no exhiba estos campos. Mitigación: listas de escritura permitidas y autorización por propiedad.

Caso C: consumo ilimitado

Una búsqueda acepta un millón de resultados o procesa archivos gigantes. Mitigación: límites de tamaño, duración, cuotas, paginación y observabilidad.

Caso D: llamada a URL proporcionada por el usuario

Una función de importación solicita una URL y el servidor descarga desde destinos arbitrarios. Mitigación: lista de destinos confiables, validación de redirecciones y bloqueo de redes internas; riesgo SSRF.

Caso E: fuga por mensajes y registros

Un error devuelve secretos, detalles de SQL o trazas internas al cliente. Mitigación: mensaje público genérico, correlación por ID y trazas completas únicamente en observabilidad restringida.

9. Controles HTTP, despliegue y operación

Control	Qué comprobar
HTTPS / TLS	Redirección segura y terminación TLS confiable
CORS	Orígenes explícitos; CORS no es un mecanismo de autorización
CSRF	Protección para autenticación basada en cookies
Headers	HSTS y políticas pertinentes al contexto
Secretos	Variables protegidas o gestor de secretos; rotación
Dependencias	Actualización y análisis de vulnerabilidades
Logs	Sin credenciales, datos mínimos, correlación y retención
Backups	Cifrado, control de acceso y restauración comprobada

Configuración ilustrativa de Django

```
# En produccion: configurar desde el entorno y probar
DEBUG = False
SESSION_COOKIE_SECURE = True
CSRF_COOKIE_SECURE = True
SECURE_HSTS_SECONDS = 31536000  # despues de validar HTTPS
# ALLOWED_HOSTS y CSRF_TRUSTED_ORIGINS deben ser explicitos
# Nunca publicar SECRET_KEY en el repositorio.
```

Los valores de despliegue son orientativos y deben ajustarse al proxy, dominio, autenticación y entorno real. HSTS debe activarse tras verificar toda la infraestructura HTTPS.

10. Pruebas de seguridad automatizadas

Una prueba útil intenta demostrar que una operación permitida funciona y que una operación prohibida falla. Los controles de autorización se prueban cruzando usuarios, roles, recursos y métodos HTTP.

```
import pytest
from rest_framework.test import APIClient

@pytest.mark.django_db
def test_estudiante_no_lee_matricula_ajena(
        usuario_ana, matricula_de_luis):
    client = APIClient()
    client.force_authenticate(user=usuario_ana)
    resp = client.get(
        f"/api/v1/matriculas/{matricula_de_luis.pk}/")
    assert resp.status_code in (403, 404)
```

Se asume que los fixtures y rutas se crean en el proyecto del taller. El código es una prueba ilustrativa, no una aplicación completa.

Casos mínimos de comprobación

- Usuario anónimo obtiene 401 o el comportamiento de autenticación documentado.
- Usuario autenticado sin permiso obtiene 403 o 404 según la política de divulgación.
- Un estudiante no consulta ni cambia objetos ajenos.
- Datos inesperados, demasiado largos o de tipo incorrecto son rechazados.
- Credenciales, cookies y tokens no aparecen en respuestas ni registros.
- Los límites de paginación y carga funcionan bajo solicitudes adversas.

11. Taller guiado: proteger un módulo de matrículas

Situación: implementar GET /api/v1/matriculas/ y GET /api/v1/matriculas/{id}/; agregar una operación de solicitud de matrícula. El sistema admite cuentas de estudiantes y, posteriormente, docentes y Dirección.

Etapa 1. Delimitar recursos y sujetos

- Dibujar entidades: Usuario, Estudiante, Asignatura y Matrícula.
- Redactar políticas: quién consulta, crea, modifica y anula.

Etapa 2. Implementar controles

- Habilitar autenticación y permisos por defecto en DRF.
- Filtrar querysets según rol y ámbito; restringir campos modificables.
- Validar datos de entrada y reglas del proceso de matrícula.

Etapa 3. Verificar

- Construir pruebas con dos estudiantes y matrículas distintas.
- Probar solicitudes anónimas, identificadores ajenos, valores inválidos y asignaturas sin cupo.
- Registrar resultado esperado, observado, evidencia y corrección.

Entregables

Producto	Contenido mínimo
Matriz de permisos	Acciones, recursos, roles, alcance y excepciones
Implementación	Serializers, views, permisos, validaciones
Pruebas	Ejecución automatizada y evidencia de casos negativos
Informe de riesgos	Amenaza, impacto, control y riesgo residual

12. Actividades, evaluación y autoevaluación

Actividades individuales

- A1. Explica con un ejemplo la diferencia entre autenticación y autorización.
- A2. Diseña la matriz de permisos para estudiantes, docentes y Dirección.
- A3. Detecta al menos cuatro debilidades en una API que permite modificar cualquier matrícula por ID.
- A4. Redacta pruebas para una validación de longitud, una propiedad prohibida y una lectura no autorizada.

Preguntas de repaso

- ¿Por qué un UUID no evita BOLA?
- ¿Por qué CORS no reemplaza los controles de permisos?
- ¿Qué diferencia hay entre seguridad de una cookie y seguridad de un token bearer?
- ¿Cuándo corresponde un 401 y cuándo un 403?
- ¿Qué controles pueden limitar el abuso sin bloquear usuarios legítimos?

Rúbrica sugerida: escala 2 / 1 / 0

Criterio	2 - logrado	1 - parcial	0 - insuficiente
Conceptos	Distingue conceptos con ejemplos	Presenta imprecisiones	Confunde conceptos
Autorización	Permisos por acción y objeto	Controles incompletos	Accesos indebidos
Validación	Incluye tipos y reglas de negocio	Cubre solo una parte	Acepta datos inválidos
Pruebas	Casos positivos y negativos	Solo casos básicos	Sin evidencia
Documentación	Riesgos y decisiones justificados	Justificación incompleta	Sin justificación

Puntuación: 10 puntos máximos. Interpretación orientativa: 9-10 sólido, 7-8 satisfactorio, 5-6 en desarrollo, 0-4 requiere refuerzo.

13. Glosario y bibliografía

Término	Definición breve
MFA	Autenticación que combina factores independientes
RBAC / ABAC	Políticas basadas en roles / atributos
BOLA	Falla de autorización a nivel de objeto
CSRF	Acción no deseada mediante el contexto autenticado de una víctima
SSRF	El servidor realiza solicitudes hacia destinos indebidos
Hash de contraseña	Representación no reversible, diseñada para verificación
Rate limiting	Control de la frecuencia o cuota de solicitudes
Least privilege	Conceder solo permisos necesarios

Lecturas oficiales recomendadas

OWASP API Security Top 10 (2023): https://api-security.owasp.org/editions/2023/en/0x11-t10/

OWASP Authorization Cheat Sheet:
https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html

OWASP Input Validation Cheat Sheet:
https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html

OWASP ASVS: https://owasp.org/www-project-application-security-verification-standard/

NIST SP 800-63B-4 (2025): https://csrc.nist.gov/pubs/sp/800/63/b/4/final

Django Security: https://docs.djangoproject.com/en/stable/topics/security/

DRF Permissions: https://www.django-rest-framework.org/api-guide/permissions/

Nota didáctica: los ejemplos son fragmentos para laboratorio y no sustituyen una evaluación de seguridad, integración, pruebas de despliegue o revisión del diseño del sistema concreto.

SERIE BACKEND PROFESIONAL · GUÍA 03

Bases de datos para un backend confiable

Modelo de datos · integridad · migraciones · índices · transacciones

Guía didáctica para estudiantes de Computación

Conceptos claros, ejemplos SQL y Django ORM, situaciones reales, laboratorio, autoevaluación y rúbrica.

Caso integrador: sistema de matrícula académica (estudiantes, asignaturas, períodos y matrículas).

Enfoque: PostgreSQL como motor relacional y Django como ejemplo de implementación. Los principios aplican a otros lenguajes y sistemas gestores.

1. Propósito y resultados de aprendizaje

Una base de datos no es simplemente un lugar donde almacenar información. Es un componente que debe preservar reglas, evitar inconsistencias, responder con eficiencia y resistir errores concurrentes. El backend y la base de datos comparten responsabilidades; las validaciones de aplicación no reemplazan las restricciones persistentes.

Al finalizar, el estudiante podrá:

• Diseñar un modelo entidad-relación y transformarlo en tablas normalizadas.

• Definir claves, relaciones, restricciones e integridad referencial.

• Evolucionar el esquema mediante migraciones seguras y reversibles cuando sea posible.

• Seleccionar índices según consultas reales y leer planes de ejecución.

• Utilizar transacciones y niveles de aislamiento para mantener consistencia.

• Implementar y probar estos conceptos en SQL y Django ORM.

Mapa conceptual

Diseño	Confiabilidad	Evolución	Rendimiento
Entidades, atributos, cardinalidad, normalización	PK, FK, UNIQUE, CHECK, NOT NULL	Versionado del esquema y migraciones	Índices y análisis de consultas
Relaciones y reglas de negocio	Transacciones, ACID, bloqueos	Compatibilidad con datos antiguos	Costo de escritura y selectividad

Pregunta inicial: ¿qué ocurre si dos estudiantes ocupan simultáneamente el último cupo disponible de una asignatura? La respuesta requiere modelo de datos, reglas de integridad y control transaccional.

2. Conceptos fundamentales

Concepto	Definición sencilla	Ejemplo
SGBD	Software que almacena, consulta y administra datos.	PostgreSQL, MySQL, SQLite
Tabla	Conjunto de filas con una estructura común.	estudiante
Fila o registro	Instancia de una entidad.	Estudiante 1001
Columna o campo	Atributo con un tipo de dato.	correo VARCHAR(255)
Esquema	Definición de tablas, tipos, relaciones y restricciones.	public en PostgreSQL
Consulta	Operación para recuperar o modificar información.	SELECT / UPDATE

Modelos de datos

Relacional: tablas vinculadas por claves, consulta SQL y restricciones declarativas. Documental: documentos JSON y estructuras flexibles. Clave-valor: acceso rápido por clave. Grafo: relaciones como primera clase. La elección depende de consistencia, patrones de consulta, volumen y necesidades del negocio; no de modas.

3. Del problema al modelo de datos

El sistema académico registra estudiantes, asignaturas, períodos y matrículas. Una matrícula vincula a un estudiante con una asignatura durante un período y guarda su estado. Las reglas de negocio deben identificarse antes de crear tablas.

Reglas de negocio del caso

• Cada estudiante tiene un identificador y un correo institucional único.

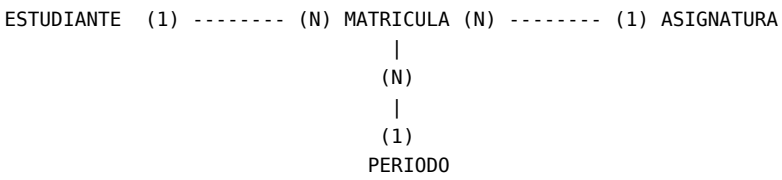
• Una asignatura tiene código único y una capacidad máxima positiva.

• Una matrícula pertenece exactamente a un estudiante, una asignatura y un período.

• No puede existir más de una matrícula del mismo estudiante en la misma asignatura y período.

• Una matrícula solo puede tener estados: inscrita, retirada o aprobada.

Relaciones y cardinalidades



La entidad MATRÍCULA resuelve una relación muchos a muchos entre ESTUDIANTE y ASIGNATURA, con el contexto adicional del PERÍODO. Esto se denomina entidad asociativa.

Diccionario mínimo de datos

Tabla	Campos clave	Reglas
estudiante	id, nombre, correo	correo único; nombre obligatorio
asignatura	id, codigo, nombre, capacidad	código único; capacidad > 0
periodo	id, nombre, fecha_inicio, fecha_fin	inicio anterior o igual a fin
matricula	id, estudiante_id, asignatura_id, periodo_id, estado	FK válidas; combinación única; estado permitido

4. Normalización sin complicaciones

Normalizar significa organizar los datos para minimizar redundancia y anomalías. No consiste en dividir tablas indiscriminadamente.

Ejemplo de diseño problemático

```
matricula_plana(
  estudiante_id, estudiante_nombre, estudiante_correo,
  asignatura_codigo, asignatura_nombre, periodo, estado
)
```

Problemas: al cambiar un correo deben actualizarse muchas filas (anomalía de actualización); no se puede registrar una nueva asignatura sin matrícula (inserción); al borrar la última matrícula puede desaparecer la única descripción de la asignatura (eliminación).

Forma normal	Idea principal	Cómo reconocerla
1FN	Valores atómicos, sin grupos repetidos.	No guardar varias asignaturas en una celda.
2FN	Todo atributo no clave depende de toda la clave candidata.	En una clave compuesta, evitar dependencias parciales.
3FN	Evitar dependencias transitivas de atributos no clave respecto a claves.	No repetir nombre de asignatura dentro de matrícula.

La 3FN suele ser una buena base para sistemas transaccionales. La desnormalización puede ayudar en lecturas específicas, pero debe justificarse y controlar la consistencia.

5. Integridad: proteger los datos por diseño

La integridad es el conjunto de garantías que hacen válidos los datos persistidos. Incluye integridad de entidad, referencial y de dominio.

Restricción	Objetivo	Ejemplo
PRIMARY KEY	Identifica de manera única cada fila.	estudiante.id
FOREIGN KEY	Evita referencias a filas inexistentes.	matricula.estudiante_id
UNIQUE	Evita duplicados de una clave natural o compuesta.	correo; combinación estudiante-asignatura-período
NOT NULL	Exige un dato obligatorio.	asignatura.nombre
CHECK	Limita valores con una expresión.	capacidad > 0
DEFAULT	Establece un valor si no se proporciona.	estado = inscrita

Ejemplo SQL: creación de tablas (PostgreSQL)

```
CREATE TABLE estudiante (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  nombre VARCHAR(120) NOT NULL,
  correo VARCHAR(255) NOT NULL UNIQUE
);
CREATE TABLE asignatura (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  codigo VARCHAR(20) NOT NULL UNIQUE,
  nombre VARCHAR(120) NOT NULL,
  capacidad INTEGER NOT NULL CHECK (capacidad > 0)
);
CREATE TABLE periodo (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  nombre VARCHAR(40) NOT NULL UNIQUE,
  fecha_inicio DATE NOT NULL, fecha_fin DATE NOT NULL,
  CONSTRAINT periodo_fechas CHECK (fecha_inicio <= fecha_fin)
);

CREATE TABLE matricula (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  estudiante_id BIGINT NOT NULL REFERENCES estudiante(id),
  asignatura_id BIGINT NOT NULL REFERENCES asignatura(id),
  periodo_id BIGINT NOT NULL REFERENCES periodo(id),
  estado VARCHAR(12) NOT NULL DEFAULT 'inscrita',
  CONSTRAINT estado_valido CHECK
    (estado IN ('inscrita','retirada','aprobada')),
  CONSTRAINT matricula_unica UNIQUE
    (estudiante_id, asignatura_id, periodo_id)
);
```

La restricción UNIQUE impide matrículas duplicadas incluso si dos servidores intentan insertarlas simultáneamente. Es una protección más fuerte que comprobar previamente su existencia en Python.

¿Qué ocurre al eliminar una fila referenciada?

Una FK utiliza comportamientos como NO ACTION, RESTRICT, CASCADE, SET NULL o SET DEFAULT. No siempre corresponde CASCADE: borrar un estudiante y todas sus matrículas podría destruir evidencias históricas. La política de borrado debe corresponder al dominio, protección de datos y retención aplicable.

6. Transacciones: varias operaciones, una unidad

Una transacción agrupa cambios que deben confirmarse juntos o deshacerse. Las propiedades ACID describen los objetivos de un procesamiento transaccional confiable.

Propiedad	Explicación	Ejemplo
Atomicidad	Todo se aplica o nada.	No crear matrícula si falla el descuento de cupo.
Consistencia	Las restricciones y reglas se respetan al confirmar.	No superar capacidad.
Aislamiento	Las transacciones concurrentes se coordinan.	Dos inscripciones al último cupo.

Propiedad	Explicación	Ejemplo
Durabilidad	Un cambio confirmado persiste tras fallos, según garantías del SGBD.	Matrícula confirmada después de reinicio.

COMMIT y ROLLBACK

```
BEGIN;
INSERT INTO matricula (estudiante_id,asignatura_id,periodo_id)
VALUES (1001,20,4);
-- Si las validaciones son correctas:
COMMIT;
-- Ante un fallo: ROLLBACK; en lugar de COMMIT
```

Importante: BEGIN/COMMIT no impide por sí solo una carrera de concurrencia. También se requiere modelar correctamente la reserva de cupos y elegir bloqueos o mecanismos atómicos.

7. Concurrencia y niveles de aislamiento

Si dos solicitudes leen “queda 1 cupo” y luego insertan, ambas pueden aprobar el ingreso si no existe una coordinación adicional. Es el clásico problema de check-then-act.

Niveles de aislamiento (visión general)

Nivel	Descripción didáctica
READ UNCOMMITTED	Permite el aislamiento más débil en la teoría SQL; PostgreSQL lo trata como READ COMMITTED.
READ COMMITTED	Cada sentencia ve datos confirmados al iniciar su ejecución; por defecto en PostgreSQL.
REPEATABLE READ	Las lecturas de una transacción conservan una instantánea consistente, con diferencias según motor.
SERIALIZABLE	Busca comportamiento equivalente a ejecuciones seriales; puede requerir reintentos por conflictos.

Patrón práctico: bloquear la asignatura

```
BEGIN;
SELECT capacidad FROM asignatura WHERE id = 20 FOR UPDATE;
SELECT COUNT(*) FROM matricula
 WHERE asignatura_id=20 AND periodo_id=4
   AND estado='inscrita';
-- Si el contador es inferior a la capacidad, insertar matrícula.
INSERT INTO matricula (estudiante_id,asignatura_id,periodo_id)
VALUES (1001,20,4);
COMMIT;
```

Este patrón funciona cuando todas las rutas de matrícula siguen el mismo bloqueo y regla de capacidad, y cuando la definición de “cupo ocupado” es coherente. Para alta concurrencia también pueden evaluarse contadores atómicos o SERIALIZABLE con reintentos.

Problemas típicos

• Actualización perdida: dos procesos escriben sobre el mismo valor sin coordinarse.

• Lectura no repetible: una segunda lectura puede observar un valor diferente.

• Phantom: aparecen o desaparecen filas que cumplen un filtro.

• Deadlock: transacciones esperan mutuamente recursos bloqueados.

Buenas prácticas: transacciones cortas, orden estable para adquirir bloqueos, no llamar servicios externos mientras se mantienen bloqueos y reintentar solo errores transitorios de forma controlada.

8. Índices: acelerar consultas sin indexarlo todo

Un índice es una estructura adicional que reduce el trabajo necesario para localizar determinadas filas. A cambio, ocupa espacio y encarece INSERT, UPDATE y DELETE. Los índices deben diseñarse en función de consultas reales.

Ejemplo de una consulta frecuente

```
SELECT id, estudiante_id, estado
FROM matricula
WHERE periodo_id=4 AND asignatura_id=20
ORDER BY id DESC
LIMIT 25;

CREATE INDEX idx_matricula_periodo_asignatura_id
ON matricula (periodo_id, asignatura_id, id DESC);
```

El orden de columnas de un índice compuesto importa: un índice (periodo_id, asignatura_id, id) puede beneficiar determinados filtros que comienzan por periodo_id, pero no reemplaza automáticamente todos los demás índices.

Tipos habituales en PostgreSQL

Tipo	Uso típico	Precaución
B-tree	Igualdad, rangos, ordenación	Predeterminado y versátil.
GIN	Arrays, JSONB y búsqueda de texto	Más costo de mantenimiento.
GiST	Geoespacial y operadores especializados	Requiere operador compatible.
BRIN	Tablas muy grandes con correlación física	Menor precisión; depende de distribución.

Medir antes y después

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT id FROM matricula
WHERE periodo_id=4 AND asignatura_id=20;
```

EXPLAIN muestra el plan; ANALYZE ejecuta realmente la consulta, por lo que debe usarse con cuidado en sentencias que modifiquen datos. Compare tiempos reales, filas estimadas/reales, buffers y

secuencias de lectura. En tablas pequeñas una búsqueda secuencial puede ser correcta.

> Regla de oro: nunca agregar índices porque “la columna parece importante”. Primero identificar consultas, volumen, selectividad y frecuencia de escritura.

9. Migraciones: evolución controlada del esquema

Una migración es una modificación versionada del esquema y, a veces, de los datos. Permite que distintos entornos evolucionen de manera reproducible: desarrollo, pruebas y producción.

Flujo recomendado

```
Cambiar modelo > Generar migración > Revisar SQL > Probar
     > Backup/plan de reversión > Desplegar > Verificar
```

Ejemplo en Django

```
python manage.py makemigrations
python manage.py sqlmigrate academico 0002
python manage.py migrate
python manage.py showmigrations academico
```

No edite manualmente migraciones ya aplicadas en otros entornos. Las migraciones de datos deben ser idempotentes cuando sea viable y ejecutarse con control de tiempos, volúmenes y dependencias.

Cambio seguro: agregar un campo obligatorio

Una estrategia de expandir y contraer evita exigir de inmediato una columna nueva a una versión antigua de la aplicación:

- 1. Agregar columna nueva permitiendo NULL o con un default seguro.
- 2. Desplegar código que escriba el nuevo valor.
- 3. Rellenar filas antiguas por lotes (backfill) y validar datos.
- 4. Aplicar NOT NULL u otras restricciones cuando todos los datos cumplan.
- 5. Retirar el código de compatibilidad y columnas antiguas en un despliegue posterior.

Consideración: en bases grandes algunos ALTER TABLE pueden bloquear o reescribir datos. El plan operativo depende de motor, versión y volumen; evaluar despliegue sin interrupción cuando sea necesario.

10. Django ORM: implementación del caso

El ORM traduce modelos Python a operaciones SQL, pero no elimina la necesidad de comprender claves, índices y transacciones.

```
from django.db import models

class Estudiante(models.Model):
    nombre = models.CharField(max_length=120)
    correo = models.EmailField(unique=True)

class Asignatura(models.Model):
```

```
    codigo = models.CharField(max_length=20, unique=True)
    nombre = models.CharField(max_length=120)
    capacidad = models.PositiveIntegerField()

class Periodo(models.Model):
    nombre = models.CharField(max_length=40, unique=True)
    fecha_inicio = models.DateField()
    fecha_fin = models.DateField()

class Matricula(models.Model):
    estudiante = models.ForeignKey(Estudiante, on_delete=models.PROTECT)
    asignatura = models.ForeignKey(Asignatura, on_delete=models.PROTECT)
    periodo = models.ForeignKey(Periodo, on_delete=models.PROTECT)
    estado = models.CharField(max_length=12, default='inscrita')

    class Meta:
        constraints = [
            models.UniqueConstraint(
                fields=['estudiante','asignatura','periodo'],
                name='matricula_unica'
            ),
            models.CheckConstraint(
                condition=models.Q(estado__in=[
                    'inscrita','retirada','aprobada']),
                name='matricula_estado_valido'
            )
        ]
        indexes = [models.Index(
            fields=['periodo','asignatura','-id'],
            name='mat_per_asig_id_idx'
        )]
```

Nota: PositiveIntegerField y las validaciones del formulario/serializador no sustituyen todas las restricciones de base. Para capacidad estrictamente positiva puede agregarse CheckConstraint(capacidad__gt=0); también debe garantizarse que fecha_inicio no supere fecha_fin.

Servicio de matrícula seguro con bloqueo

```
from django.db import transaction
from django.core.exceptions import ValidationError

@transaction.atomic
def matricular(estudiante, asignatura_id, periodo):
    asignatura = Asignatura.objects.select_for_update().get(
        pk=asignatura_id)
    ocupados = Matricula.objects.filter(
        asignatura=asignatura, periodo=periodo,
        estado='inscrita').count()
    if ocupados >= asignatura.capacidad:
        raise ValidationError('No existen cupos disponibles')
    return Matricula.objects.create(
        estudiante=estudiante, asignatura=asignatura,
        periodo=periodo, estado='inscrita')
```

El ejemplo supone PostgreSQL y un protocolo único de inscripción. Un intento duplicado puede producir IntegrityError por la restricción UNIQUE: capturarlo fuera del bloque transaccional apropiado y transformarlo en respuesta de negocio. Para pruebas de concurrencia utilizar TransactionTestCase.

Evitar consultas N+1

```
# Ineficiente: consulta asignatura por cada matrícula
matriculas = Matricula.objects.all()
# Mejor cuando se leen claves foráneas:
matriculas = Matricula.objects.select_related(
    'estudiante', 'asignatura', 'periodo')
```

11. Separación de responsabilidades del backend

Capa	Responsabilidad	Ejemplo
API / serializer	Validar forma de entrada y salida HTTP.	Correo y campos de entrada válidos.
Servicio / aplicación	Orquestar caso de uso y su transacción.	Matricular verificando cupos.
Dominio	Definir reglas académicas.	Estados y restricciones de inscripción.
Persistencia / ORM	Guardar, consultar, aplicar integridad.	FK, UNIQUE, SELECT FOR UPDATE.

Evite lógica de negocio extensa dentro de vistas o serializadores, y evite confiar en que toda operación llegará desde una única API: tareas programadas, importaciones y otros servicios también escriben datos.

12. Buenas prácticas operativas

• Usar un usuario de aplicación con permisos mínimos. Separar cuentas de lectura, migración y administración cuando proceda.

• Gestionar credenciales fuera del repositorio, con TLS y rotación de secretos.

• Definir RPO (pérdida máxima tolerable) y RTO (tiempo máximo de recuperación); probar restauraciones periódicamente.

• Monitorear conexiones, latencia, consultas lentas, locks, tamaño de índices y errores de transacción.

• Evaluar retención de datos personales, trazabilidad de cambios y mecanismos de anonimización cuando corresponda.

• Separar datos de prueba de datos reales; nunca usar producción como laboratorio.

13. Laboratorio guiado: sistema de matrícula

Objetivo: construir el subsistema mínimo de inscripciones y demostrar que sus datos se mantienen correctos bajo condiciones normales y concurrentes.

Etapa	Producto esperado	Criterio de comprobación
A. Análisis	Reglas y diagrama entidad-relación	Cardinalidades y claves justificadas.

Etapa	Producto esperado	Criterio de comprobación
B. Diseño SQL	Script con 4 tablas y restricciones	Duplicados y FK inválidas son rechazados.
C. Django ORM	Modelos y migraciones versionadas	Migraciones aplican en BD vacía.
D. Consultas	Filtros e índice propuesto	Comparación EXPLAIN antes/después.
E. Transacciones	Servicio de matrícula atómico	No se excede capacidad con concurrencia.
F. Informe	Decisiones, evidencias y limitaciones	Capturas de pruebas y explicación técnica.

Pruebas indispensables

• Intentar registrar dos estudiantes con el mismo correo. ¿Cuál es el error de BD?

• Intentar crear matrícula con estudiante_id inexistente. ¿Qué protege la FK?

• Crear por segunda vez la misma matrícula. ¿Cómo responde UNIQUE?

• Agregar una asignatura con capacidad 0. ¿Qué restricción falta si se permite?

• Ejecutar dos solicitudes concurrentes sobre el último cupo. ¿Se sobreinscribe?

• Comparar el plan de consulta con y sin índice usando datos suficientes.

14. Autoevaluación

1. ¿Qué diferencia hay entre clave primaria y clave foránea?

2. ¿Por qué UNIQUE es más confiable que una validación previa en Python?

3. ¿Qué anomalías previene la normalización?

4. ¿En qué casos CASCADE podría ser peligroso?

5. ¿Qué significa atomicidad en una matrícula?

6. ¿Por qué READ COMMITTED no garantiza por sí solo el control del último cupo?

7. ¿Cuándo podría no usarse un índice existente?

8. ¿Qué información aporta EXPLAIN ANALYZE?

9. ¿Por qué versionar migraciones?

10. ¿Para qué sirve expandir y contraer?

Respuestas orientativas breves

1) PK identifica; FK referencia. 2) La BD coordina escrituras simultáneas. 3) Duplicidad y anomalías. 4) Puede eliminar histórico. 5) Todo o nada. 6) La comprobación separada puede competir. 7) Tabla pequeña, baja selectividad o costo. 8) Plan y medidas reales. 9) Reproducibilidad y trazabilidad. 10) Despliegue compatible por fases.

15. Rúbrica de evaluación (2 / 1 / 0)

Criterio	2 — Logrado	1 — Parcial	0 — No logrado
Modelo y normalización	Entidades, relaciones y dependencias coherentes.	Errores menores o redundancias.	Inconsistente o ausente.
Integridad	Restricciones efectivas y probadas.	Restricciones incompletas.	Sin controles fiables.
Migraciones	Versionadas, probadas y documentadas.	Aplican, pero sin revisión completa.	No reproducibles.
Índices	Justificados con consultas y planes.	Índices sin suficiente evidencia.	No analiza rendimiento.
Transacciones	Control de cupos concurrente probado.	Transacción sin prueba concurrente.	Puede sobreinscribir.
Documentación	Informe claro con pruebas y decisiones.	Evidencia incompleta.	Sin evidencia.

Puntaje máximo: 12 puntos. Interpretación sugerida: 10–12 satisfactorio; 7–9 en desarrollo; 0–6 requiere refuerzo. Ajustar según los resultados de aprendizaje y planificación académica.

16. Glosario esencial

ACID: atomicidad, consistencia, aislamiento y durabilidad. Cardinalidad: cantidad de registros que pueden relacionarse. Concurrencia: operaciones simultáneas. FK: clave foránea. Índice: estructura auxiliar de acceso. Integridad referencial: validez de referencias. Migración: cambio versionado del esquema. Normalización: organización para reducir anomalías. ORM: mapeador objeto-relacional. PK: clave primaria. Rollback: deshacer transacción. Transacción: unidad de trabajo confirmada o revertida.

17. Referencias para profundizar

• PostgreSQL Documentation: Data Definition, Constraints, Indexes, Transaction Isolation, EXPLAIN. https://www.postgresql.org/docs/current/

• Django Documentation: Models, Migrations, Database transactions, QuerySet API, Constraints. https://docs.djangoproject.com/

• Silberschatz, A., Korth, H. F. y Sudarshan, S. Database System Concepts. McGraw-Hill.

• Elmasri, R. y Navathe, S. B. Fundamentals of Database Systems. Pearson.

• Kleppmann, M. Designing Data-Intensive Applications. O’Reilly.

Material educativo. Los comandos se muestran como ejemplos: verifique compatibilidad con las versiones concretas del motor y del framework utilizadas en el laboratorio.

SERIE BACKEND PROFESIONAL / GUÍA DIDÁCTICA 18

Quarto para documentación técnica de un backend profesional

Documentación como código · Markdown y QMD · Sitios web · Libros técnicos · Diagramas · GitHub Pages

Público: estudiantes universitarios de Computación. Proyecto transversal: API de matrículas académicas con Django REST Framework y PostgreSQL.

Resultados de aprendizaje: construir un sitio de documentación reproducible, mantener contratos y decisiones técnicas enlazados, y automatizar su publicación.

Enfoque: explicación conceptual, ejemplos de código, laboratorio guiado, evidencia y evaluación.

Fecha: octubre de 2026 | Nivel: inicial a intermedio

1. ¿Qué es Quarto y por qué utilizarlo?

Quarto es un sistema de publicación técnica y científica basado en archivos de texto (principalmente .qmd) que permite producir documentos, sitios web, libros, presentaciones y otros formatos. Integra Markdown, metadatos YAML, fragmentos de código y referencias cruzadas.

En un backend profesional, Quarto no sustituye OpenAPI, C4, ADR ni el código fuente: los reúne, contextualiza y publica de una forma navegable, versionada y revisable mediante Git.

Documentación como código (Docs as Code)

• Fuente versionada: los archivos .qmd, .yml y diagramas se guardan en el repositorio.

• Revisión: los cambios de documentación pueden aprobarse mediante pull requests.

• Reproducibilidad: el sitio se genera a partir de las mismas fuentes y configuración.

• Automatización: CI puede comprobar enlaces, generar HTML y publicar.

• Coherencia: una funcionalidad no se considera terminada sin actualizar sus páginas y contratos.

Quarto frente a otras herramientas

Herramienta	Función principal	Papel en el proyecto
OpenAPI / Swagger UI	Contrato y exploración HTTP	Referencia precisa de endpoints
Structurizr / PlantUML	Vistas y diagramas de arquitectura	Describe estructura y decisiones
ADR	Registro de decisiones	Explica por qué se eligió una solución
Quarto	Publicación y navegación	Integra guías, enlaces y evidencias

2. Instalación y comandos esenciales

Instalar Quarto desde su sitio oficial y comprobar la instalación. Para producir PDF desde Quarto puede necesitarse un motor de composición adicional; la guía se centra en HTML y explica la exportación como extensión.

```
quarto --version
quarto check
quarto create project website docs
cd docs
quarto preview
quarto render
```

preview inicia una vista local para edición; render genera el sitio estático (por defecto, `_site/`), que es el artefacto a publicar. El comando `quarto check` facilita el diagnóstico de dependencias.

Archivo .qmd mínimo

```
---
title: "Arquitectura del backend"
---

# Visión general

El backend expone una API REST bajo `/api/v1`.

## Tecnologías

- Django REST Framework
- PostgreSQL
```

3. Organización de un proyecto de documentación

Es recomendable separar la documentación del código de ejecución, pero mantenerlos en el mismo repositorio cuando el equipo necesita cambios sincronizados.

```
backend-matriculas/
├── apps/
├── tests/
├── docs/
│   ├── _quarto.yml
│   ├── index.qmd
│   ├── arquitectura/
│   │   ├── c1-contexto.qmd
│   │   └── c2-contenedores.qmd
│   ├── api/contratos.qmd
│   ├── operaciones/despliegue.qmd
│   ├── decisiones/adr-001.qmd
│   └── assets/
├── openapi.yaml
└── README.md
```

Este diseño permite que un cambio en el módulo de matrículas afecte, en un único pull request, a código, pruebas, contrato OpenAPI y documentación.

4. Configuración de un sitio Quarto

El archivo `_quarto.yml` contiene la configuración del proyecto, navegación, formatos y opciones globales.

```
project:
  type: website
  output-dir: _site

website:
  title: "Backend de Matrículas"
  sidebar:
    contents:
      - index.qmd
      - section: "Arquitectura"
        contents:
          - arquitectura/c1-contexto.qmd
          - arquitectura/c2-contenedores.qmd
      - api/contratos.qmd
      - operaciones/despliegue.qmd
      - decisiones/adr-001.qmd

format:
  html:
```

```
    theme: cosmo
    toc: true
    number-sections: true
```

YAML exige indentación consistente. Es preferible nombrar rutas de archivos relativas y revisar la navegación después de cada modificación.

5. Escritura técnica con Markdown y QMD

Estructura pedagógica sugerida para una página

• Propósito del módulo y prerrequisitos.

• Conceptos y decisiones arquitectónicas.

• Especificaciones, ejemplos y diagramas.

• Comandos reproducibles y resultados esperados.

• Errores comunes, criterios de aceptación y enlaces relacionados.

Tablas, enlaces y bloques de código

````
## Endpoint de matrícula

Ver el [contrato](../api/contratos.qmd).

| Método | Ruta | Descripción |
|---|---|---|
| POST | /api/v1/matriculas | Crear matrícula |

```json
{"success": true, "data": {"id": 42}}
```
````

El JSON anterior es un ejemplo abreviado dentro de la explicación: el contrato universal completo del proyecto también requiere `message`, `errors` y `meta`.

6. Diagramas y arquitectura C4

Quarto puede incorporar diagramas exportados como SVG/PNG y bloques Mermaid. Para C1-C3 se recomienda mantener el modelo fuente en Structurizr DSL y, para C4, diagramas UML de clases en PlantUML. Quarto es el punto de consulta, no el repositorio semántico primario del diagrama.

Diagrama Mermaid dentro del documento

````
```{mermaid}
flowchart LR
  Cliente[Frontend Angular] --> API[API Django]
  API --> DB[(PostgreSQL)]
  API --> Worker[Worker Celery]
```
````

Uso de imágenes versionadas

```
![C2 - Contenedores del sistema](../assets/c2.png){#fig-c2}

Como se observa en @fig-c2, la API almacena datos
en PostgreSQL y delega reportes al worker.
```

Un diagrama debe identificar límites, responsabilidades, tecnología y relaciones; las figuras no deben convertirse en capturas decorativas sin explicación.

7. Documentación de API y contrato universal

La documentación funcional de un endpoint debe explicar su propósito, permisos, reglas de negocio, método, ruta, códigos HTTP y relación con la especificación OpenAPI.

```
### POST /api/v1/matriculas

**Regla:** el estudiante debe cumplir prerrequisitos.
**Permiso:** usuario autenticado y autorizado.
**Éxito:** HTTP 201 + contrato universal.
**Error:** HTTP 409, código `CUPO_AGOTADO`.
```

Ejemplo del contrato completo

```
{
  "success": false,
  "message": "No existen cupos disponibles.",
  "data": null,
  "errors": {"code": "CUPO_AGOTADO", "details": []},
  "meta": {
    "request_id": "req_123",
    "timestamp": "2026-10-13T20:00:00Z",
    "version": "v1"
  }
}
```

El archivo OpenAPI es la fuente contractual. Evitar copiar manualmente decenas de endpoints en páginas .qmd: enlazar o generar contenido desde la especificación donde sea viable y documentar aquí las decisiones de uso.

8. ADR y decisiones técnicas

Un ADR (Architecture Decision Record) registra contexto, decisión, alternativas, consecuencias y estado. Cada decisión relevante puede convertirse en una página versionada y relacionada con los diagramas C4.

```
---
title: "ADR-001: monolito modular"
---

**Estado:** aceptada

**Contexto:** equipo pequeño y despliegue centralizado.

**Decisión:** monolito modular en Django/DRF.

**Alternativas:** microservicios; monolito sin módulos.

**Consecuencias:** despliegue simple y límites explícitos.
```

9. Código ejecutable y reproducibilidad

Quarto permite ejecutar celdas mediante motores compatibles, como Jupyter, cuando están instalados. Esto es útil para análisis de rendimiento y ejemplos educativos, pero no debe utilizarse para ejecutar acciones destructivas sobre bases de datos reales al generar la documentación.

````
```{python}
# | echo: true
# | warning: false

latencias_ms = [80, 90, 120, 160]
promedio = sum(latencias_ms) / len(latencias_ms)
print(f"Latencia promedio: {promedio:.1f} ms")
```
````

Seguridad: usar conjuntos de datos sintéticos; nunca incluir tokens, contraseñas o datos personales. Documentar dependencias y entornos reproducibles. Si el proyecto no requiere cálculos, es válido usar solo Markdown y bloques estáticos.

Congelación de resultados

```
execute:
  freeze: auto
```

La opción `freeze` puede evitar la reejecución innecesaria durante renders de proyecto. Debe entenderse su comportamiento y revisar qué archivos de resultados se versionan.

10. Publicación en GitHub Pages

Quarto documenta varias estrategias: exportar al directorio `docs`, publicar con `quarto publish gh-pages` o automatizar mediante GitHub Actions. Para fines educativos se muestra la alternativa del directorio `docs` generado en una rama publicada.

Publicación sencilla desde la raíz del repositorio

```
project:
  type: website
  output-dir: docs

quarto render
git add docs
git commit -m "docs: publicar sitio técnico"
git push origin main
```

En GitHub configurar Settings > Pages para publicar desde la rama y directorio seleccionados. En un proyecto donde el código y el sitio comparten repositorio, elegir cuidadosamente la carpeta de salida para no sobrescribir documentación fuente.

Alternativa automatizada

El equipo puede configurar GitHub Actions para instalar Quarto, renderizar documentación en cada cambio y desplegar el resultado a GitHub Pages. Antes de automatizar, confirmar permisos mínimos del workflow, protección de ramas y si los bloques ejecutables deben utilizar datos de prueba.

11. Integración con GitFlow, SDD y CI/CD

Momento	Cambio técnico	Evidencia documental
feature/matriculas	Nuevo caso de uso y pruebas	Página .qmd, enlace a OpenAPI
release/1.2	Validación de compatibilidad	Notas de versión y ADR si aplica
hotfix/1.2.1	Corrección en producción	Runbook / incidente actualizado
Pull request	Pruebas y revisión	Render Quarto verificado

El objetivo es impedir el desfase entre código y documentación. La especificación SDD define el cambio; los diagramas y los ADR explican el diseño; el contrato OpenAPI define la interfaz; Quarto publica la guía integral.

12. Laboratorio guiado: portal técnico de matrículas

Meta: crear un sitio Quarto de cuatro secciones que permita a una persona nueva comprender, ejecutar y diagnosticar el backend.

• Paso 1. Crear proyecto de sitio, comprobar instalación y generar página inicial.

• Paso 2. Organizar navegación: Arquitectura, API, Desarrollo y Operación.

• Paso 3. Documentar C1 y C2 con diagramas exportados o Mermaid auxiliar.

• Paso 4. Documentar POST /matriculas, el contrato universal y tres errores de negocio.

• Paso 5. Redactar ADR-001 sobre el monolito modular.

• Paso 6. Incorporar README de instalación y runbook de restauración simulado.

• Paso 7. Ejecutar `quarto render`, revisar navegación, enlaces y fragmentos de código.

• Paso 8. Publicar en Pages en un repositorio de práctica y adjuntar URL y commit.

Entregables y evidencias

Entregable	Criterio observable
Código fuente .qmd + _quarto.yml	Compila desde un clon limpio
Sitio navegable	Secciones y rutas accesibles
API y arquitectura	Coherentes con código / especificaciones
ADR y guía de operación	Motivaciones y procedimientos claros
Pruebas de publicación	URL o evidencia de render sin errores

13. Rúbrica de evaluación (2 / 1 / 0)

Criterio	2 puntos	1 punto	0 puntos
Estructura	Navegación coherente	Incompleta	Ausente
Contenido	Preciso y verificable	Parcial	Incorrecto
Trazabilidad	Enlaces a código y API	Algunos enlaces	Sin enlaces
Reproducibilidad	Render correcto	Con ajustes	No renderiza
Presentación	Legible y accesible	Detalles por corregir	Deficiente
Gestión Git	Cambios revisables	Historial confuso	Sin versionado

Interpretación: 12 puntos posibles. Se recomienda exigir además que la generación del sitio finalice sin errores como criterio mínimo independiente del puntaje.

14. Autoevaluación

☐ ¿En qué se diferencia Quarto de Swagger UI?

☐ ¿Por qué la especificación OpenAPI debe ser fuente de verdad contractual?

☐ ¿Qué archivos mínimos necesita un proyecto Quarto website?

☐ ¿Qué riesgos hay al ejecutar notebooks durante el render de CI?

☐ ¿Cómo evitar la divergencia entre un ADR, un diagrama C4 y el código?

☐ ¿Cuándo conviene GitHub Pages y cuándo se requiere un servidor protegido?

15. Lista de verificación final

☐ Sitio compila desde un clon limpio.

☐ Los enlaces internos y recursos gráficos funcionan.

☐ No hay claves, secretos ni datos personales en las fuentes.

☐ C4, ADR, OpenAPI y documentación son coherentes.

☐ Existe guía de instalación y guía de operación.

☐ La publicación está versionada y tiene responsable.

☐ El sitio es legible en escritorio y móvil.

16. Glosario y fuentes de consulta

QMD: documento Quarto Markdown. YAML: formato de configuración. Render: transformación de fuentes a entregables. Docs as Code: gestión de documentación con prácticas de ingeniería. Freeze: mecanismo de reutilización de resultados de ejecución. ADR: registro de decisiones de arquitectura.

https://quarto.org/docs/guide/

https://quarto.org/docs/websites/

https://quarto.org/docs/projects/quarto-projects.html

https://quarto.org/docs/books/

https://quarto.org/docs/publishing/github-pages.html

https://quarto.org/docs/authoring/diagrams.html

Nota didáctica: los ejemplos son modelos iniciales; los nombres de carpetas, permisos y pipelines deberán adaptarse al repositorio y a las políticas de la institución.

SERIE BACKEND PROFESIONAL · GUÍA DIDÁCTICA 21

GitHub Spec Kit: del requisito al código verificable

Una guía aplicada de instalación, inicialización, agentes de programación, especificaciones, artefactos, comandos y control de calidad para construir software con SDD.

Caso transversal: API de matrículas académicas con Django REST Framework, PostgreSQL, contrato universal de respuesta, GitFlow, C4 y documentación Quarto.

Audiencia: estudiantes universitarios de Computación. Conocimientos previos: Git, HTTP/REST, Python y fundamentos de SDD.

Aprendizaje esperado	Producto verificable
Preparar un proyecto con Spec Kit	Instalación comprobada y estructura inicial
Ejecutar un ciclo de trabajo con agente de IA	Constitución, especificación, plan, tareas y evidencias
Evaluar la correspondencia especificación-código	Pruebas, análisis y convergencia documentados

1. ¿Qué es GitHub Spec Kit?

GitHub Spec Kit es un conjunto de herramientas para organizar el desarrollo basado en especificaciones con agentes de programación. Proporciona una CLI para preparar el proyecto y habilidades/instrucciones para que un agente genere y verifique artefactos de desarrollo.

No reemplaza Git, un framework, un sistema de pruebas o una metodología de gestión. Es una estructura de trabajo que hace explícitas la intención, el diseño, las tareas y la comprobación de conformidad.

Diferencias que debes recordar

Concepto	Propósito
SDD	Enfoque: desarrollar a partir de especificaciones verificables.
Spec Kit	Herramienta y flujo para ejecutar SDD con un agente.
specify (CLI)	Inicializar, comprobar y consultar la instalación. Se usa en terminal.
/speckit-*	Habilidades/instrucciones que se ejecutan en el chat del agente, no en la terminal.
GitFlow	Convención de ramas que puede convivir con Spec Kit, pero no es obligatoria.

2. Requisitos e instalación

La documentación oficial describe Python 3.11+, un gestor como uv y un agente de programación compatible. Git es opcional en la configuración base; la extensión Git habilita flujos específicos de repositorio.

```
# Comprobar herramientas desde la terminal
python --version
uv --version

# Instalar desde PyPI
uv tool install specify-cli

# Validar instalación
specify version
specify check
```

Para proyectos de docencia o producción, conviene fijar una versión aprobada del paquete y documentarla en la bitácora del equipo. La sintaxis exacta puede evolucionar entre versiones.

Inicializar un proyecto nuevo

```
specify init matriculas-academicas --integration copilot
cd matriculas-academicas
# Abrir esta carpeta en el editor e iniciar el agente
```

Inicializar en un repositorio existente

```
# En el directorio del repositorio existente:
specify init --here --integration copilot
# Revisar los archivos nuevos antes de confirmarlos en Git.
```

Precaución: no utilizar opciones de sobrescritura forzada sobre un repositorio con trabajo sin respaldar. Seguir la guía oficial de adopción para bases de código existentes.

3. Estructura y artefactos del proyecto

La ubicación concreta puede cambiar según la versión y la integración seleccionada. Es más importante reconocer el propósito que memorizar una ruta fija.

```
matriculas-academicas/
  .specify/
    memory/                # constitución del proyecto
    templates/             # plantillas del flujo
    scripts/               # auxiliares generados
    feature.json           # característica activa (según versión)
  specs/
    <feature>/
      spec.md              # comportamiento esperado
      plan.md              # diseño técnico
      tasks.md             # secuencia de trabajo
      ...                  # investigación, modelos, contratos
  src/                     # implementación (según proyecto)
  tests/                   # pruebas automatizadas
  docs/                    # C4, ADR, Quarto y guías
```

El repositorio sigue siendo la fuente versionada. Los artefactos generados requieren revisión; nunca se presumen correctos solo por haberlos producido un agente.

4. Flujo completo recomendado

Orden	Comando de agente (modo skills)	Qué produce o controla
0	/speckit-constitution	Principios no negociables del proyecto (una vez al inicio).
1	/speckit-specify	Especificación de la funcionalidad; qué y por qué.
2	/speckit-clarify	Ambigüedades resueltas y supuestos documentados.
3	/speckit-plan	Arquitectura, stack, modelos y contratos.
4	/speckit-checklist	Lista de verificación de calidad de la especificación.
5	/speckit-tasks	Tareas ejecutables y sus dependencias.
6	/speckit-analyze	Coherencia entre artefactos antes de programar.
7	/speckit-implement	Implementación incremental y ejecución de tareas.
8	/speckit-converge	Compara código y especificación; detecta tareas pendientes.

La forma de invocación depende del agente. Algunos muestran comandos con puntos (/speckit.specify), guiones (/speckit-specify) o prefijos especiales. La tabla utiliza la presentación skills empleada en el Quickstart oficial para Copilot; confirma las instrucciones que ofrece tu integración.

5. Constitución: las reglas de todo el backend

La constitución fija principios que deben cumplirse en todas las funcionalidades. Es mejor escribir reglas comprobables en vez de declaraciones genéricas como «código limpio».

```
/speckit-constitution Define una constitución para un backend
institucional: monolito modular Django/DRF y PostgreSQL;
contrato universal success/message/data/errors/meta;
autorización por objeto, validaciones y pruebas obligatorias;
```

```
OpenAPI, C4 y ADR actualizados; GitFlow y CI; ningún secreto
en el repositorio; auditoría y request_id para operaciones críticas.
```

Control docente: solicitar al grupo que explique por qué cada regla existe y qué prueba o evidencia demostraría su cumplimiento.

6. Especificación: del problema a los criterios

Al especificar, evitar fijar tecnologías prematuramente: describir actores, necesidades, reglas, condiciones y resultados. La elección de herramientas corresponde al plan.

```
/speckit-specify Implementar la matrícula de un estudiante
para una oferta académica. Debe validar período habilitado,
prerrequisitos, disponibilidad de cupos, ausencia de matrícula
duplicada y autorización. Documentar casos felices, errores,
reintentos y concurrencia por el último cupo. Incluir criterios
de aceptación verificables y escenarios negativos.
```

Ejemplo de criterios de aceptación

Código	Condición comprobable
CA-01	Si hay cupo, período abierto y requisitos válidos, la API crea la matrícula.
CA-02	No pueden existir dos matrículas activas del mismo estudiante en la misma oferta.
CA-03	Un estudiante no puede consultar ni modificar matrículas ajenas.
CA-04	Si dos estudiantes compiten por el último cupo, como máximo uno lo obtiene.

7. Clarificar antes de diseñar

Preguntas comunes: ¿matrícula inmediata o pendiente de aprobación?, ¿se permite anular?, ¿qué sucede ante un reintento?, ¿se reserva el cupo?, ¿el administrador puede crear matrículas en nombre de otro usuario?

```
/speckit-clarify Revisa las ambigüedades de matrícula:
reglas de duplicidad, estados permitidos, cupos y permisos.
No supongas políticas institucionales sin confirmación;
marca decisiones pendientes explícitamente.
```

Las decisiones deben quedar reflejadas en la especificación y revisarse con responsables del negocio. Sin esto, la IA podría transformar supuestos en requisitos inexistentes.

8. Planificar la solución

Con requisitos claros, incorporar arquitectura C4, contratos HTTP, seguridad, transacciones y operaciones. En este punto sí corresponde declarar el stack.

```
/speckit-plan Implementar con Django REST Framework,
PostgreSQL y arquitectura modular por capas. Definir modelos,
servicios de aplicación, dominio y repositorios cuando aporten
valor. Crear OpenAPI /api/v1, envelope universal, permisos,
transacciones seguras y pruebas concurrentes. Documentar C4,
ADR y estrategia de trazabilidad de solicitudes.
```

Decisiones que deben justificarse

Decisión	Razón / evidencia
Monolito modular	Reduce complejidad operativa y permite evolución por módulos.
PostgreSQL	Restricciones transaccionales y concurrencia verificable.
select_for_update	Bloqueo explícito cuando corresponda en operación de cupos.
Contrato universal	Consumo uniforme por frontend, móvil e integraciones.
HTTP 409	Conflictos de negocio como matrícula duplicada o cupo agotado.

9. Checklists, tareas y análisis

Las listas de comprobación valoran la calidad de requisitos: si cada historia tiene pruebas, si se describen permisos, si los estados y excepciones están resueltos. No deben confundirse con la ejecución de las pruebas.

```
/speckit-checklist Genera un checklist para requisitos
funcionales, seguridad, datos, concurrencia, errores y API.

/speckit-tasks Divide el trabajo en tareas pequeñas con
identificadores, dependencias y evidencias. Prioriza una
implementación vertical verificable.

/speckit-analyze Busca contradicciones entre constitución,
spec, plan y tareas. Señala requisitos sin cobertura.
```

Fragmento didáctico de tasks.md

```
T001 Crear tablas Oferta y Matricula con constraints.
T002 Implementar validar_prerrequisitos().
T003 Crear servicio registrar_matricula() transaccional.
T004 Agregar permisos por objeto a endpoints.
T005 Aplicar envelope universal y errores estructurados.
T006 Probar concurrencia del último cupo con PostgreSQL.
T007 Actualizar OpenAPI, C4, ADR y guía Quarto.
T008 Ejecutar CI y resolver divergencias.
```

10. Implementar y converger

Implementar por lotes pequeños. El agente debe ejecutar pruebas y reportar qué pudo comprobar. No aprobar cambios automáticamente cuando falten dependencias, permisos o accesos a servicios.

```
/speckit-implement Implementa las tareas aprobadas,
respeta la constitución, agrega pruebas y reporta los
archivos cambiados, comandos ejecutados y fallos pendientes.

/speckit-converge Compara implementación con requisitos,
plan, tareas y pruebas; registra brechas verificables y
crea tareas faltantes si son necesarias.
```

Si convergencia encuentra defectos, resolverlos y volver a ejecutar el análisis. La palabra «converged» es una señal del flujo, no una garantía independiente de que el sistema sea seguro o apto para producción.

11. Ejemplo de contrato universal

```
HTTP/1.1 409 Conflict
Content-Type: application/json

{
  "success": false,
  "message": "No existen cupos disponibles.",
  "data": null,
  "errors": {
    "code": "CUPOS_AGOTADOS",
    "details": [{"field":"oferta_id",
                 "message":"La oferta no tiene cupos."}]
  },
  "meta": {
    "request_id": "req_ejemplo",
    "timestamp": "2026-10-13T18:00:00Z",
    "version": "v1"
  }
}
```

Los endpoints JSON usan la envoltura acordada, sin desvirtuar los códigos HTTP. Las respuestas 204/304 y HEAD son excepciones legítimas porque no incluyen cuerpo.

12. Gestión de ramas y trabajo colaborativo

Spec Kit no obliga a usar GitFlow. En proyectos docentes que sí lo adopten, el equipo puede crear feature/matriculas-cupos desde develop, llevar en esa rama especificación/código/pruebas y abrir un pull request con evidencia.

Según la documentación actual, el núcleo de Spec Kit no exige Git; la extensión Git integra funciones de control de versiones. Cambiar de rama por sí solo no garantiza cambiar la característica activa registrada por la herramienta.

```
# Ejemplo externo a Spec Kit: GitFlow del equipo
git switch develop
git switch -c feature/matriculas-cupos
# Implementar y revisar
pytest -q
git add .
git commit -m "feat: validar cupos de matrícula"
```

13. Integración con los artefactos de la serie

Documento técnico	Papel en el ciclo Spec Kit
BPMN ejecutivo/operativo/técnico	Representa el proceso antes de desarrollar y permite rastrear actividades.
C4 C1-C4	Expone contexto, despliegue lógico, componentes y clases importantes.
OpenAPI	Define y verifica contratos entre clientes y API.
ADR	Explica decisiones, alternativas y consecuencias.
Pruebas automatizadas	Verifican criterios de aceptación y previenen regresión.
GitFlow y CI/CD	Controlan revisión, integración y despliegues.
Quarto	Publica especificaciones, arquitectura y guía de operación.

14. Laboratorio: módulo de matrícula académica

Paso 1. Preparar repositorio, instalar Spec Kit e inicializar la integración elegida.

Paso 2. Redactar una constitución técnica verificable que incorpore contrato universal y seguridad.

Paso 3. Especificar matrícula, validación de cupos, cancelación y consulta por propietario.

Paso 4. Resolver ambigüedades de estados, permisos, duplicidad y concurrencia.

Paso 5. Planificar arquitectura por capas, C4, modelo relacional y endpoints OpenAPI.

Paso 6. Crear checklist, tareas y analizar consistencia entre los documentos.

Paso 7. Implementar endpoints, reglas, transacciones, permisos y respuestas de error.

Paso 8. Ejecutar pytest en PostgreSQL, documentar resultados y errores encontrados.

Paso 9. Ejecutar convergencia; actualizar requisitos y código cuando existan discrepancias.

Paso 10. Publicar sitio Quarto y abrir PR con evidencias de CI y revisión de pares.

15. Matriz de verificación y evaluación

Criterio	Evidencia requerida
Configuración reproducible	Versión CLI, agente integrado, instrucciones para instalar.
Especificación verificable	Historias, escenarios, reglas, estados y criterios comprobables.
Plan coherente	Arquitectura, OpenAPI y ADR consistentes con las restricciones.
Trazabilidad	Requisitos vinculados a tareas y pruebas.
Calidad del código	Pruebas, permisos, control transaccional, contrato universal.
Convergencia real	Registro de brechas, correcciones y verificación final.

Rúbrica académica (2 / 1 / 0)

Calificar los seis criterios anteriores con 2 si son completos y reproducibles, 1 si presentan omisiones parciales y 0 si están ausentes o no se pueden comprobar. Máximo: 12 puntos. La valoración final debe basarse en evidencia, no en cantidad de prompts.

16. Problemas frecuentes y soluciones

Problema	Diagnóstico / solución
Comando no reconocido	Verificar integración del agente y la sintaxis mostrada por su interfaz.
specify no disponible	Verificar PATH e instalación con specify version.
Funcionalidad equivocada	Inspeccionar estado de característica activa y directorio .specify.
Inconsistencias spec/código	Ejecutar análisis y convergencia; corregir artefactos y revalidar.
IA inventa reglas	Volver a clarificar supuestos y solicitar aprobación del responsable.
Pruebas de concurrencia engañosas	Usar PostgreSQL real y casos simultáneos repetibles.

17. Autoevaluación

- ¿Qué se ejecuta en terminal y qué se ejecuta en el chat del agente?
- ¿En qué se diferencia especificar de planificar?
- ¿Cuándo deben ejecutarse clarificación y análisis de consistencia?
- ¿Cómo compruebas que una regla de negocio tiene cobertura de pruebas?
- ¿Por qué convergencia no sustituye una revisión de seguridad independiente?

18. Fuentes oficiales y lecturas sugeridas

- https://github.com/github/spec-kit
- https://github.com/github/spec-kit/blob/main/docs/quickstart.md
- https://github.com/github/spec-kit/blob/main/docs/reference/core.md
- https://github.com/github/spec-kit/blob/main/docs/reference/agentic-sdd.md
- https://github.github.com/spec-kit/installation.html
- https://github.github.com/spec-kit/guides/existing-projects.html

Nota de vigencia: sintaxis y archivos pueden cambiar según la versión de Spec Kit y la integración del agente. Consultar las instrucciones efectivas del proyecto al ejecutar el laboratorio.

GUÍA DIDÁCTICA 20

Desarrollo Basado en Especificaciones (SDD)

De la intención al software verificable con Spec Kit

Serie Backend Profesional · Nivel universitario · Ejemplo transversal: sistema de matrículas académicas.

Objetivo: comprender, aplicar y evaluar un flujo de desarrollo donde la especificación guía el diseño, la implementación y la verificación; la IA es un asistente, no un sustituto de la responsabilidad técnica.

Resultados de aprendizaje	Evidencias
Definir requisitos comprobables	spec.md con historias, escenarios y criterios
Justificar una solución técnica	plan.md, C4, ADR y contratos OpenAPI
Organizar y ejecutar el trabajo	tasks.md, código, pruebas y revisiones
Demostrar conformidad	Matriz requisito-prueba, reportes y convergencia

Duración sugerida: 2 a 3 sesiones conceptuales y un laboratorio de 6 a 8 horas.

1. ¿Qué es SDD?

Spec-Driven Development (SDD) es un enfoque en el que el desarrollo comienza con una especificación explícita de comportamientos y restricciones, continúa con planificación y tareas trazables, y culmina con la implementación verificada frente a esa especificación.

No equivale a escribir documentos largos ni a pedirle a una IA que genere todo el código. Su propósito es convertir necesidades y decisiones en criterios observables: qué debe ocurrir, para quién, en qué condiciones y cómo se sabrá que funciona.

SDD frente a otros enfoques

Enfoque	Pregunta central	Relación con SDD
Desarrollo ad hoc	¿Qué código escribimos ahora?	SDD exige primero clarificar la intención.
TDD	¿Qué prueba debe pasar?	Las pruebas verifican requisitos definidos en la especificación.
BDD	¿Qué comportamiento observa el usuario?	Sus escenarios pueden alimentar la especificación.
SDD	¿Qué y por qué construir, y con qué evidencias?	Orquesta especificación, plan, tareas y verificación.

2. Principios rectores

• Intención primero: describir el problema sin adelantar prematuramente la solución.

• Claridad y ausencia de ambigüedad: identificar actores, límites, reglas, estados y excepciones.

• Trazabilidad: cada requisito debe vincularse con una tarea y una prueba.

• Verificabilidad: criterios observables, repetibles y medibles.

• Cambio controlado: actualizar artefactos cuando cambian requisitos, decisiones o comportamientos.

• Revisión humana: validar resultados y asumir responsabilidad sobre seguridad, datos y decisiones.

3. Flujo de trabajo de extremo a extremo

Constitución → Especificación → Clarificación → Plan → Tareas → Implementación → Convergencia. El análisis de consistencia y los checklists se realizan cuando el proyecto lo necesita; no son trámites vacíos.

Fase	Entrada	Salida / control
Constitución	Valores y restricciones del proyecto	Principios transversales y criterios no negociables
Especificar	Necesidad del usuario	Historias, reglas, casos límite, criterios de aceptación
Clarificar	Ambigüedades	Preguntas resueltas, supuestos documentados
Planificar	Requisitos aprobados	Arquitectura, modelo de datos, contratos y riesgos

Fase	Entrada	Salida / control
Tareas	Plan técnico	Orden, dependencias, evidencias de completitud
Implementar	Tareas verificables	Código, pruebas, migraciones, documentación
Converger	Implementación y especificación	Informe de brechas, corrección y nueva verificación

4. Ejemplo: matrícula académica

Necesidad inicial (demasiado vaga): «El estudiante debe poder matricularse en asignaturas»; no precisa condiciones, actores, excepciones ni resultados.

Requisito funcional RF-01: Un estudiante autenticado puede solicitar una matrícula en una asignatura ofertada dentro del período habilitado si cumple los prerrequisitos y existe cupo.

Reglas: RN-01 no puede matricular dos veces la misma oferta; RN-02 el cupo no puede ser negativo; RN-03 solo el dueño o un operador autorizado consulta el detalle.

Restricciones: las operaciones deben ser transaccionales; las respuestas JSON siguen el contrato universal; cualquier conflicto por duplicidad devuelve HTTP 409.

Escenarios estilo Given / When / Then

```
Escenario: matrícula válida
Dado que el período está abierto
Y que el estudiante cumple prerrequisitos
Y que hay un cupo disponible
Cuando envía la solicitud de matrícula
Entonces se registra una matrícula aprobada
Y se devuelve HTTP 201 con el contrato universal

Escenario: último cupo agotado
Dado que no hay cupos disponibles
Cuando el estudiante intenta matricularse
Entonces no se crea el registro
Y se devuelve HTTP 409 con code=CUPOS_AGOTADOS
```

5. Artefactos que deben existir

Archivo / elemento	Contenido obligatorio	Qué revisa el docente
constitution.md	Seguridad, estándares, calidad y disciplina de cambios	Consistencia de principios
spec.md	Historias, requisitos, reglas, supuestos y criterios	Completitud, claridad, verificabilidad
plan.md	Tecnologías, arquitectura, datos, API, riesgos	Viabilidad y congruencia
tasks.md	Tareas pequeñas, orden y dependencias	Trazabilidad y tamaño de trabajo
OpenAPI + C4 + ADR	Contratos, componentes, decisiones	Diseño sustentado
Pruebas y reportes	Casos, resultados y cobertura relevante	Evidencias reproducibles

Los nombres y ubicaciones pueden cambiar según la versión de la herramienta y la organización del repositorio; el valor reside en sus responsabilidades, no en el nombre del archivo.

6. Ejemplo de constitución técnica

```
# Constitution (extracto)
1. Toda API JSON responde con success, message, data, errors, meta.
2. Toda regla crítica tiene al menos una prueba automatizada.
3. Autorización por objeto obligatoria en recursos privados.
4. Ningún secreto se guarda en Git.
5. Migraciones reversibles cuando sea viable; cambios destructivos,
   sujetos a plan de respaldo y despliegue.
6. OpenAPI y diagramas C4 deben reflejar la implementación.
7. CI no admite merge si fallan pruebas o verificaciones de seguridad.
```

7. Especificar: cómo redactar bien

Una buena historia describe actor, necesidad y beneficio. Una buena condición de aceptación agrega contexto, acción y resultado medible. Debe evitar verbos imprecisos como «ser amigable», «funcionar bien» o «ser rápido» sin criterios observables.

Ejemplo de aceptación comprobable

Código	Criterio verificable
CA-01	Con cupo y prerrequisitos válidos, POST /api/v1/matriculas devuelve 201.
CA-02	Si ya existe matrícula activa de la misma oferta, devuelve 409.
CA-03	Un estudiante no puede consultar matrículas de otro: devuelve 403 o 404 según política documentada.
CA-04	Ante dos solicitudes concurrentes por el último cupo, como máximo una obtiene matrícula.

8. Planificación técnica

El plan transforma el qué en un cómo razonado: monolito modular, Django REST Framework, PostgreSQL, capas de presentación/aplicación/dominio/infraestructura, OpenAPI, autenticación y observabilidad.

Debe producir o actualizar los diagramas C4 C1-C3; el nivel de código C4 se reserva para componentes donde proporcione valor. Documentar ADR para decisiones importantes, por ejemplo el uso de bloqueo de filas y una restricción única para proteger el último cupo.

Relación con BPMN, C4 y OpenAPI

BPMN describe el proceso de solicitud y validación; C4 explica dónde se ejecutan sus responsabilidades; OpenAPI define las interfaces HTTP; las pruebas evidencian que el comportamiento documentado es real.

9. Preparar tareas ejecutables

```
T001 [P] Crear modelo OfertaAcademica y migración.
T002 [P] Crear modelo Matricula y restricción de unicidad.
T003 Implementar servicio registrar_matricula con transacción.
T004 Publicar POST /api/v1/matriculas con permisos y envelope.
T005 Añadir casos unitarios de prerrequisitos y cupos.
T006 Añadir prueba de integración para último cupo.
T007 Actualizar esquema OpenAPI, ADR y guía Quarto.
T008 Ejecutar CI, revisar hallazgos y corregir divergencias.
```

[P] denota tareas potencialmente paralelizables si sus dependencias lo permiten. Cada tarea debe tener una definición explícita de terminación y evidencia.

10. Implementación de ejemplo (Django)

```
from django.db import transaction
from django.core.exceptions import ValidationError

@transaction.atomic
def registrar_matricula(*, estudiante, oferta_id):
    oferta = (OfertaAcademica.objects
              .select_for_update().get(pk=oferta_id))
    if oferta.cupos_disponibles <= 0:
        raise CuposAgotados("No existen cupos disponibles")
    if Matricula.objects.filter(
        estudiante=estudiante, oferta=oferta, activa=True
    ).exists():
        raise MatriculaDuplicada("La matrícula ya existe")
    # Verificar prerrequisitos antes de confirmar.
    validar_prerrequisitos(estudiante, oferta)
    matricula = Matricula.objects.create(
        estudiante=estudiante, oferta=oferta, activa=True
    )
    oferta.cupos_disponibles -= 1
    oferta.save(update_fields=["cupos_disponibles"])
    return matricula
```

Este es un ejemplo pedagógico parcial: requiere restricciones en base de datos, política de autorización, estrategia para reintentos e integración con el manejador de excepciones del proyecto.

11. Contrato universal de la API

```
{
  "success": false,
  "message": "No existen cupos disponibles.",
  "data": null,
  "errors": {
    "code": "CUPOS_AGOTADOS",
    "details": [{"field": "oferta_id",
                 "message": "La oferta ya no dispone de cupos."}]
  },
  "meta": {
    "request_id": "req_abc",
    "timestamp": "2026-10-13T15:00:00Z",
    "version": "v1"
  }
}
```

Mantener HTTP 409 para conflictos, 400/422 para errores de entrada según contrato, 401/403 para autenticación/permisos y 500 para fallos internos. HTTP 204, 304 y HEAD no deben llevar cuerpo JSON.

12. Validación: de la especificación a las pruebas

Requisito	Tarea	Evidencia esperada
CA-01	T003, T004	test_matricula_exitosa → 201
CA-02	T002, T003	test_duplicado → 409 + código
CA-03	T004	test_permiso_objeto → acceso denegado
CA-04	T003, T006	Prueba concurrente en PostgreSQL; nunca dos éxitos

```
def test_no_permita_matricula_duplicada(api_client, estudiante, oferta):
    api_client.force_authenticate(estudiante.usuario)
    payload = {"oferta_id": oferta.id}
    primera = api_client.post("/api/v1/matriculas/", payload)
    segunda = api_client.post("/api/v1/matriculas/", payload)
    assert primera.status_code == 201
    assert segunda.status_code == 409
    assert segunda.data["success"] is False
    assert segunda.data["errors"]["code"] == "MATRICULA_DUPLICADA"
```

Para pruebas realmente concurrentes debe usarse un motor que soporte los bloqueos previstos, como PostgreSQL; SQLite no es sustituto suficiente para verificar ese escenario.

13. Spec Kit: flujo de uso actual

Spec Kit es una herramienta de código abierto que estructura el trabajo con agentes de programación. La documentación vigente recomienda preparar una constitución una vez por proyecto y, por funcionalidad, seguir especificación, planificación, tareas, implementación y convergencia. Las invocaciones siguientes son instrucciones del chat del agente, no comandos de terminal.

```
# Terminal (instalación y creación del proyecto)
uv tool install specify-cli
specify init matriculas --integration copilot

# Chat del agente (se ejecutan por separado)
/speckit-constitution
/speckit-specify
/speckit-clarify
/speckit-plan
/speckit-tasks
/speckit-implement
/speckit-converge
```

La clarificación es especialmente importante cuando existen decisiones de negocio sin resolver. También pueden utilizarse análisis y listas de comprobación antes de implementar. Comprueba la sintaxis compatible con tu agente y la versión instalada.

14. Control de calidad y convergencia

La convergencia no es que el agente afirme «terminé»; es que exista evidencia de coincidencia entre requisitos, diseño, contratos, código, tests y documentación. Cuando hay discrepancias, se corrige el artefacto adecuado y se vuelve a verificar.

Control	Pregunta de revisión
Especificación	¿Todos los criterios son observables y no contradictorios?
Seguridad	¿Permisos por objeto, validaciones y límites están probados?
Integridad	¿Se protegen simultaneidad, duplicados y transacciones?
Contrato API	¿Todas las respuestas pertinentes siguen el envelope?
Documentación	¿C4, OpenAPI, ADR y Quarto concuerdan con el código?
Entrega	¿CI, revisión y pruebas de regresión pasan?

15. Integración con GitFlow, CI/CD y Quarto

Cada cambio se desarrolla en una rama feature, se revisa mediante pull request contra develop y se integra únicamente si supera las verificaciones. GitFlow no está impuesto por Spec Kit: es una convención del proyecto.

Quarto publica el relato técnico del proyecto: especificaciones, decisiones, diagramas y guías. CI debe verificar formatos, tests, calidad, seguridad y, cuando sea viable, generación de documentación.

16. Laboratorio integrador para estudiantes

Actividad 1. Crear la constitución del repositorio y justificar cinco políticas innegociables.

Actividad 2. Especificar matrícula, cancelación y consulta de matrícula con escenarios normales y excepciones.

Actividad 3. Diseñar C1, C2 y C3; diagramar clases clave en UML; documentar dos ADR.

Actividad 4. Construir OpenAPI de tres endpoints y el contrato universal de respuesta.

Actividad 5. Dividir el desarrollo en tareas trazables; implementar una iteración vertical.

Actividad 6. Escribir tests de negocio, API, permisos y concurrencia con PostgreSQL.

Actividad 7. Ejecutar checklist y análisis de consistencia; solucionar discrepancias.

Actividad 8. Publicar documentación Quarto y entregar demostración con evidencias CI.

Entregables

Repositorio Git con historial de cambios; constitución; spec; plan; tasks; C4/ADR; OpenAPI; backend funcional; pruebas; reporte de convergencia; portal Quarto y memoria reflexiva sobre decisiones y uso declarado de IA.

17. Rúbrica analítica (2 / 1 / 0)

Criterio	2 puntos	1 punto	0 puntos
Especificación	Clara, completa, verificable	Parcial o ambigua	Ausente
Plan y arquitectura	Coherentes y justificados	Incompletos	No corresponden
Trazabilidad	Requisitos ↔ tareas ↔ tests	Vínculos parciales	Sin trazabilidad
Implementación	Cumple casos y políticas	Funciona parcialmente	No cumple
Seguridad y calidad	Permisos, validaciones y CI	Evidencia parcial	Sin controles
Documentación	Actualizada y reproducible	Incompleta	Ausente

Puntaje máximo: 12 puntos. Puede convertirse proporcionalmente a la escala de evaluación institucional.

18. Autoevaluación

- ¿Qué diferencia una especificación verificable de una lista de deseos?
- ¿Por qué debe separarse el qué del cómo durante la definición?
- ¿Qué documentos deben actualizarse al modificar una regla de matrícula?
- ¿Qué evidencia permite afirmar que el sistema convergió con la especificación?
- ¿Cómo impedirías que la IA introduzca funcionalidades no solicitadas?

19. Glosario y referencias

SDD: desarrollo guiado por especificaciones. Constitución: reglas rectoras del proyecto. Convergencia: verificación y resolución de brechas entre intención y producto. ADR: registro de decisiones arquitectónicas. Criterio de aceptación: condición observable de cumplimiento.

Fuentes oficiales consultadas (octubre de 2026):

- https://github.com/github/spec-kit
- https://github.com/github/spec-kit/blob/main/docs/concepts/sdd.md
- https://github.github.io/spec-kit/quickstart.html
- https://c4model.com/
- https://spec.openapis.org/oas/latest.html

SERIE BACKEND PROFESIONAL · GUÍA DIDÁCTICA 19

BPMN en tres niveles: ejecutivo, operativo y técnico

Modelado de procesos de negocio desde la dirección institucional hasta su implementación en software

Caso transversal: proceso de matrícula académica · Audiencia: estudiantes de Computación · Herramientas: modeladores BPMN 2.0, Django REST Framework, Quarto y Git.

Resultados de aprendizaje. Al finalizar, el estudiante podrá diferenciar las tres perspectivas de representación, modelar eventos, tareas y compuertas con semántica correcta, refinar un proceso de negocio hasta especificaciones técnicas y justificar su correspondencia con servicios de backend.

Nota normativa. Ejecutivo, operativo y técnico son tres vistas didácticas por audiencia propuestas en esta guía: no son niveles oficiales de BPMN. El estándar de referencia es BPMN 2.0.2, publicado por Object Management Group (OMG).

Ruta de aprendizaje

Unidad	Pregunta que resuelve	Producto
Ejecutivo	¿Qué proceso existe y qué valor produce?	Mapa de proceso simplificado
Operativo	¿Quién hace qué, cuándo y con qué decisiones?	Colaboración BPMN detallada
Técnico	¿Cómo se soporta o automatiza?	Tareas de servicio, eventos y contratos

1. Fundamentos de BPMN

BPMN significa Business Process Model and Notation. Es una notación estandarizada para representar procesos, actividades, decisiones, eventos e interacciones entre participantes. Su meta es que analistas, responsables del negocio y especialistas técnicos compartan una representación entendible.

Conceptos imprescindibles

Elemento	Representación conceptual	Uso
Evento	Círculo	Inicio, recepción, espera o finalización
Actividad / tarea	Rectángulo con esquinas redondeadas	Trabajo realizado por persona o sistema
Subproceso	Actividad expandible o colapsada	Agrupa pasos con mayor nivel de detalle
Compuerta exclusiva (XOR)	Rombo con X	Selecciona una ruta según condiciones
Compuerta paralela (AND)	Rombo con +	Divide o sincroniza caminos simultáneos
Flujo de secuencia	Flecha continua	Orden dentro de un proceso
Flujo de mensaje	Línea discontinua	Comunicación entre participantes/pools
Pool / lane	Participante / carril	Responsabilidades y colaboración

Reglas que evitan errores

• Los flujos de secuencia no atraviesan límites de pools; entre participantes se emplean flujos de mensaje.

• Una lane organiza actividades dentro de un participante y no equivale necesariamente a un sistema desplegable.

• Una compuerta exclusiva representa rutas alternativas; una paralela permite ejecución simultánea.

• Cada diagrama debe tener un propósito, un límite definido y una complejidad apropiada para sus lectores.

Tres perspectivas, un mismo proceso

Ejecutivo describe objetivos y resultados; operativo explica responsabilidades y escenarios; técnico detalla interacciones y automatización. Las perspectivas deben ser coherentes, pero no es necesario reproducir todos los elementos de una en otra.

2. Nivel ejecutivo: visión institucional

Audiencia: autoridades, responsables de procesos y tomadores de decisiones. Objetivo: comunicar valor, participantes relevantes, hitos, decisiones principales e indicadores, sin detalles de implementación.

Ejemplo: proceso de matrícula

Solicitud Estudiante inicia	Validación Elegibilidad y periodo	Decisión ¿Cumple condiciones?	Resultado Matrícula o rechazo

El esquema anterior es un resumen conceptual para dirección; no es un diagrama BPMN formal. Para formalizarlo, se deben usar eventos, actividades, compuertas y flujos propios de la notación.

Elementos mínimos del ejecutivo

• Objetivo: formalizar la matrícula de estudiantes habilitados.

• Inicio: solicitud de matrícula; fin: matrícula confirmada o notificación de rechazo.

• Participantes: estudiante, unidad académica y sistema de matrícula.

• Reglas principales: periodo vigente, prerrequisitos, cupo disponible y ausencia de duplicidad.

• Indicadores propuestos: tiempo total del proceso, tasa de éxito, rechazos por causa y proporción automatizada.

Ficha de proceso

Campo	Ejemplo
Código / nombre	PRO-MAT-01 / Gestionar matrícula académica
Propietario	Unidad de gestión académica
Disparador	Solicitud del estudiante
Entrada	Solicitud y datos de asignaturas
Salida	Matrícula confirmada o respuesta motivada
KPI	Tiempo p95 de trámite, % de matrículas finalizadas
Riesgos	Cupo concurrente, periodos cerrados, duplicidades

3. Nivel operativo: actividades y responsabilidades

Audiencia: usuarios, analistas de procesos y personal responsable de ejecutarlos. Objetivo: describir quién hace qué, condiciones de decisión, rutas alternativas, excepciones y comunicaciones entre participantes.

Descomposición operativa de la matrícula

Paso	Responsable	Actividad / decisión
1	Estudiante	Selecciona asignaturas y presenta la solicitud
2	Sistema	Verifica periodo habilitado y condición académica
3	Sistema	Comprueba prerrequisitos, duplicidad y disponibilidad de cupos
4	Sistema	¿La solicitud cumple todas las reglas?
5A	Sistema	Si cumple, registra matrícula y reserva cupos
5B	Sistema	Si no cumple, genera causas estructuradas
6	Estudiante	Recibe confirmación o información de rechazo

Rutas alternativas y excepciones

• Periodo cerrado: finalizar con rechazo motivado, sin modificar registros.

• Cupo agotado: revalidar al confirmar dentro de transacción; notificar conflicto si corresponde.

• Servicio externo no disponible: definir timeout, reintento controlado o tarea pendiente.

• Solicitud duplicada: devolver estado existente o conflicto, según contrato e idempotencia definida.

Cómo representarlo correctamente

Use un pool para el participante externo Estudiante y otro para la Institución si desea representar su intercambio como colaboración; dentro del pool institucional agregue lanes para los roles de negocio. Si modela solamente el proceso interno, puede usar un pool con sus lanes. Diferencie siempre la interacción por mensajes de la secuencia interna de trabajo.

Una decisión sobre requisitos se expresa con compuerta XOR y dos flujos condicionados: válido e inválido. Para la reserva de cupo, identifique actividades transaccionales; evite suponer que BPMN por sí solo garantiza integridad de base de datos.

4. Nivel técnico: orquestación e implementación

Audiencia: arquitectos, desarrolladores e integradores. Objetivo: relacionar el proceso con servicios, APIs, datos, eventos, jobs y mecanismos de recuperación.

Correspondencia BPMN ↔ backend

Concepto de proceso	Implementación orientativa
Tarea de usuario	Formulario web / móvil con autenticación
Tarea de servicio	Invocación de caso de uso en Django
Regla de decisión	Política de dominio y reglas parametrizadas
Evento de mensaje	Webhook, evento de integración o respuesta externa
Evento de temporizador	Fecha límite, timeout o vencimiento de solicitud
Evento de error	Excepción controlada y flujo de recuperación
Subproceso	Orquestación de caso de uso o flujo de tareas
Dato de proceso	Entidad de dominio y persistencia

Flujo de ejecución sugerido

POST /api/v1/matriculas	Validar permisos y reglas	Transacción reservar cupo	Respuesta contrato universal

Resumen técnico conceptual. El diagrama BPMN ejecutable, si se requiere, debe especificar tipos de tareas, eventos y semántica compatibles con el motor seleccionado. El flujo HTTP no es por sí mismo una especificación BPMN.

Ejemplo de reglas técnicas

```
POST /api/v1/matriculas
Authorization: Bearer <token>
Content-Type: application/json

{"estudiante_id": 25, "asignatura_id": 104}

201 Created  -> success=true;  data={"estado":"CONFIRMADA",...}
409 Conflict -> success=false; errors.code="CUPO_AGOTADO"
422 Unprocessable Content -> error de regla o validacion semantica
```

Para solicitudes asíncronas complejas puede utilizarse 202 Accepted con identificador de trabajo y endpoint de seguimiento. No convierta tareas normales en asíncronas sin una justificación de rendimiento o confiabilidad.

5. Profundización: eventos y compuertas

¿Qué evento utilizar?

Situación	Elemento BPMN sugerido
Estudiante presenta solicitud	Evento de inicio de mensaje, si inicia por mensaje en una colaboración
Solicitud debe llegar antes del cierre	Evento de temporizador para plazo o espera
Falla al verificar datos externos	Evento de error asociado a actividad/subproceso cuando corresponda
Se emite la confirmación	Tarea de envío o evento de mensaje según el diseño
Solicitud termina	Evento de fin explícito

Condiciones simultáneas versus alternativas

Con XOR, si los documentos son inválidos se rechaza, de lo contrario se continúa; con AND se ejecutan, por ejemplo, dos verificaciones independientes a la vez y después se sincroniza. No agregue paralelismo si altera reglas de concurrencia o consistencia.

Subprocesos y trazabilidad

Un subproceso Validar matrícula puede contener: verificar periodo → verificar estudiante → verificar prerrequisitos → verificar cupo. Una instancia de proceso debe poder correlacionarse con `request_id` y `job_id` cuando corresponda. El identificador del proceso no debe incluir datos personales ni secretos.

Atributos de calidad

- Seguridad: controles de acceso en API y en servicios, no únicamente en el diagrama.
- Fiabilidad: reintentos con límites, idempotencia y recuperación después de errores.
- Observabilidad: medir duración del proceso y motivos de excepción.
- Evolución: versionar diagramas y reglas de negocio con el código.

6. Integración con C4, OpenAPI, ADR y Quarto

BPMN representa comportamiento del proceso; C4 describe estructura del sistema. Son complementarios: los participantes y servicios dibujados en BPMN deben tener correspondencia clara con los contenedores/componentes C4, sin exigir una relación de uno a uno.

Artefacto	Qué aporta	Relación con BPMN
C1/C2/C3 (C4)	Entorno y estructura tecnológica	Identifica sistemas responsables de actividades
OpenAPI	Operaciones y esquemas de comunicación	Contratos de tareas de servicio
ADR	Decisiones justificadas	Por qué se usa orquestación síncrona o asíncrona
Pruebas	Evidencia verificable	Verifica rutas normales, alternas y fallos
Quarto	Portal documental versionado	Publica fichas, modelos y ejemplos

Carpetas recomendadas

```
docs/
  procesos/
    01-matricula/
      ejecutivo.qmd
      operativo.qmd
      tecnico.qmd
      matricula.bpmn
      trazabilidad.md
  arquitectura/c4/
  api/openapi.yaml
  decisiones/ADR-001.md
  pruebas/proceso-matricula.md
```

Matriz de trazabilidad mínima

Regla	Actividad	Componente	Prueba
RN-01 Periodo abierto	Verificar periodo	MatriculaService	test_periodo_cerrado
RN-02 Cupo disponible	Reservar cupo	CupoRepository	test_ultimo_cupo
RN-03 Sin duplicados	Validar solicitud	MatriculaService	test_matricula_duplicada

7. Taller práctico guiado

Propósito: modelar un mismo proceso de matrícula en tres perspectivas y conectarlo con la arquitectura del backend.

Actividad A — Ejecutivo

- Redacte objetivo, propietario, disparador, salida, actores y dos KPI.
- Modele un flujo de entre 4 y 7 actividades con una decisión relevante.
- Produzca una ficha de proceso comprensible sin conocimientos de desarrollo.

Actividad B — Operativo

- Identifique participantes y lanes; detalle al menos 8 actividades.
- Incorpore dos alternativas y un error o evento de espera justificado.
- Verifique flujos de mensaje entre pools y flujos de secuencia dentro del pool.

Actividad C — Técnico

- Asigne servicios o casos de uso a las tareas automatizadas.
- Relacione tres actividades con endpoints OpenAPI y sus códigos HTTP.
- Explique el uso de transacción, permisos e idempotencia en confirmación de matrícula.
- Prepare al menos cinco escenarios automatizados de prueba.

Entregables verificables

Entregable	Criterio mínimo
Modelo ejecutivo	Objetivo, inicio/fin, decisión y KPI
Modelo operativo	Responsables, excepciones y flujo semánticamente válido
Modelo técnico	Servicios, condiciones, errores y correlación técnica
Matriz de trazabilidad	Reglas ↔ actividades ↔ APIs ↔ pruebas
Documentación Quarto	Tres páginas .qmd y modelos versionados

8. Evaluación y criterios de calidad

Rúbrica basada en la escala didáctica 2 / 1 / 0: 2 = cumple con evidencia suficiente; 1 = cumplimiento parcial; 0 = no cumple o no presenta evidencia.

Criterio	2 puntos	1 punto	0 puntos
Vista ejecutiva	Clara y orientada a objetivos	Exceso o falta de detalle	Ausente o confusa
Vista operativa	Responsables y rutas correctas	Rutas incompletas	Semántica incorrecta
Vista técnica	Automatización y fallos trazados	Relación técnica parcial	Sin vínculo con implementación
Notación BPMN	Elementos y flujos correctos	Errores menores	Errores conceptuales graves
Trazabilidad	Reglas, API y pruebas conectadas	Vínculos incompletos	No existe
Documentación	Versionada y comprensible	Parcial	No reproducible

Puntaje total: 12 puntos. Esta rúbrica puede adaptarse a los criterios de evaluación de la asignatura.

Preguntas de autoevaluación

- ¿Por qué los tres niveles propuestos no deben presentarse como parte normativa de BPMN?
- ¿Cuál es la diferencia entre un pool y una lane?
- ¿Cuándo corresponde utilizar flujo de mensaje en lugar de secuencia?
- ¿Cómo cambia el detalle de una validación entre el ejecutivo y el técnico?
- ¿Qué pruebas permitirían detectar una reserva doble de último cupo?
- ¿Por qué un diagrama BPMN no sustituye OpenAPI ni C4?

Errores frecuentes

- Dibujar tres diagramas idénticos con títulos distintos; cada vista debe responder a su audiencia.
- Confundir procesos de negocio con clases, métodos o endpoints.
- Hacer cruzar flechas de secuencia entre pools.
- Usar compuertas sin expresar condiciones o sin sincronizar rutas cuando sea necesario.
- Suponer que una imagen BPMN se ejecuta automáticamente en Django.

9. Lista de comprobación final

☐ El proceso tiene propietario, disparador y resultados definidos.

☐ La vista ejecutiva resume valor, decisiones y KPI.

☐ La vista operativa identifica participantes, roles, excepciones y reglas.

☐ La vista técnica relaciona tareas con servicios, contratos y fallos.

☐ La semántica de eventos, compuertas, pools y flujos es correcta.

☐ Los tres modelos son coherentes, aunque tengan distinto grado de detalle.

☐ Las reglas clave están ligadas a pruebas automatizadas.

☐ Los modelos se conservan en formato editable y están versionados.

☐ La documentación incluye versión, responsable y fecha de revisión.

☐ El modelo fue validado por personas del negocio y del equipo técnico.

Glosario breve

Término	Definición
BPMN	Notación estandarizada para modelar procesos de negocio
Evento	Ocurrencia que inicia, afecta o termina un flujo
Tarea	Actividad atómica dentro del proceso modelado
Subproceso	Actividad que contiene otras actividades
Pool	Participante de una colaboración
Lane	Agrupación de responsabilidades dentro de un participante
Gateway	Nodo de divergencia o convergencia de caminos
Correlación	Vinculación de eventos o mensajes a una instancia concreta

Referencias para ampliar

Object Management Group. Business Process Model and Notation (BPMN), Version 2.0.2. https://www.omg.org/spec/BPMN/2.0.2/

Object Management Group. BPMN 2.0.2 — Specification Documents and examples. https://www.omg.org/spec/BPMN/

Modelo C4. Software architecture diagrams. https://c4model.com/diagrams

Quarto. Technical documentation and publishing. https://quarto.org/docs/

Cierre: modelar en tres perspectivas ayuda a mantener continuidad entre estrategia institucional, operación y automatización sin convertir BPMN en un diagrama exclusivamente técnico.

SERIE BACKEND PROFESIONAL · GUÍA DIDÁCTICA 22

Guía Completa de Metodologías de Versionado

Flujos de trabajo Git, estrategias de ramas, versionado semántico, historial de cambios, versionado de APIs e integración con SDD / Spec Kit.

Proyecto transversal: backend de matrículas académicas con Django REST Framework, OpenAPI, C4/UML, pruebas, Quarto y CI/CD.

Propósito: elegir conscientemente cómo colaborar, integrar cambios y publicar versiones reproducibles, seguras y trazables.

Al finalizar podrás...	Evidencia de aprendizaje
Comparar GitFlow, GitHub Flow, GitLab Flow y Trunk-Based	Matriz de selección fundamentada
Definir una política de ramas y revisiones	Documento CONTRIBUTING y reglas de PR
Aplicar SemVer y gestionar releases	CHANGELOG, etiquetas y release notes
Gestionar evolución de contratos API	Política de compatibilidad y deprecación
Conectar SDD, QA y CI/CD	Trazabilidad requisito → PR → versión

1. Fundamentos y vocabulario

Control de versiones registra cambios a archivos y permite identificar, comparar, recuperar e integrar estados del proyecto. Git es un sistema distribuido: cada copia del repositorio dispone del historial. Metodología de ramas define cómo coordinar el trabajo; versionado de producto identifica entregas; versionado de API gobierna compatibilidad para consumidores. Son decisiones relacionadas, pero distintas.

Concepto	Significado y ejemplo
Commit	Unidad de cambio rastreable: “fix(matriculas): impedir duplicados”.
Branch	Línea de evolución de código; ejemplo feature/registro-matricula.
Pull/Merge Request	Propuesta revisable para integrar cambios; incorpora discusión y pruebas.
Tag	Referencia a un commit concreto; ej. v1.2.0.
Release	Versión distribuida con notas, artefactos y condiciones de uso.
Rollback / revert	Recuperar servicio o revertir un cambio con registro histórico.
Compatibilidad	Posibilidad de que consumidores existentes sigan funcionando.

Distinguir tres números de versión

Una aplicación puede tener versión del producto 2.3.1, exponer /api/v1/ y desplegar la revisión de compilación a1b2c3d. No tienen obligación de coincidir. Documenta para qué sirve cada identificador.

2. GitFlow: versiones planificadas

GitFlow combina ramas persistentes main y develop con ramas temporales feature/*, release/* y hotfix/*. Es adecuado cuando hay ventanas de publicación, aprobaciones y versiones en mantenimiento. Genera trabajo adicional de sincronización y puede retardar la integración.

Rama	Parte de	Se integra en	Finalidad
main	Historial de releases	-	Código oficialmente publicado
develop	main al inicio	release o main según práctica	Integración de funcionalidades
feature/*	develop	develop	Crear una característica aislada
release/*	develop	main y develop	Estabilizar una versión
hotfix/*	main	main y develop	Corregir producción rápidamente

Ejemplo guiado

Una funcionalidad de consulta de matrículas se trabaja en feature/listar-matriculas. Tras revisión y pruebas, se integra en develop. La versión release/1.1.0 estabiliza documentación, pruebas y contratos; al aprobarse, llega a main, se etiqueta como v1.1.0 y se sincroniza con develop. Si la versión publicada presenta una brecha de autorización, se crea hotfix/1.1.1 desde main, se prueba, se publica y se incorpora también en develop.

Antipatrones

No desarrollar funcionalidades nuevas en release salvo cambios de estabilización; no fusionar sin pruebas; no mantener ramas feature durante semanas sin sincronizar; no borrar la trazabilidad de una corrección urgente.

3. GitHub Flow: cambios pequeños revisados

GitHub Flow utiliza una rama principal publicable y ramas cortas para cambios. El proceso habitual: crear rama a partir de main → modificar y verificar → abrir Pull Request → revisar y ejecutar CI → integrar → desplegar. Es más simple para servicios web que despliegan frecuentemente.

Situación	Aplicación
Agregar un endpoint GET /matriculas/	Rama corta feat/listar-matriculas, PR, CI y merge a main.
Corregir un error crítico	Rama fix/permiso-objeto, pruebas de regresión y despliegue.
Trabajo incompleto	Ocultar funcionalidad con feature flag o dividir en cambios pequeños.

4. Trunk-Based Development

Los cambios se integran con frecuencia en una rama troncal, normalmente main. Se privilegian incrementos pequeños, CI rápida, pruebas fiables y feature flags. Reduce divergencias, pero exige disciplina técnica: tronco siempre utilizable y capacidad de deshacer cambios.

Ejemplo

La búsqueda avanzada se divide en: filtro por estado, paginación, contrato OpenAPI, pruebas de permisos y documentación. Cada paso se integra al tronco por separado; una bandera controla la exposición del recurso incompleto.

Distinción: GitHub Flow y Trunk-Based comparten ramas cortas e integración frecuente, aunque GitHub Flow formaliza generalmente la revisión mediante Pull Request.

5. GitLab Flow y otros flujos

GitLab Flow combina revisiones mediante Merge Requests con ramas o promociones alineadas con entornos y/o versiones, según la política de despliegue. No existe una única receta universal: la implementación debe documentar claramente qué ramas representan desarrollo, preproducción o producción.

Estrategia	Cuándo aporta valor
GitLab Flow con entornos	Equipos que necesitan promoción controlada entre staging y producción.
Feature Branch Workflow	Aislar y revisar cambios sin imponer todo GitFlow.
Forking Workflow	Colaboración abierta con contribuyentes sin escritura en repositorio principal.
Release branches	Mantener una línea v1.x mientras evoluciona v2.x.

6. Comparación y criterio de elección

Criterio	GitFlow	GitHub Flow	Trunk-Based	GitLab Flow
Ramas duraderas	main + develop	main	main	Según entornos
Frecuencia de despliegue	Planificada	Alta	Muy alta	Variable
Complejidad	Alta	Baja	Media en disciplina	Media
Ventaja	Control de releases	Simplicidad	Integración rápida	Promoción por entornos
Riesgo	Desincronización	CI insuficiente	Tronco inestable	Ambigüedad de ramas

Regla práctica: si el sistema académico tendrá entregas planificadas, versiones institucionales y mantenimiento paralelo, GitFlow puede ser apropiado. Si se desplegará una sola versión frecuentemente y existe CI fiable, GitHub Flow o Trunk-Based pueden ser más eficientes. Justificar por escrito la selección en un ADR.

7. Versionado semántico (SemVer 2.0.0)

SemVer utiliza MAJOR.MINOR.PATCH. MAJOR cambia cuando la API pública introduce incompatibilidades; MINOR incorpora funcionalidad compatible; PATCH corrige defectos sin romper compatibilidad. Puede añadir preversiones: 2.0.0-rc.1 y metadatos: 2.0.0+build.45.

Cambio	Versión de ejemplo	Motivo
Arreglar permisos sin alterar contratos públicos	1.4.2 → 1.4.3	PATCH
Agregar filtro opcional compatible	1.4.3 → 1.5.0	MINOR
Eliminar campo antes obligatorio en consumidores	1.5.0 → 2.0.0	MAJOR
Liberar versión candidata	2.0.0-rc.1	Pre-release

Atención: la versión 0.y.z identifica desarrollo inicial en SemVer y puede contener cambios incompatibles. Una versión inmutable publicada nunca debe reemplazarse silenciosamente por otra con el mismo número.

8. Conventional Commits y mensajes

Conventional Commits agrega significado al historial mediante tipo(ámbito): descripción. Formatos de ejemplo:

```
feat(matriculas): permitir filtro por periodo
fix(permisos): restringir consulta por estudiante
refactor(dominio): extraer servicio de inscripcion
docs(openapi): documentar errores 409
feat(api)!: renombrar recurso de oferta academica
```

feat se relaciona con incremento MINOR y fix con PATCH en automatizaciones compatibles; un cambio marcado como BREAKING CHANGE o con ! indica incompatibilidad potencialmente MAJOR. La clasificación final se revisa respecto al contrato público real y a las reglas del repositorio.

9. Gestión de releases y CHANGELOG

Una release debe poder reconstruirse y auditarse. Conserva tag, commit, versión, fecha, notas de cambios, incidencias resueltas, estado de pruebas, artefacto desplegado y procedimiento de reversión.

```
# CHANGELOG (extracto)
## [1.2.0] - 2026-10-14
### Added
- Filtro de matriculas por periodo.
### Fixed
- Validacion de cupo bajo concurrencia.
### Security
- Restriccion de consulta por estudiante.
### Documentation
- Actualizacion de OpenAPI y diagramas C4.
```

No confundas etiquetas Git con estado real de producción: registra la revisión exacta desplegada en cada entorno. No modificar una release publicada: publica una nueva versión.

10. Versionado de APIs REST

Una API puede usar rutas como /api/v1/matriculas para identificar una familia de contratos. El número de la ruta no debe incrementarse por cada corrección PATCH del backend. Una nueva versión mayor de producto puede seguir exponiendo /api/v1 si el contrato externo no ha cambiado de forma incompatible.

Cambio en contrato	Compatibilidad probable	Acción
Añadir campo opcional	Generalmente compatible	Actualizar OpenAPI y tests de contrato
Eliminar o renombrar campo requerido	Incompatible	Nueva versión o transición pactada
Cambiar tipo de un campo	Potencialmente incompatible	Evaluar nueva versión
Agregar nuevo código de error	Depende de consumidores	Documentar y probar clientes
Cambiar orden de una lista	Puede afectar consumidores	Definir orden garantizado o no garantizado

Política sugerida: deprecar antes de retirar; publicar una fecha o condición de retiro; dar período de coexistencia; mantener pruebas de compatibilidad de clientes web y móviles y los cinco campos del contrato universal (success, message, data, errors, meta).

11. Integración con SDD / Spec Kit

Cada cambio comienza por especificar su intención. Antes de abrir un PR, el equipo vincula requisitos, reglas de negocio, arquitectura C4 (hasta código UML cuando aporte valor), contrato OpenAPI, tareas, implementación y pruebas. La gestión de ramas no sustituye el análisis: organiza el cambio controlado.

Secuencia de una funcionalidad

Fase	Artefacto / evidencia
1. Formular necesidad	Descripción textual y reglas de matrícula
2. Especificar	Historia, escenarios y criterios verificables en Spec Kit
3. Diseñar	BPMN aplicable, C4 C1-C4/UML, ADR, OpenAPI
4. Planificar	Tareas pequeñas y dependencias
5. Implementar	Branch / PR, código y pruebas
6. Verificar	CI, seguridad, contrato, regresión
7. Publicar	Tag, changelog y release notes
8. Mantener	Monitoreo, incidencias, rollback y correcciones

Prompts orientativos para el agente

```
Analiza la siguiente solicitud de cambio del modulo de matriculas.
Separa requisitos observables, reglas de negocio, criterios de
aceptacion y riesgos de compatibilidad. No generes codigo aun.
Identifica que contratos OpenAPI y diagramas C4 deben actualizarse.

Revisa la implementacion y sus pruebas respecto a spec.md,
plan.md y tasks.md. Identifica desviaciones de seguridad,
compatibilidad de API y contrato universal. Propone correcciones
minimas, sin cambios ajenos al alcance y sin aprobar tu propio trabajo.
```

12. Política de ramas, revisión y calidad

Una política completa especifica: convención de nombres, destino de PR, tamaño recomendado del cambio, revisores, controles CI, prohibición de secretos, criterios de merge, etiquetas de release y gestión de incidencias.

Regla	Criterio comprobable
Nombres de ramas	feature/xxx, fix/xxx, hotfix/xxx, release/x.y.z según flujo
Pull Request	Contexto, requisito, capturas cuando corresponda, riesgos y pruebas
Automatización	Tests, lint, seguridad, contratos y documentación
Revisión	Al menos una revisión independiente para cambios críticos
Integración	No fusionar con verificaciones obligatorias fallidas
Entrega	Tag firmado cuando sea viable, changelog y artefacto trazable

13. Caso guiado: evolución de API académica

Estado inicial: API v1, backend 1.0.0, GET /api/v1/matriculas con paginación y control por objeto.

Paso 1. Nueva funcionalidad compatible

Incorporar filtro de período opcional, actualizar spec/OpenAPI, pruebas, revisión y publicar 1.1.0.

Paso 2. Corrección de seguridad

Detectar acceso cruzado; corregir permiso por objeto, añadir prueba de regresión y publicar 1.1.1.

Paso 3. Cambio incompatible

Rediseñar estructura de respuestas; analizar consumidores, versión mayor, coexistencia y plan de migración.

Paso 4. Evidencia de entrega

Registrar PR, historias y criterios, cambios en C4/UML cuando correspondan, changelog, versión y despliegue.

14. Taller integrador

Reto: el equipo recibe tres solicitudes simultáneas: agregar búsqueda por período, solucionar un fallo crítico de permisos y modificar de forma incompatible el formato de un campo. Debe proponer el flujo de ramas, distribuir cambios, justificar SemVer, planificar releases y demostrar la coherencia de OpenAPI, código y pruebas.

Productos: ADR de selección de flujo, mapa de ramas, tabla de trazabilidad, política de commits, tres propuestas de PR, matriz de versiones, fragmento de CHANGELOG, plan de compatibilidad y verificación CI.

15. Rúbrica analítica (2 / 1 / 0)

Criterio	2	1	0
Selección de flujo	Justificada por contexto	Justificación parcial	Sin justificación
Ramas y PR	Reglas claras y aplicables	Incompletas	Incoherentes
SemVer	Clasifica correctamente	Errores menores	Incorrecto
Compatibilidad API	Gestiona evolución y clientes	Parcial	No considerada
Trazabilidad SDD	Requisitos → PR → release	Parcial	Ausente
Verificación y documentación	Evidencia CI y changelog	Evidencia parcial	Ausente

Máximo: 12 puntos. Se valoran evidencias verificables, no solo capturas o textos producidos por IA.

16. Lista final de comprobación

☐ La metodología responde al ciclo de releases y a la frecuencia de despliegue.

☐ Todas las ramas tienen rol, origen, destino y regla de eliminación.

☐ Los cambios críticos incluyen pruebas de seguridad y regresión.

☐ El producto sigue una política de SemVer explícita.

☐ Los contratos HTTP se versionan por compatibilidad, no por cada despliegue.

☐ Todos los releases poseen tag, changelog y trazabilidad.

☐ Spec Kit, BPMN, C4, UML, OpenAPI, Quarto y CI reflejan el cambio.

☐ Existe estrategia documentada de rollback y soporte de versiones anteriores.

17. Referencias oficiales y lecturas recomendadas

GitFlow (Atlassian): https://www.atlassian.com/git/tutorials/comparing-workflows/gitflow-workflow

GitHub Flow (GitHub): https://docs.github.com/en/get-started/using-github/github-flow

GitLab ramas y Merge Requests: https://docs.gitlab.com/user/project/repository/branches/

Trunk-Based Development (Atlassian): https://www.atlassian.com/continuous-delivery/continuous-integration/trunk-based-development

Semantic Versioning 2.0.0: https://semver.org/

Conventional Commits 1.0.0: https://www.conventionalcommits.org/en/v1.0.0/

GitHub Spec Kit: https://github.com/github/spec-kit

SERIE BACKEND PROFESIONAL · GUÍA DIDÁCTICA 21

GitHub Spec Kit: del requisito al código verificable

Una guía aplicada de instalación, inicialización, agentes de programación, especificaciones, artefactos, comandos y control de calidad para construir software con SDD.

Caso transversal: API de matrículas académicas con Django REST Framework, PostgreSQL, contrato universal de respuesta, GitFlow, C4 y documentación Quarto.

Audiencia: estudiantes universitarios de Computación. Conocimientos previos: Git, HTTP/REST, Python y fundamentos de SDD.

Aprendizaje esperado	Producto verificable
Preparar un proyecto con Spec Kit	Instalación comprobada y estructura inicial
Ejecutar un ciclo de trabajo con agente de IA	Constitución, especificación, plan, tareas y evidencias
Evaluar la correspondencia especificación-código	Pruebas, análisis y convergencia documentados

1. ¿Qué es GitHub Spec Kit?

GitHub Spec Kit es un conjunto de herramientas para organizar el desarrollo basado en especificaciones con agentes de programación. Proporciona una CLI para preparar el proyecto y habilidades/instrucciones para que un agente genere y verifique artefactos de desarrollo.

No reemplaza Git, un framework, un sistema de pruebas o una metodología de gestión. Es una estructura de trabajo que hace explícitas la intención, el diseño, las tareas y la comprobación de conformidad.

Diferencias que debes recordar

Concepto	Propósito
SDD	Enfoque: desarrollar a partir de especificaciones verificables.
Spec Kit	Herramienta y flujo para ejecutar SDD con un agente.
specify (CLI)	Inicializar, comprobar y consultar la instalación. Se usa en terminal.
/speckit-*	Habilidades/instrucciones que se ejecutan en el chat del agente, no en la terminal.
GitFlow	Convención de ramas que puede convivir con Spec Kit, pero no es obligatoria.

2. Requisitos e instalación

La documentación oficial describe Python 3.11+, un gestor como uv y un agente de programación compatible. Git es opcional en la configuración base; la extensión Git habilita flujos específicos de repositorio.

```
# Comprobar herramientas desde la terminal
python --version
uv --version

# Instalar desde PyPI
uv tool install specify-cli

# Validar instalación
specify version
specify check
```

Para proyectos de docencia o producción, conviene fijar una versión aprobada del paquete y documentarla en la bitácora del equipo. La sintaxis exacta puede evolucionar entre versiones.

Inicializar un proyecto nuevo

```
specify init matriculas-academicas --integration copilot
cd matriculas-academicas
# Abrir esta carpeta en el editor e iniciar el agente
```

Inicializar en un repositorio existente

```
# En el directorio del repositorio existente:
specify init --here --integration copilot
# Revisar los archivos nuevos antes de confirmarlos en Git.
```

Precaución: no utilizar opciones de sobrescritura forzada sobre un repositorio con trabajo sin respaldar. Seguir la guía oficial de adopción para bases de código existentes.

3. Estructura y artefactos del proyecto

La ubicación concreta puede cambiar según la versión y la integración seleccionada. Es más importante reconocer el propósito que memorizar una ruta fija.

```
matriculas-academicas/
  .specify/
    memory/                # constitución del proyecto
    templates/             # plantillas del flujo
    scripts/               # auxiliares generados
    feature.json           # característica activa (según versión)
  specs/
    <feature>/
      spec.md              # comportamiento esperado
      plan.md              # diseño técnico
      tasks.md             # secuencia de trabajo
      ...                  # investigación, modelos, contratos
  src/                     # implementación (según proyecto)
  tests/                   # pruebas automatizadas
  docs/                    # C4, ADR, Quarto y guías
```

El repositorio sigue siendo la fuente versionada. Los artefactos generados requieren revisión; nunca se presumen correctos solo por haberlos producido un agente.

4. Flujo completo recomendado

Orden	Comando de agente (modo skills)	Qué produce o controla
0	/speckit-constitution	Principios no negociables del proyecto (una vez al inicio).
1	/speckit-specify	Especificación de la funcionalidad; qué y por qué.
2	/speckit-clarify	Ambigüedades resueltas y supuestos documentados.
3	/speckit-plan	Arquitectura, stack, modelos y contratos.
4	/speckit-checklist	Lista de verificación de calidad de la especificación.
5	/speckit-tasks	Tareas ejecutables y sus dependencias.
6	/speckit-analyze	Coherencia entre artefactos antes de programar.
7	/speckit-implement	Implementación incremental y ejecución de tareas.
8	/speckit-converge	Compara código y especificación; detecta tareas pendientes.

La forma de invocación depende del agente. Algunos muestran comandos con puntos (/speckit.specify), guiones (/speckit-specify) o prefijos especiales. La tabla utiliza la presentación skills empleada en el Quickstart oficial para Copilot; confirma las instrucciones que ofrece tu integración.

5. Constitución: las reglas de todo el backend

La constitución fija principios que deben cumplirse en todas las funcionalidades. Es mejor escribir reglas comprobables en vez de declaraciones genéricas como «código limpio».

```
/speckit-constitution Define una constitución para un backend
institucional: monolito modular Django/DRF y PostgreSQL;
contrato universal success/message/data/errors/meta;
autorización por objeto, validaciones y pruebas obligatorias;
```

```
OpenAPI, C4 y ADR actualizados; GitFlow y CI; ningún secreto
en el repositorio; auditoría y request_id para operaciones críticas.
```

Control docente: solicitar al grupo que explique por qué cada regla existe y qué prueba o evidencia demostraría su cumplimiento.

6. Especificación: del problema a los criterios

Al especificar, evitar fijar tecnologías prematuramente: describir actores, necesidades, reglas, condiciones y resultados. La elección de herramientas corresponde al plan.

```
/speckit-specify Implementar la matrícula de un estudiante
para una oferta académica. Debe validar período habilitado,
prerrequisitos, disponibilidad de cupos, ausencia de matrícula
duplicada y autorización. Documentar casos felices, errores,
reintentos y concurrencia por el último cupo. Incluir criterios
de aceptación verificables y escenarios negativos.
```

Ejemplo de criterios de aceptación

Código	Condición comprobable
CA-01	Si hay cupo, período abierto y requisitos válidos, la API crea la matrícula.
CA-02	No pueden existir dos matrículas activas del mismo estudiante en la misma oferta.
CA-03	Un estudiante no puede consultar ni modificar matrículas ajenas.
CA-04	Si dos estudiantes compiten por el último cupo, como máximo uno lo obtiene.

7. Clarificar antes de diseñar

Preguntas comunes: ¿matrícula inmediata o pendiente de aprobación?, ¿se permite anular?, ¿qué sucede ante un reintento?, ¿se reserva el cupo?, ¿el administrador puede crear matrículas en nombre de otro usuario?

```
/speckit-clarify Revisa las ambigüedades de matrícula:
reglas de duplicidad, estados permitidos, cupos y permisos.
No supongas políticas institucionales sin confirmación;
marca decisiones pendientes explícitamente.
```

Las decisiones deben quedar reflejadas en la especificación y revisarse con responsables del negocio. Sin esto, la IA podría transformar supuestos en requisitos inexistentes.

8. Planificar la solución

Con requisitos claros, incorporar arquitectura C4, contratos HTTP, seguridad, transacciones y operaciones. En este punto sí corresponde declarar el stack.

```
/speckit-plan Implementar con Django REST Framework,
PostgreSQL y arquitectura modular por capas. Definir modelos,
servicios de aplicación, dominio y repositorios cuando aporten
valor. Crear OpenAPI /api/v1, envelope universal, permisos,
transacciones seguras y pruebas concurrentes. Documentar C4,
ADR y estrategia de trazabilidad de solicitudes.
```

Decisiones que deben justificarse

Decisión	Razón / evidencia
Monolito modular	Reduce complejidad operativa y permite evolución por módulos.
PostgreSQL	Restricciones transaccionales y concurrencia verificable.
select_for_update	Bloqueo explícito cuando corresponda en operación de cupos.
Contrato universal	Consumo uniforme por frontend, móvil e integraciones.
HTTP 409	Conflictos de negocio como matrícula duplicada o cupo agotado.

9. Checklists, tareas y análisis

Las listas de comprobación valoran la calidad de requisitos: si cada historia tiene pruebas, si se describen permisos, si los estados y excepciones están resueltos. No deben confundirse con la ejecución de las pruebas.

```
/speckit-checklist Genera un checklist para requisitos
funcionales, seguridad, datos, concurrencia, errores y API.

/speckit-tasks Divide el trabajo en tareas pequeñas con
identificadores, dependencias y evidencias. Prioriza una
implementación vertical verificable.

/speckit-analyze Busca contradicciones entre constitución,
spec, plan y tareas. Señala requisitos sin cobertura.
```

Fragmento didáctico de tasks.md

```
T001 Crear tablas Oferta y Matricula con constraints.
T002 Implementar validar_prerrequisitos().
T003 Crear servicio registrar_matricula() transaccional.
T004 Agregar permisos por objeto a endpoints.
T005 Aplicar envelope universal y errores estructurados.
T006 Probar concurrencia del último cupo con PostgreSQL.
T007 Actualizar OpenAPI, C4, ADR y guía Quarto.
T008 Ejecutar CI y resolver divergencias.
```

10. Implementar y converger

Implementar por lotes pequeños. El agente debe ejecutar pruebas y reportar qué pudo comprobar. No aprobar cambios automáticamente cuando falten dependencias, permisos o accesos a servicios.

```
/speckit-implement Implementa las tareas aprobadas,
respeta la constitución, agrega pruebas y reporta los
archivos cambiados, comandos ejecutados y fallos pendientes.

/speckit-converge Compara implementación con requisitos,
plan, tareas y pruebas; registra brechas verificables y
crea tareas faltantes si son necesarias.
```

Si convergencia encuentra defectos, resolverlos y volver a ejecutar el análisis. La palabra «converged» es una señal del flujo, no una garantía independiente de que el sistema sea seguro o apto para producción.

11. Ejemplo de contrato universal

```
HTTP/1.1 409 Conflict
Content-Type: application/json

{
  "success": false,
  "message": "No existen cupos disponibles.",
  "data": null,
  "errors": {
    "code": "CUPOS_AGOTADOS",
    "details": [{"field":"oferta_id",
                 "message":"La oferta no tiene cupos."}]
  },
  "meta": {
    "request_id": "req_ejemplo",
    "timestamp": "2026-10-13T18:00:00Z",
    "version": "v1"
  }
}
```

Los endpoints JSON usan la envoltura acordada, sin desvirtuar los códigos HTTP. Las respuestas 204/304 y HEAD son excepciones legítimas porque no incluyen cuerpo.

12. Gestión de ramas y trabajo colaborativo

Spec Kit no obliga a usar GitFlow. En proyectos docentes que sí lo adopten, el equipo puede crear feature/matriculas-cupos desde develop, llevar en esa rama especificación/código/pruebas y abrir un pull request con evidencia.

Según la documentación actual, el núcleo de Spec Kit no exige Git; la extensión Git integra funciones de control de versiones. Cambiar de rama por sí solo no garantiza cambiar la característica activa registrada por la herramienta.

```
# Ejemplo externo a Spec Kit: GitFlow del equipo
git switch develop
git switch -c feature/matriculas-cupos
# Implementar y revisar
pytest -q
git add .
git commit -m "feat: validar cupos de matrícula"
```

13. Integración con los artefactos de la serie

Documento técnico	Papel en el ciclo Spec Kit
BPMN ejecutivo/operativo/técnico	Representa el proceso antes de desarrollar y permite rastrear actividades.
C4 C1-C4	Expone contexto, despliegue lógico, componentes y clases importantes.
OpenAPI	Define y verifica contratos entre clientes y API.
ADR	Explica decisiones, alternativas y consecuencias.
Pruebas automatizadas	Verifican criterios de aceptación y previenen regresión.
GitFlow y CI/CD	Controlan revisión, integración y despliegues.
Quarto	Publica especificaciones, arquitectura y guía de operación.

14. Laboratorio: módulo de matrícula académica

Paso 1. Preparar repositorio, instalar Spec Kit e inicializar la integración elegida.

Paso 2. Redactar una constitución técnica verificable que incorpore contrato universal y seguridad.

Paso 3. Especificar matrícula, validación de cupos, cancelación y consulta por propietario.

Paso 4. Resolver ambigüedades de estados, permisos, duplicidad y concurrencia.

Paso 5. Planificar arquitectura por capas, C4, modelo relacional y endpoints OpenAPI.

Paso 6. Crear checklist, tareas y analizar consistencia entre los documentos.

Paso 7. Implementar endpoints, reglas, transacciones, permisos y respuestas de error.

Paso 8. Ejecutar pytest en PostgreSQL, documentar resultados y errores encontrados.

Paso 9. Ejecutar convergencia; actualizar requisitos y código cuando existan discrepancias.

Paso 10. Publicar sitio Quarto y abrir PR con evidencias de CI y revisión de pares.

15. Matriz de verificación y evaluación

Criterio	Evidencia requerida
Configuración reproducible	Versión CLI, agente integrado, instrucciones para instalar.
Especificación verificable	Historias, escenarios, reglas, estados y criterios comprobables.
Plan coherente	Arquitectura, OpenAPI y ADR consistentes con las restricciones.
Trazabilidad	Requisitos vinculados a tareas y pruebas.
Calidad del código	Pruebas, permisos, control transaccional, contrato universal.
Convergencia real	Registro de brechas, correcciones y verificación final.

Rúbrica académica (2 / 1 / 0)

Calificar los seis criterios anteriores con 2 si son completos y reproducibles, 1 si presentan omisiones parciales y 0 si están ausentes o no se pueden comprobar. Máximo: 12 puntos. La valoración final debe basarse en evidencia, no en cantidad de prompts.

16. Problemas frecuentes y soluciones

Problema	Diagnóstico / solución
Comando no reconocido	Verificar integración del agente y la sintaxis mostrada por su interfaz.
specify no disponible	Verificar PATH e instalación con specify version.
Funcionalidad equivocada	Inspeccionar estado de característica activa y directorio .specify.
Inconsistencias spec/código	Ejecutar análisis y convergencia; corregir artefactos y revalidar.
IA inventa reglas	Volver a clarificar supuestos y solicitar aprobación del responsable.
Pruebas de concurrencia engañosas	Usar PostgreSQL real y casos simultáneos repetibles.

17. Autoevaluación

- ¿Qué se ejecuta en terminal y qué se ejecuta en el chat del agente?
- ¿En qué se diferencia especificar de planificar?
- ¿Cuándo deben ejecutarse clarificación y análisis de consistencia?
- ¿Cómo compruebas que una regla de negocio tiene cobertura de pruebas?
- ¿Por qué convergencia no sustituye una revisión de seguridad independiente?

18. Fuentes oficiales y lecturas sugeridas

- https://github.com/github/spec-kit
- https://github.com/github/spec-kit/blob/main/docs/quickstart.md
- https://github.com/github/spec-kit/blob/main/docs/reference/core.md
- https://github.com/github/spec-kit/blob/main/docs/reference/agentic-sdd.md
- https://github.github.com/spec-kit/installation.html
- https://github.github.com/spec-kit/guides/existing-projects.html

Nota de vigencia: sintaxis y archivos pueden cambiar según la versión de Spec Kit y la integración del agente. Consultar las instrucciones efectivas del proyecto al ejecutar el laboratorio.